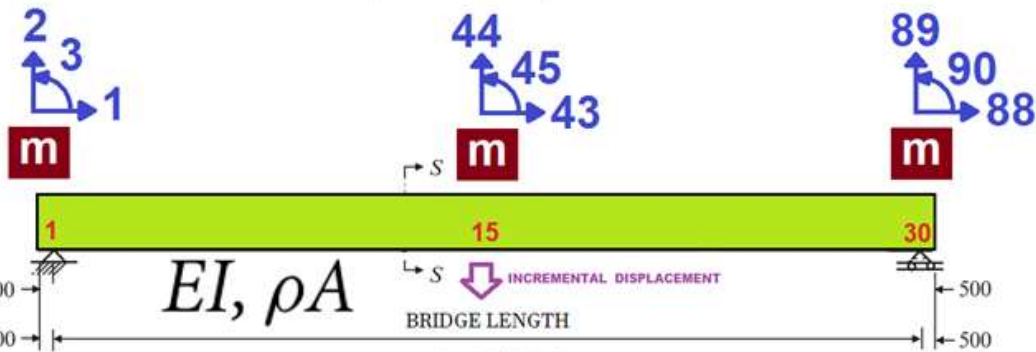
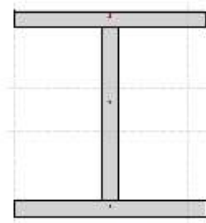


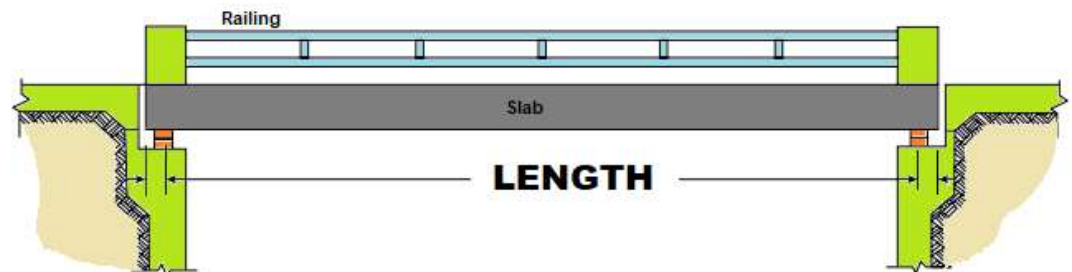
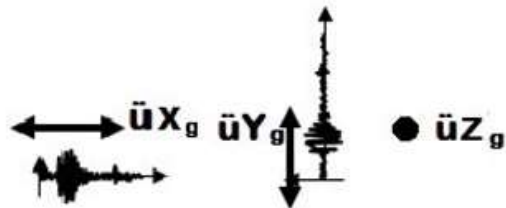
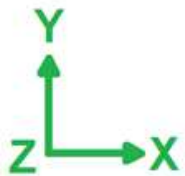
>> IN THE NAME OF ALLAH, THE MOST GRACIOUS, THE MOST MERCIFUL <<

COMPREHENSIVE NONLINEAR SEISMIC ASSESSMENT OF A PEDESTRIAN BRIDGE STEEL VIERENDEEL GIRDERS : AN OPENSEES FRAMEWORK FOR STATIC PUSHOVER, CYCLIC DEGRADATION, STATIC TIME-HISTORY AND DYNAMIC TIME-HISTORY ANALYSIS

THIS PROGRAM IS WRITTEN BY SALAR DELAVAR GHASHGHAEI (QASHQAI)



$$P(t) = P_0 e^{-0.05\bar{\omega}t} \sin(\bar{\omega}t)$$



$$\text{Structure Ductility Damage Index} = \frac{\Delta_d - \Delta_y}{\Delta_u - \Delta_y}$$

Δ_d = Lateral Displacement from Dynamic Analysis

Δ_y = Lateral Yield Displacement from Pushover Analysis

Δ_u = Lateral Ultimate Displacement from Pushover Analysis

$$P(t) = P_0 e^{-0.05\bar{\omega}t} \sin(\bar{\omega}t)$$

$$m\ddot{u} + c\dot{u} + ku = P(t)$$

$$\text{Structure Ductility Damage Index} = \frac{\Delta_d - \Delta_y}{\Delta_u - \Delta_y}$$

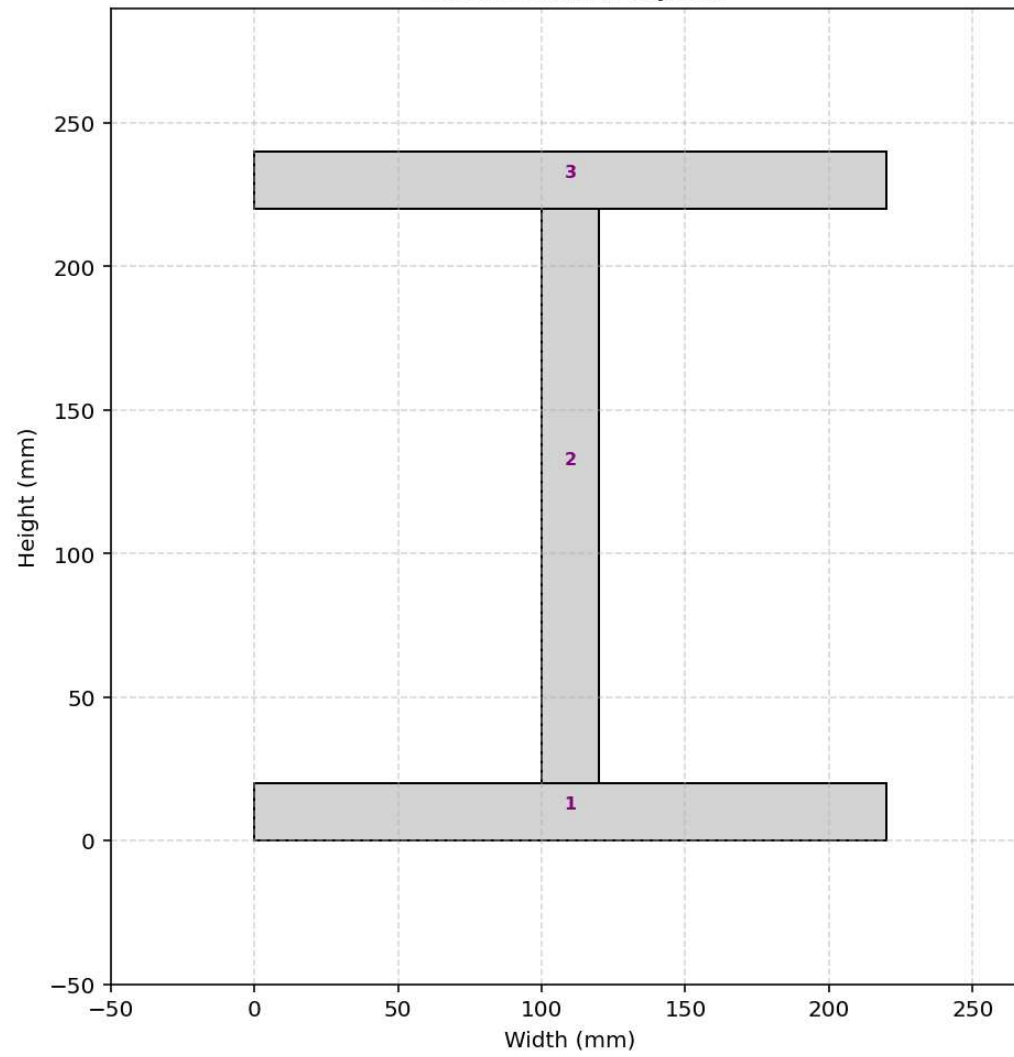
$$\rho A \frac{\partial^2 v(x,t)}{\partial t^2} + EI \frac{\partial^4 v(x,t)}{\partial x^4} = P(t)$$

boundary conditions

$$v(0,t) = v(L,t) = 0, \quad \left. \frac{\partial^2 v(x,t)}{\partial x^2} \right|_{x=0} = \left. \frac{\partial^2 v(x,t)}{\partial x^2} \right|_{x=L} = 0.$$



Steel section (3 layers)



12345678910111213141516171819202122232425262728293031323334

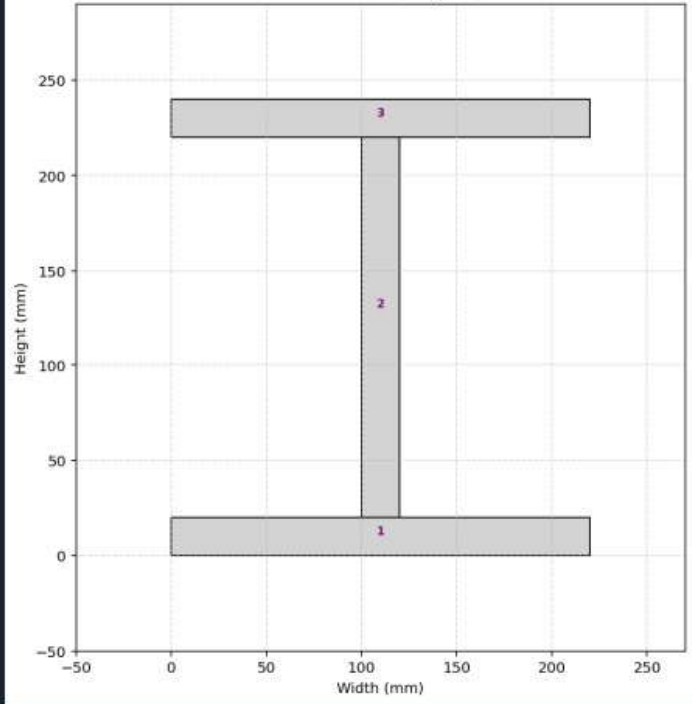
```
#####
#           >> IN THE NAME OF ALLAH, THE MOST GRACIOUS, THE MOST MERCIFUL <<
#   COMPREHENSIVE NONLINEAR SEISMIC ASSESSMENT OF A PEDESTRIAN BRIDGE STEEL VIERENDEEL GIR
#   AN OPENSEES FRAMEWORK FOR STATIC PUSHOVER, CYCLIC DEGRADATION,
#   STATIC TIME-HISTORY AND DYNAMIC TIME-HISTORY ANALYSIS
#-----
#
#           P(t) = P0 sin(wt)
#           P(t) = P0 exp(-0.05wt) sin(wt)
#-----
#   ASSESSMENT OF DUCTILITY DAMAGE INDICES FOR STRUCTURAL ELEMENTS AND SYSTEMS AND E
#   OF ENERGY DISSIPATION CAPACITY INDEX
#-----
#           THIS PYTHON SCRIPT WRITTEN BY SALAR DELAVAR GHASHGHAEE (QASHQAI)
#           EMAIL: salar.d.ghashghaei@gmail.com
#####
"""
Nonlinear Seismic Performance Assessment of a pedestrian bridge:
An OpenSeesPy Framework for Material and Geometric Nonlinearity Under Static, Cyclic, and

This OpenSeesPy script performs rigorous nonlinear static and dynamic analysis of a pedest
for performance-based earthquake engineering. The 2D model incorporates both material non
(elastic-perfectly plastic or hysteretic steel with strain hardening)
and geometric nonlinearity to capture P-delta effects
and large displacements-essential for collapse assessment.

Eight analysis protocols are implemented:
(1) [PERIOD] : Structural Period
(2) [STATIC] : Gravity load analysis establishing dead load state
(3) [PUSHOVER] : Displacement-controlled pushover generating full capacity curves
and plastic mechanism identification
(4) [CYCLIC_DISPLACEMENT] : Symmetric cyclic displacement protocol capturing hysteresis,
pinching behavior, and energy dissipation degradation
(5) [STATIC_EXTERNAL_TIME-DEPENDENT_LOADING] : Static Analysis of External time-dependent
(6) [DYNAMIC_EXTERNAL_TIME-DEPENDENT_LOADING] : Dynamic Analysis of External time-dependen
```

47 %

Steel section (3 layers)



IPython Console Files Help Variable Explorer Debugger Plots History

Save file

Inline Conda: anaconda3 (Python 3.12.7) ✓ LSP: Python Line 91, Col 2 UTF-8 CRLF RW Mem 48%

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY...TEEL_PLATE_SECTION_DIFF_WIDTH_HEIGHT_FUN_EXTRA.py

P(t)_PEDESTRIAN_BR...L_GIRDERS_8_ANA.py X I_STEEL_PLATE_SECT...EIGHT_FUN_EXTRA.py X

```
1 def I_STEEL_PLATE_SECTION_DIFF_WIDTH_HEIGHT_FUN_EXTRA(secTag, STEEL_TYPE, STEEL_DENSITY, pl
2 """
3 Create a steel with different widths and heights, fiber section (OpenSees) and,
4 if PLOT=True, draw a simple 2-D sketch of the section.
5
6 Parameters
7 -----
8 secTag      : int - section identifier
9 STEEL_TYPE   : str - 'ELASTIC' or 'INELASTIC'
10 STEEL_DENSITY: float - pc (kg/m³) - will be converted to N·s²/mm³
11 PLOT        : bool - draw the section if True
12
13 THIS PYTHON SCRIPT WRITTEN BY SALAR DELAVAR GHASHGHAEI (QASHQAI)
14 """
15 import numpy as np
16 import openseespy.opensees as ops
17 import matplotlib.pyplot as plt
18 import matplotlib.pyplot as plt, matplotlib.patches as patches
19
20 # Define materials for nonlinear elements
21 # Define parameters (units: mm, N)
22 # -----
23 # STEEL
24 fy = 240          # [N/mm²] Steel Yield Strength
25 Es = 2e5          # [N/mm²] Modulus of Elasticity
26 ey = fy/Es        # [mm/mm] Steel Yield Strain
27 fu = 1.1818*fy    # [N/mm²] Steel Ultimate Strength
28 esu = 0.15        # [mm/mm] Steel Ultimate Strain
29 Esh = (fu - fy)/(esu - ey)
30 Bs = Esh / Es
31
32 steelTag = secTag + 100
33 if STEEL_TYPE == 'ELASTIC':
34     ops.uniaxialMaterial('Elastic', steelTag, Es)
```

Steel section (3 layers)

IPython Console Files Help Variable Explorer Debugger Plots History

Inline Conda: anaconda3 (Python 3.12.7) ✓ LSP: Python Line 5, Col 1 UTF-8 CRLF RW Mem 48%

STATIC ANALYSIS

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY...(t)_PEDESTRIAN_BRIDGE_VIERENDEEL_GIRDERS_8_ANA.py

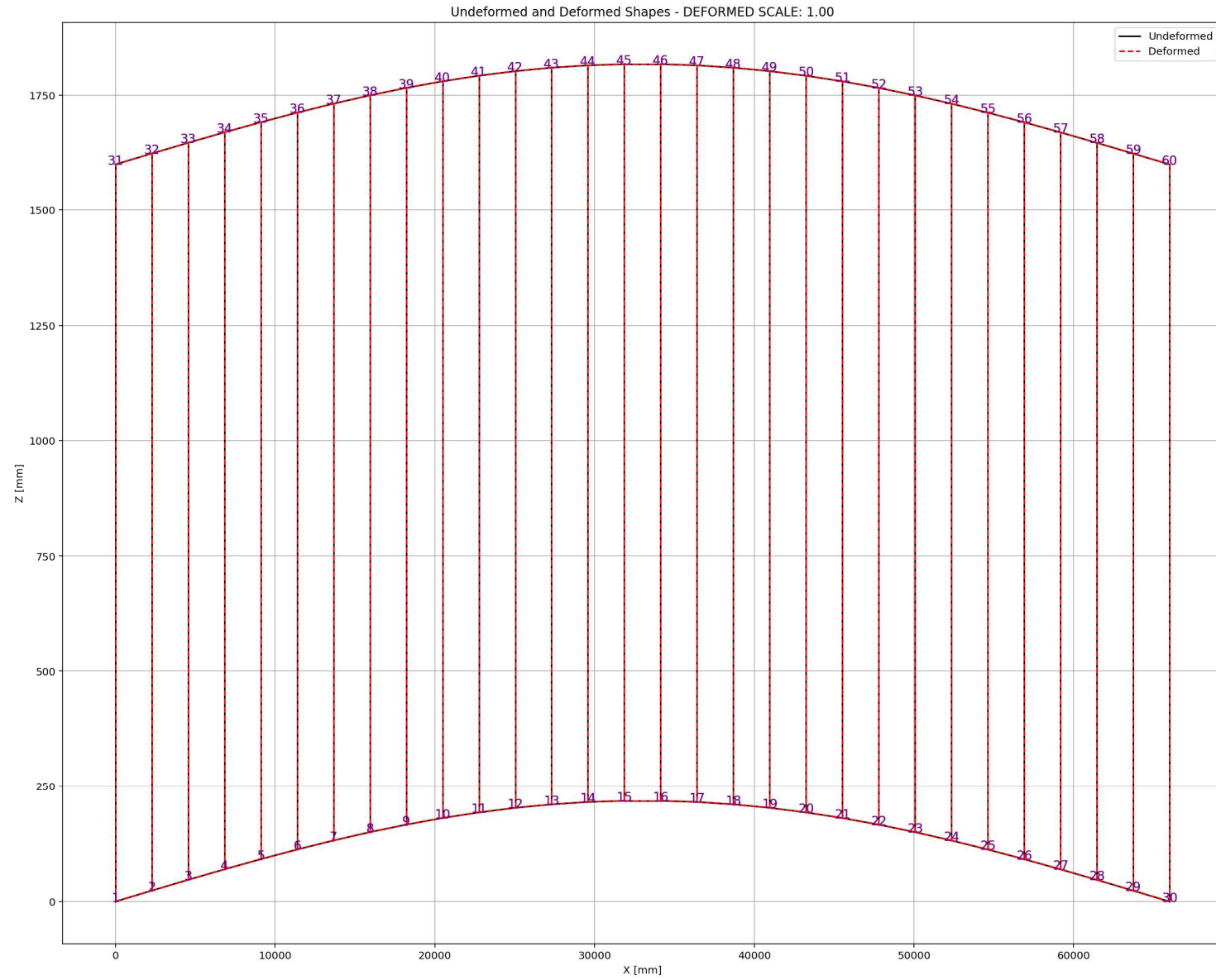
P(t)_PEDESTRIAN_BR...I_GIRDERS_8_ANA.py x I_STEEL_PLATE_SECT...EIGHT_FUN_EXTRA.py x

```
1 #####
2 # >> IN THE NAME OF ALLAH, THE MOST GRACIOUS, THE MOST MERCIFUL <<
3 # COMPREHENSIVE NONLINEAR SEISMIC ASSESSMENT OF A PEDESTRIAN BRIDGE STEEL VIERENDEEL GIR
4 # AN OPENSEES FRAMEWORK FOR STATIC PUSHOVER, CYCLIC DEGRADATION,
5 # STATIC TIME-HISTORY AND DYNAMIC TIME-HISTORY ANALYSIS
6 #-----
7 # P(t) = P0 sin(wt)
8 # P(t) = P0 exp(-0.05wt) sin(wt)
9 #-----
10 # ASSESSMENT OF DUCTILITY DAMAGE INDICES FOR STRUCTURAL ELEMENTS AND SYSTEMS AND E
11 # OF ENERGY DISSIPATION CAPACITY INDEX
12 #-----
13 # THIS PYTHON SCRIPT WRITTEN BY SALAR DELAVAR GHASHGHAEI (QASHQAI)
14 # EMAIL: salar.d.ghashghaei@gmail.com
15 #####
16 """
17 Nonlinear Seismic Performance Assessment of a pedestrain bridge:
18 An OpenSeesPy Framework for Material and Geometric Nonlinearity Under Static, Cyclic, and
19
20 This OpenSeesPy script performs rigorous nonlinear static and dynamic analysis of a pedest
21 for performance-based earthquake engineering. The 2D model incorporates both material non
22 (elastic-perfectly plastic or hysteretic steel with strain hardening)
23 and geometric nonlinearity to capture P-delta effects
24 and large displacements-essential for collapse assessment.
25
26 Eight analysis protocols are implemented:
27 (1) [PERIOD] : Structural Period
28 (2) [STATIC] : Gravity load analysis establishing dead load state
29 (3) [PUSHOVER] : Displacement-controlled pushover generating full capacity curves
30 and plastic mechanism identification
31 (4) [CYCLIC_DISPLACEMENT] : Symmetric cyclic displacement protocol capturing hysteresis,
32 pinching behavior, and energy dissipation degradation
33 (5) [STATIC_EXTERNAL_TIME-DEPENDENT_LOADING] : Static Analysis of External time-dependent
34 (6) [DYNAMIC_EXTERNAL_TIME-DEPENDENT_LOADING] : Dynamic Analysis of External time-dependen
```

Untransformed and Deformed Shapes - ENVELOPED SCALE: 1.00

IPython Console Files Help Variable Explorer Debugger Plots History

Inline Conda: anaconda3 (Python 3.12.7) ✓ LSP: Python Line 91, Col 1 UTF-8 CRLF RW Mem 44%



PUSHOVER ANALYSIS

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY...(t)_PEDESTRIAN_BRIDGE_VIERENDEEL_GIRDERS_8_ANA.py

P(t)_PEDESTRIAN_BR...I_GIRDERS_8_ANA.py X I_STEEL_PLATE_SECT...EIGHT_FUN_EXTRA.py X

```
285
286 if ANAL_TYPE == 'PUSHOVER': # STATIC TIME-HISTORY ANALYSIS
287     IDctrlDOF = 2 # 1: Horizontal Displcement - 2: Vertical Displcement
288     DINCR = -0.5 # [mm] Incremental Vertical Displacement
289     DMAX = -1200.0 # [mm] Max. Vertical Displacement
290     ops.integrator('DisplacementControl', center_node, IDctrlDOF, DINCR)
291     ops.analysis('Static')
292     Nsteps = int(np.abs(DMAX/ DINCR))
293     STEP = 0.0
294     for step in range(Nsteps):
295         OK = ops.analyze(1)
296         S02.ANALYSIS(OK, 1, MAX_TOLERANCE, MAX_ITERATIONS)
297         ops.reactions()
298         reaction.append(ops.nodeReaction(1, 2)+ops.nodeReaction(30, 2)) # SHEAR BASE R
299         disp_mid.append(ops.nodeDisp(center_node, 2)) # DISPLACEMENT
300         dispX.append(ops.nodeDisp(center_node, 1)) # DISPLACEMENT
301         dispY.append(ops.nodeDisp(center_node, 2)) # DISPLACEMENT
302         # Store elements force, strain and stress
303         for ele_id in ele_axialforce.keys():
304             ele_axialforce[ele_id].append(ops.eleResponse(ele_id, 'force')[0])
305             ele_shearforce[ele_id].append(ops.eleResponse(ele_id, 'force')[1])
306             ele_momentforce[ele_id].append(ops.eleResponse(ele_id, 'force')[2])
307         # Store displacements
308         for node_id in node_displacementsX.keys():
309             node_displacementsX[node_id].append(ops.nodeDisp(node_id, 1))
310             node_displacementsY[node_id].append(ops.nodeDisp(node_id, 2))
311         # IN EACH STEP, STRUCTURE PERIOD GOING TO BE CALCULATED
312         #PERIODmin, PERIODmax = S06.RAYLEIGH_DAMPING(3, 0.5*DR, DR, 0, 1)
313         PERIODmin, PERIODmax = S05.EIGENVALUE_ANALYSIS(3, PLOT=True)
314         PERIOD_MIN.append(PERIODmin)
315         PERIOD_MAX.append(PERIODmax)
316         # SECTION STRESS-STRAIN FOR BOTTOM FIBER
317         stressB, strainB = S10.SECTION_STRESS_STRAIN(center_node-1, SEC_TAG, -Depth*0.
318         # Store in separate lists
```

Base Reaction and Displacement of Structure During Pushover Analysis

200000
150000
100000
50000
0

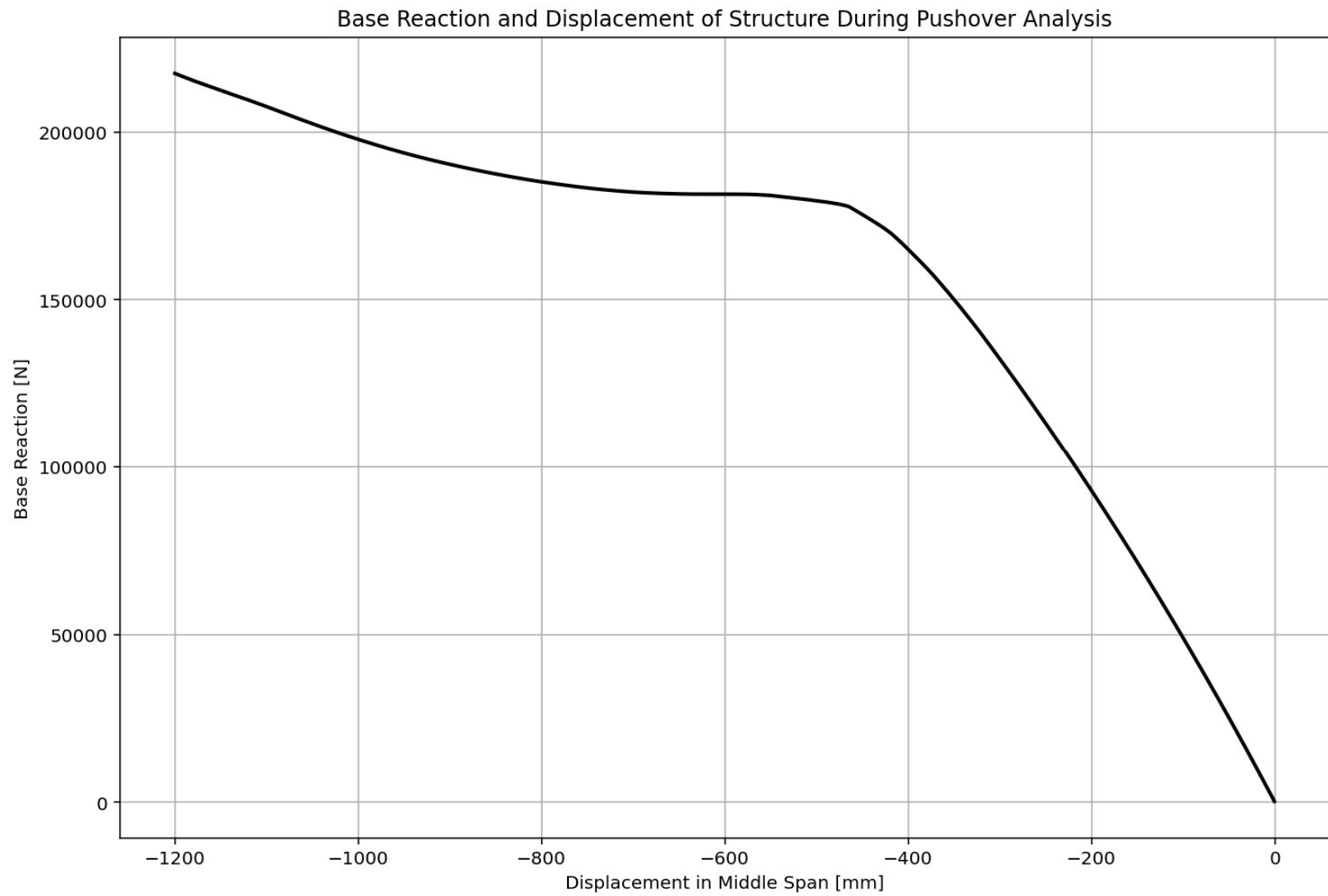
-1200 -1000 -800 -600 -400 -200 0

Displacement in Middle Span (mm)

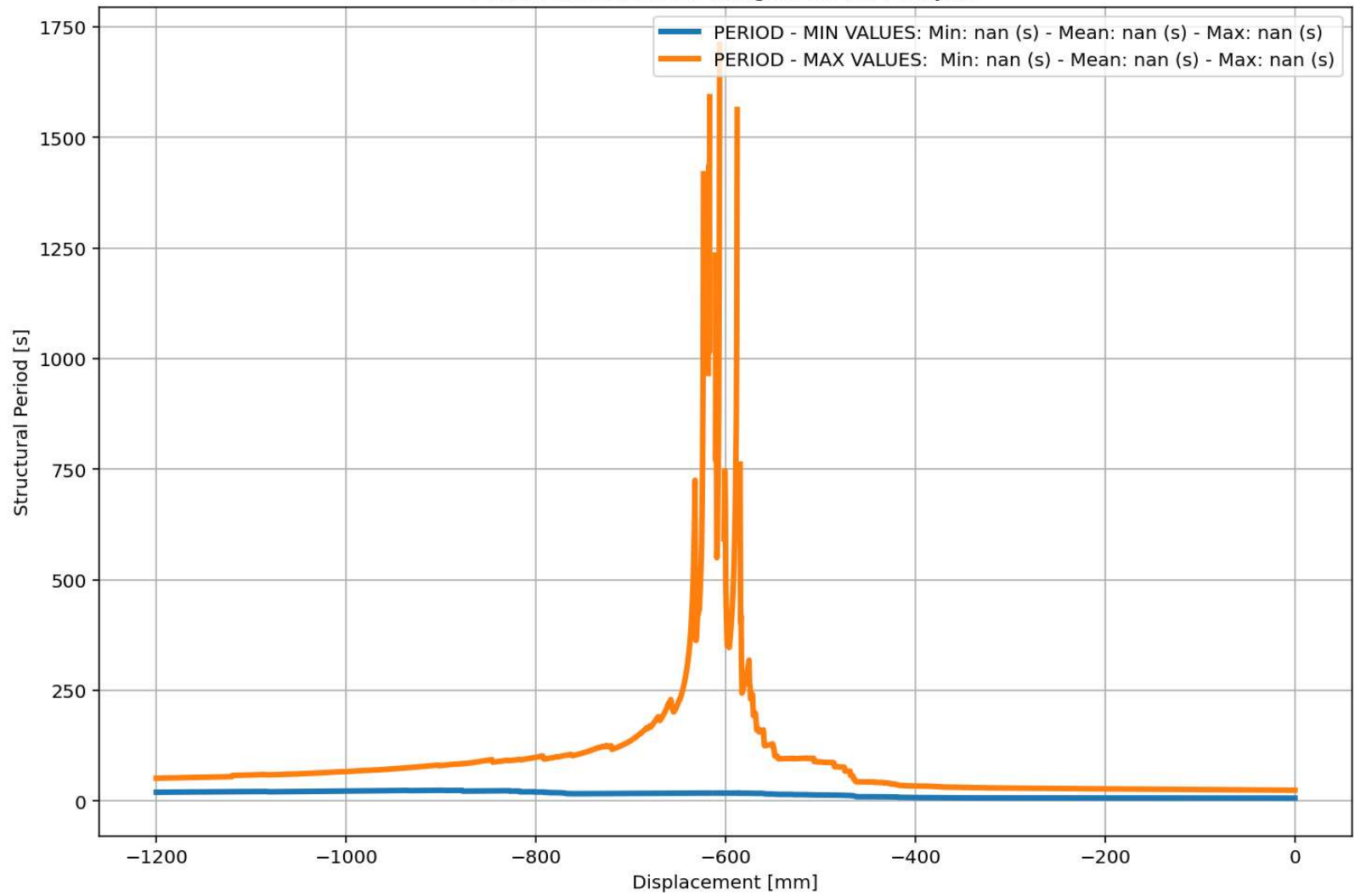
Base Reaction (N)

Python Console Files Help Variable Explorer Debugger Plots History

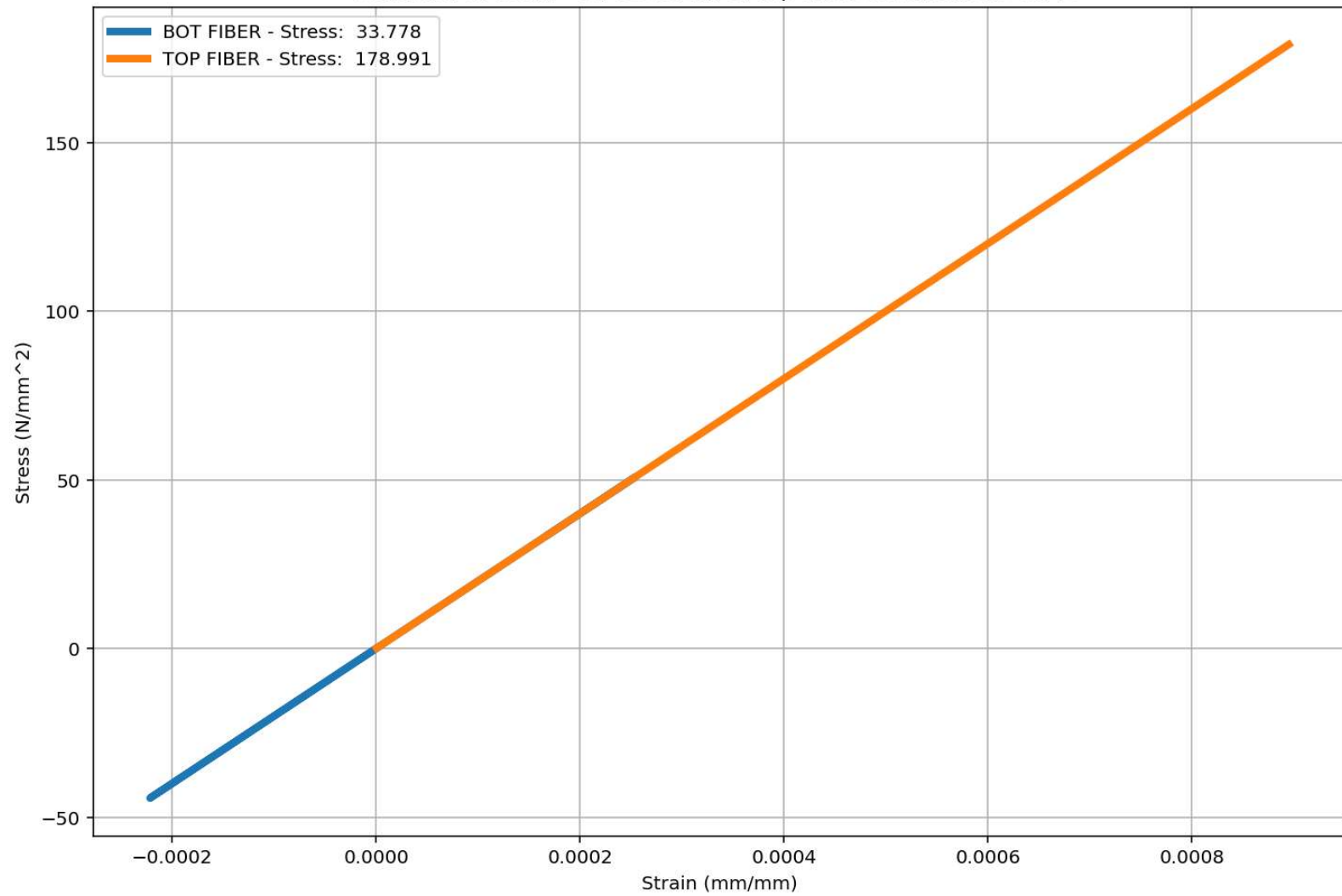
Inline Conda: anaconda3 (Python 3.12.7) ✓ LSP: Python Line 91, Col 1 UTF-8 CRLF RW Mem 46%

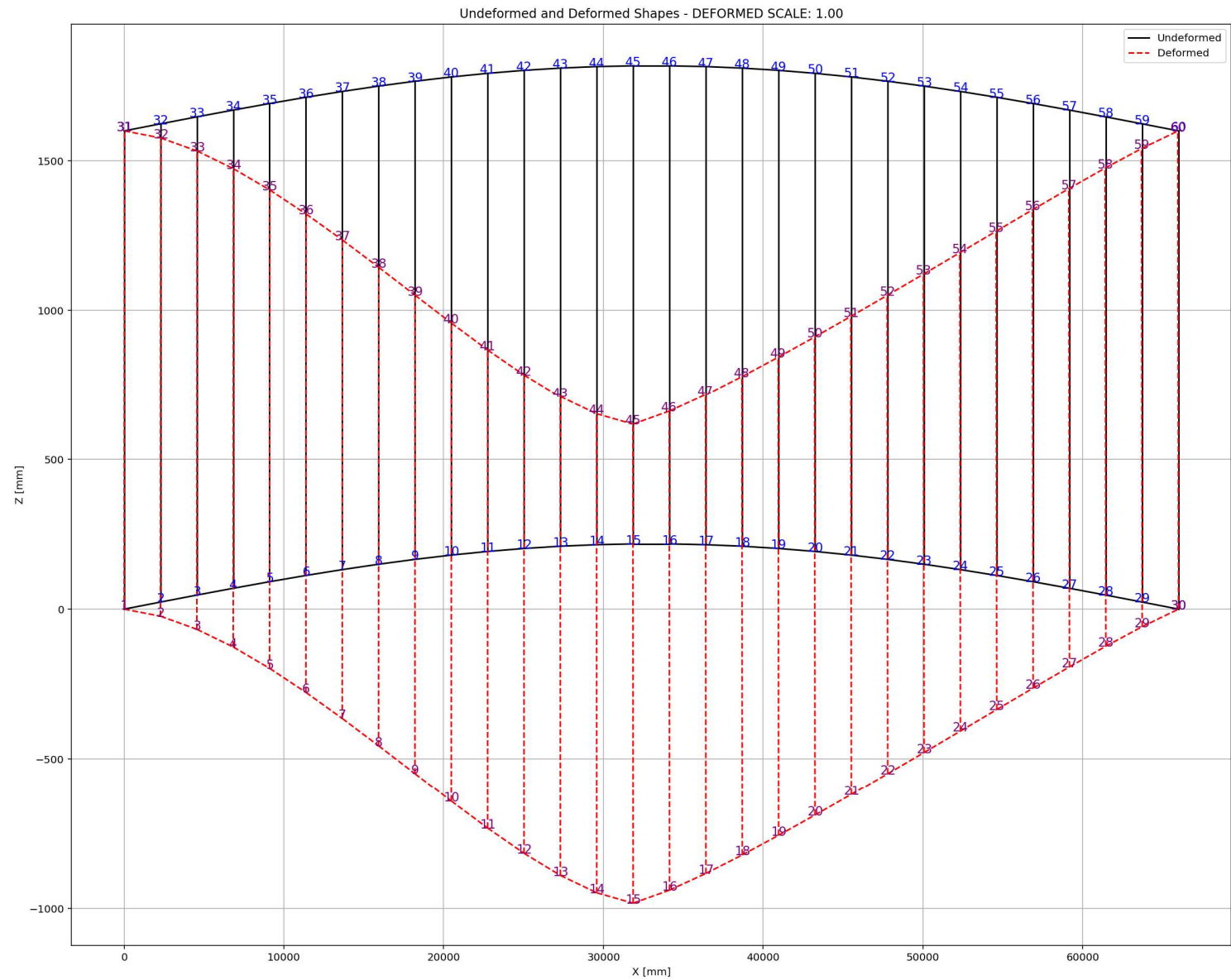


Period of Structure During Pushover Analysis



Behavior of beam - Stress-strain of Top and Bottom Fibers Curve





CYCLIC DISPLACEMENT ANALYSIS

Spyder (Python 3.12)

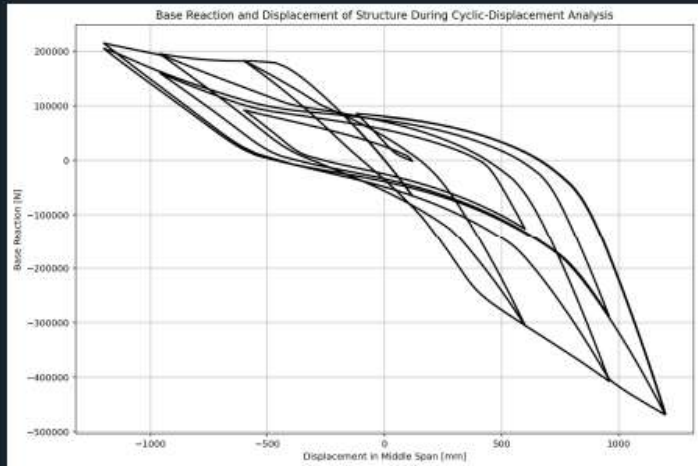
File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY...(t)_PEDESTRIAN_BRIDGE_VIERENDEEL_GIRDERS_8_ANA.py

P(t)_PEDESTRIAN_BR...I_GIRDERS_8_ANA.py I_STEEL_PLATE_SECT...EIGHT_FUN_EXTRA.py

```
346
347 if ANAL_TYPE == 'CYCLIC_DISPLACEMENT': # STATIC TIME-HISTORY ANALYSIS
348     IDctrlDOF = 2 # 1: Horizontal Displacement - 2: Vertical Displacement
349     DMAX = -1200.0 # [mm] Max. Vertical Displacement
350     n_points = 10000
351     # 4. CYCLIC DISPLACEMENT PROTOCOL
352     # Key strain points (same logic as your protocol)
353     key_disp = np.array([
354         0.0,
355         0.1*DMAX, -0.1*DMAX,
356         0.5*DMAX, -0.5*DMAX,
357         0.8*DMAX, -0.8*DMAX,
358         DMAX, -DMAX,
359         0.1*DMAX, -0.1*DMAX,
360         0.5*DMAX, -0.5*DMAX,
361         0.8*DMAX, -0.8*DMAX,
362         DMAX, -DMAX,
363         0.0
364     ])
365
366     # Generate 10000-point displacement protocol
367     disp_protocol = np.interp(
368         np.linspace(0, len(key_disp) - 1, n_points),
369         np.arange(len(key_disp)),
370         key_disp
371     )
372
373     ops.analysis('Static')
374     STEP = 0.0
375     for target_disp in disp_protocol:
376         current_disp = ops.nodeDisp(center_node, 2) # DISPALCEMENT APPLIED IN MIDDLE N
377         dU = target_disp - current_disp
378         ops.integrator('DisplacementControl', center_node, IDctrlDOF, dU)
379         OK = ops.analyze(1)
```

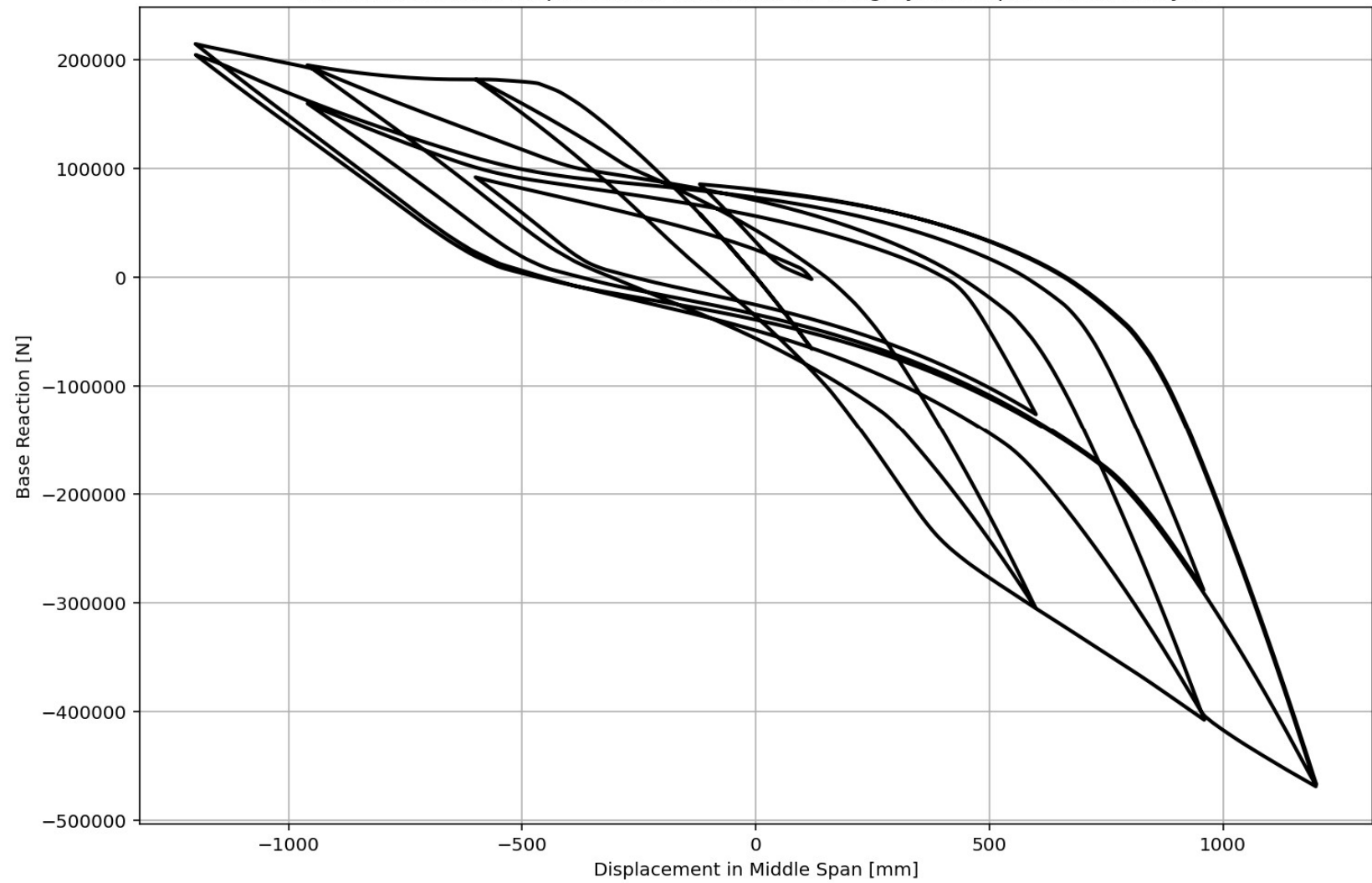
Base Reaction and Displacement of Structure During Cyclic-Displacement Analysis

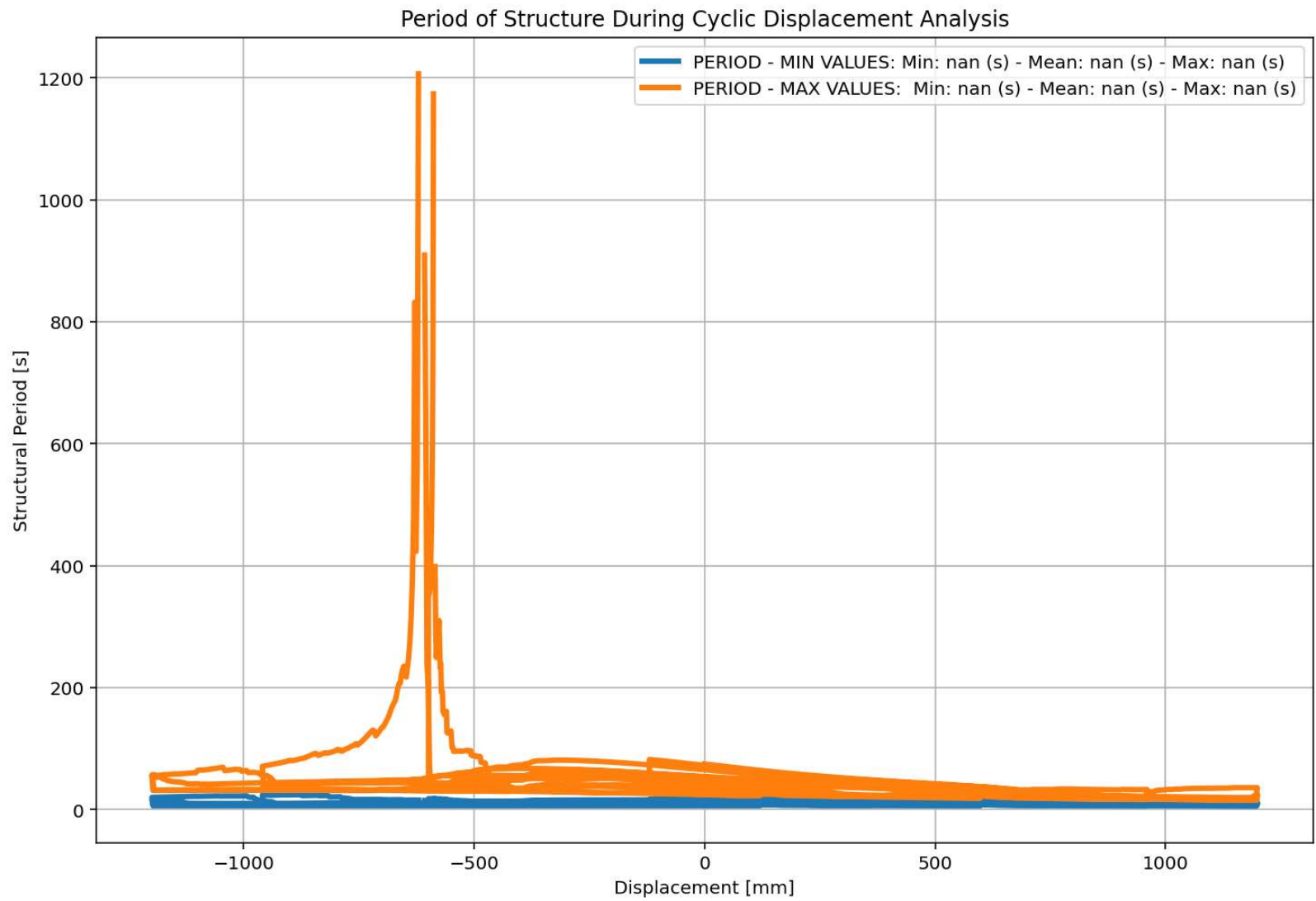


IPython Console Files Help Variable Explorer Debugger Plots History

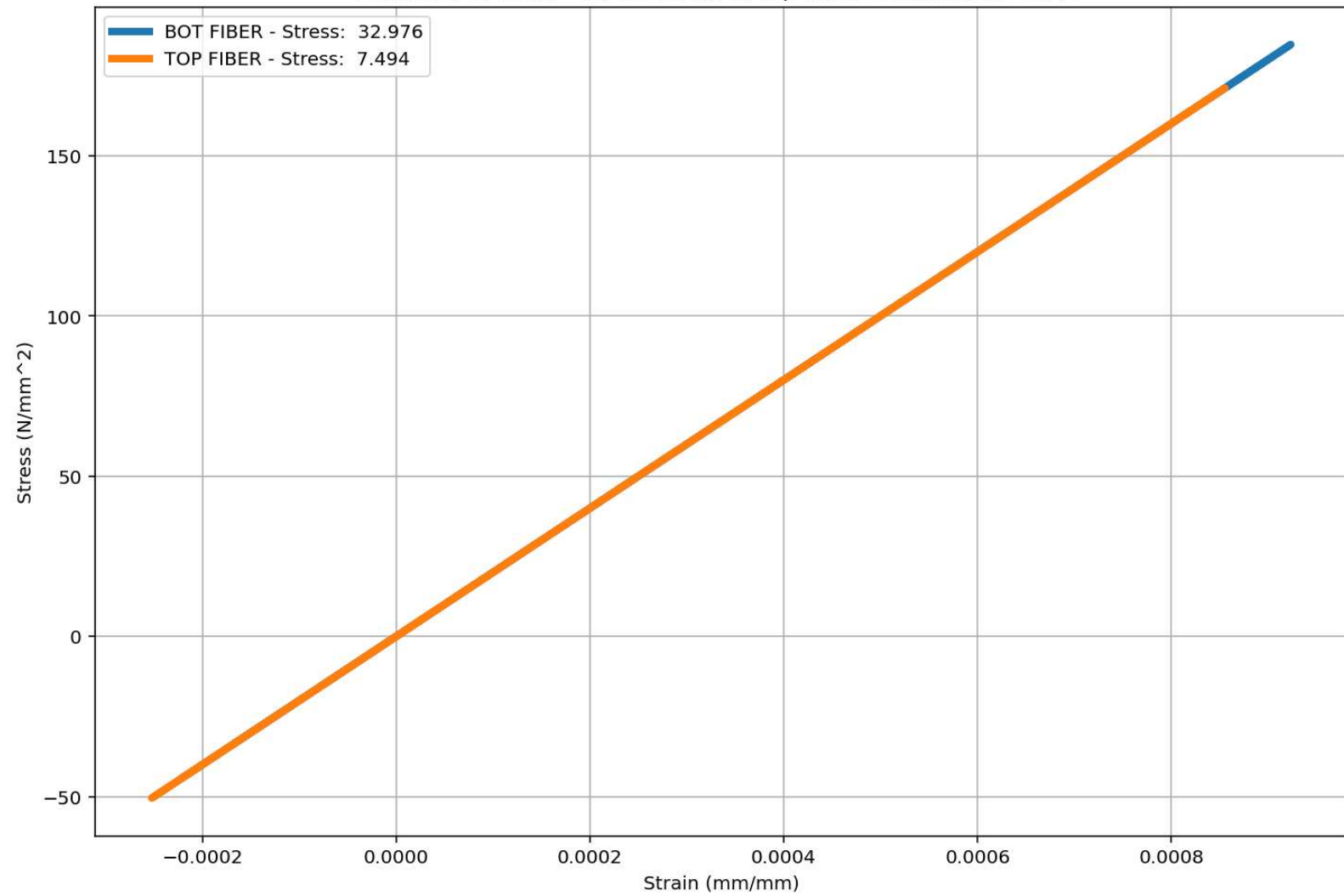
Inline Conda: anaconda3 (Python 3.12.7) LSP: Python Line 100, Col 45 UTF-8 CRLF RW Mem 46%

Base Reaction and Displacement of Structure During Cyclic-Displacement Analysis

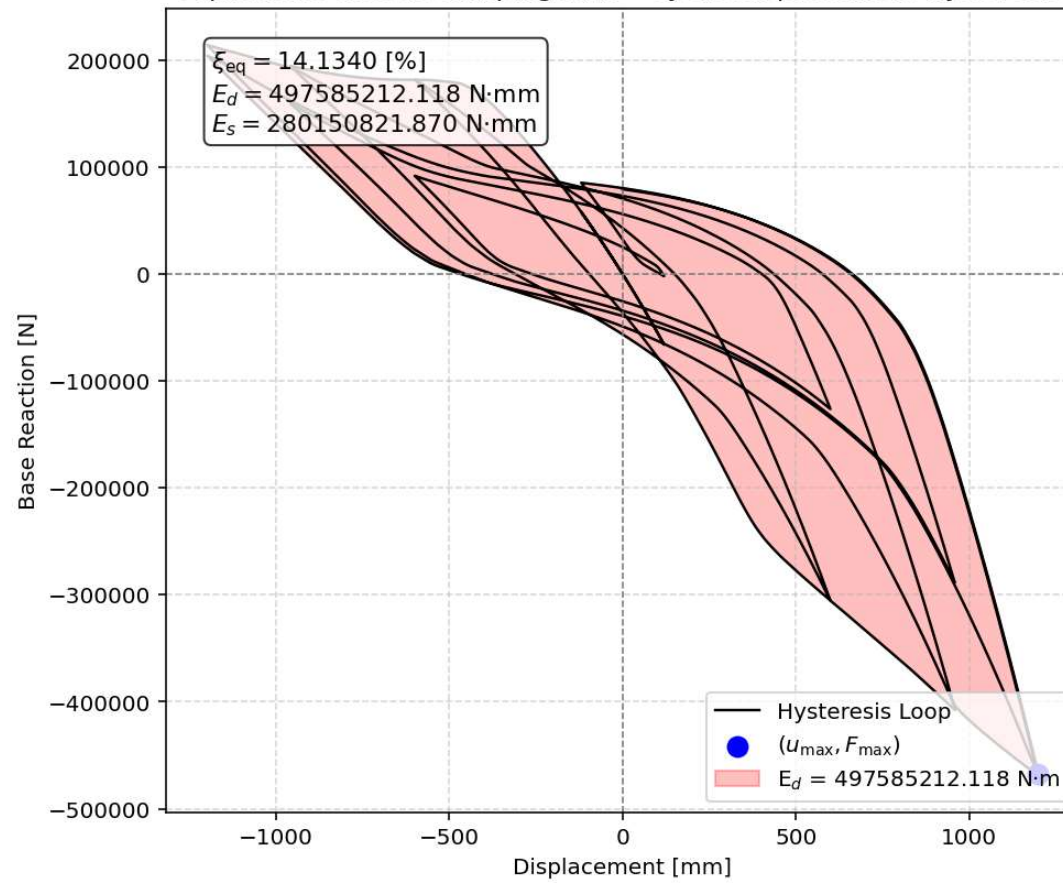


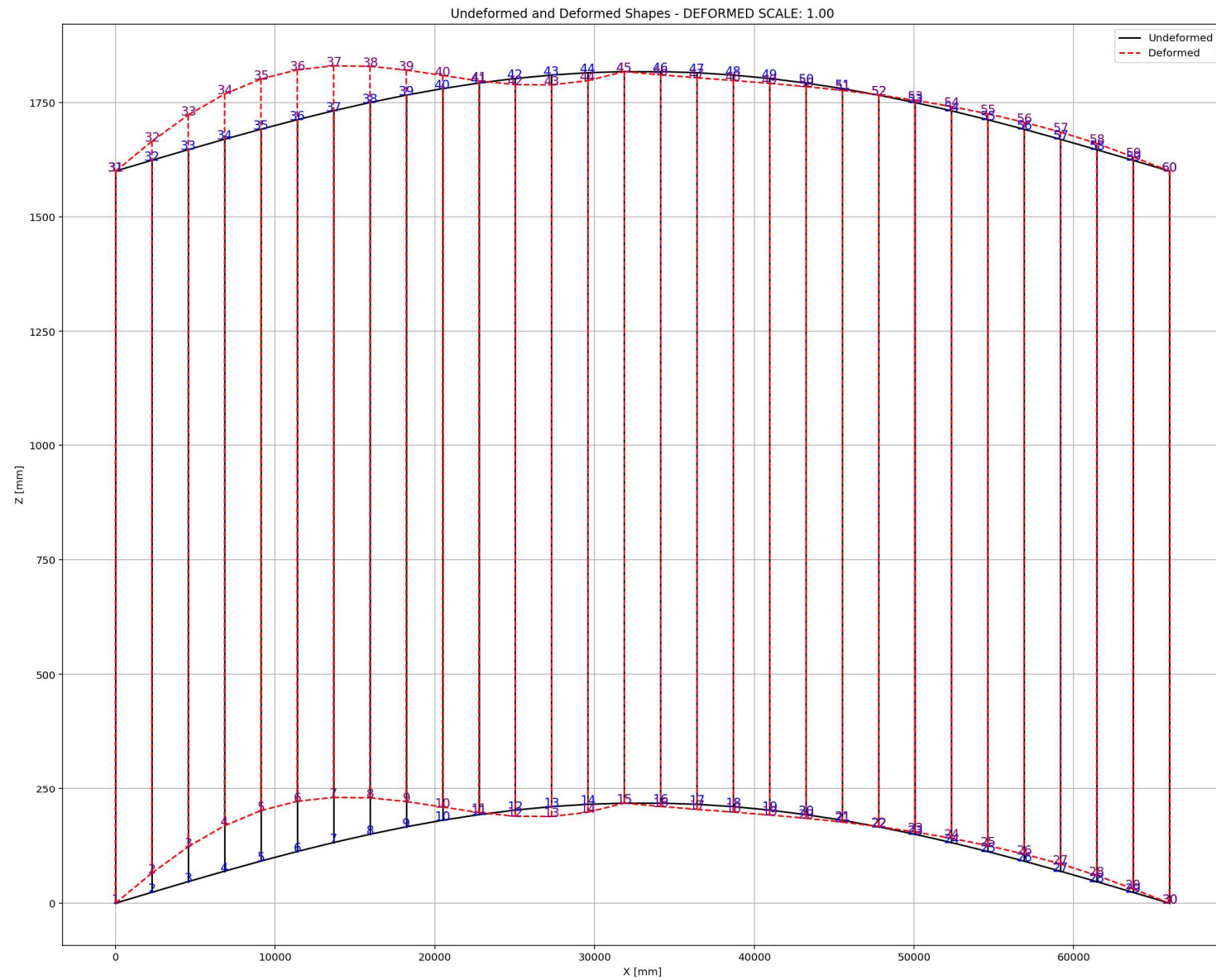


Behavior of beam - Stress-strain of Top and Bottom Fibers Curve



Equivalent viscous damping ratio - Cyclic Displacement Hysteresis





STATIC TIME-HISTORY WITH EXTERNAL TIME-DEPENDENT LOADING ANALYSIS

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY...(t)_PEDESTRIAN_BRIDGE_VIERENDEEL_GIRDERS_8_ANA.py

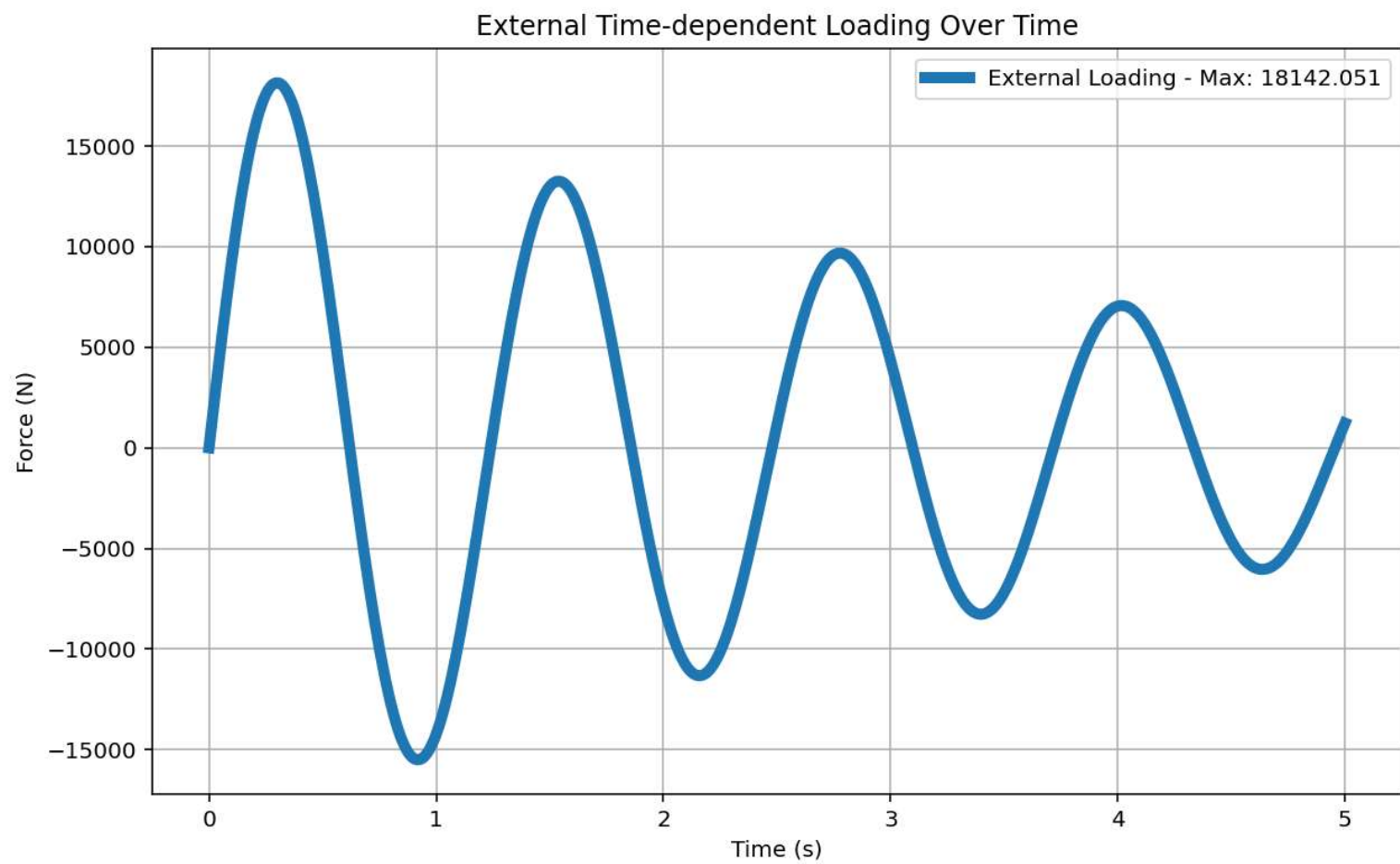
P(t)_PEDESTRIAN_BR...L_GIRDERS_8_ANA.py X I_STEEL_PLATE_SECT...EIGHT_FUN_EXTRA.py X

```
426
427 if ANAL_TYPE == 'STATIC EXTERNAL TIME-DEPENDENT LOADING': # STATIC TIME-HISTORY ANALYS
428     %% DEFINE EXTERNAL TIME-DEPENDENT LOADING PROPERTIES
429     # IN HERE ANALYSIS TIME AND DURATION ARE LOAD STEPS
430     duration = 20.0          # [s] Analysis duration
431     dt = 0.01               # [s] Time step
432     DT = dt                 # [s] Time step
433     DT_time = 5.0           # [s] Total external Load Analysis Durations [*****]
434     force_amplitude = 19600.0 # [N] Amplitude Force
435     omega_DT = 5.0715       # [rad/s] Natural angular frequency
436     time_steps = int(duration/dt)
437
438     # Check function
439     def CHECK_FUN(DT_time ,duration):
440         if DT_time > duration:
441             print('\n\nAnalysis Duration Must be greater than External Load Duration\n')
442             exit()
443         return -1
444
445     CHECK_FUN(DT_time ,duration)
446     def EXTERNAL_TIME_DEPENDENT(force_amplitude, omega_DT, DT, DT_time): # P(t) = P0 s
447         import numpy as np
448         import matplotlib.pyplot as plt
449         # External Load Durations
450         num_steps = int(DT_time / DT)
451         load_time = np.linspace(0, DT_time, num_steps)
452         target_frequency = 1.0 * omega_DT # Target excitation frequency
453         DT_load = force_amplitude * np.sin(target_frequency * load_time)
454         # Plot External Time-dependent Loading
455         plt.figure(figsize=(10, 6))
456         plt.plot(load_time, DT_load, label=f'External Loading - Max: {np.max(DT_Load)}:
457         plt.title('External Time-dependent Loading Over Time')
458         plt.xlabel('Time (s)')
459         plt.ylabel('Force (N)')
```

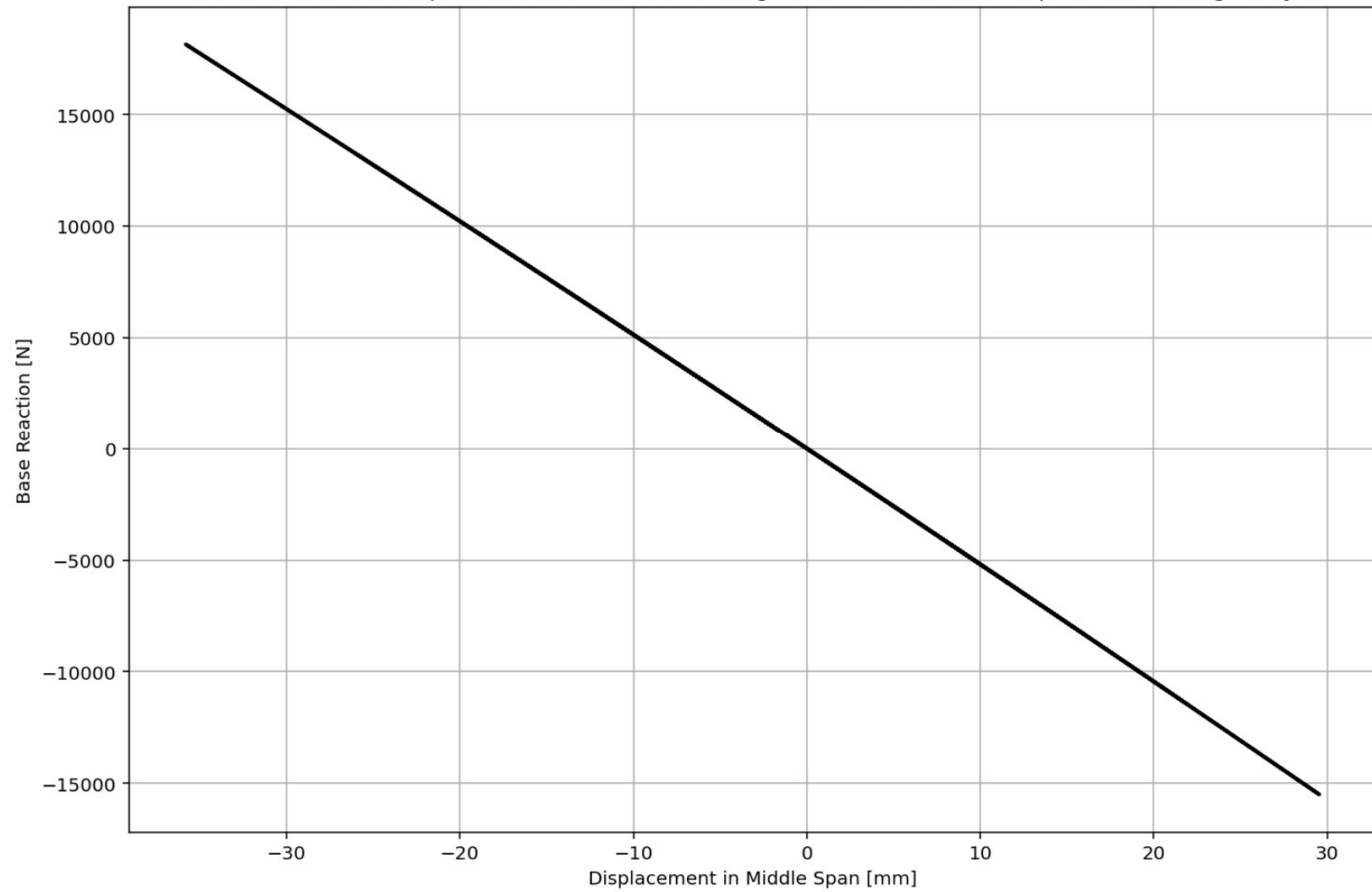
Base Reaction and Displacement of Structure During Static External Time-dependent Loading Analysis

IPython Console Files Help Variable Explorer Debugger Plots History

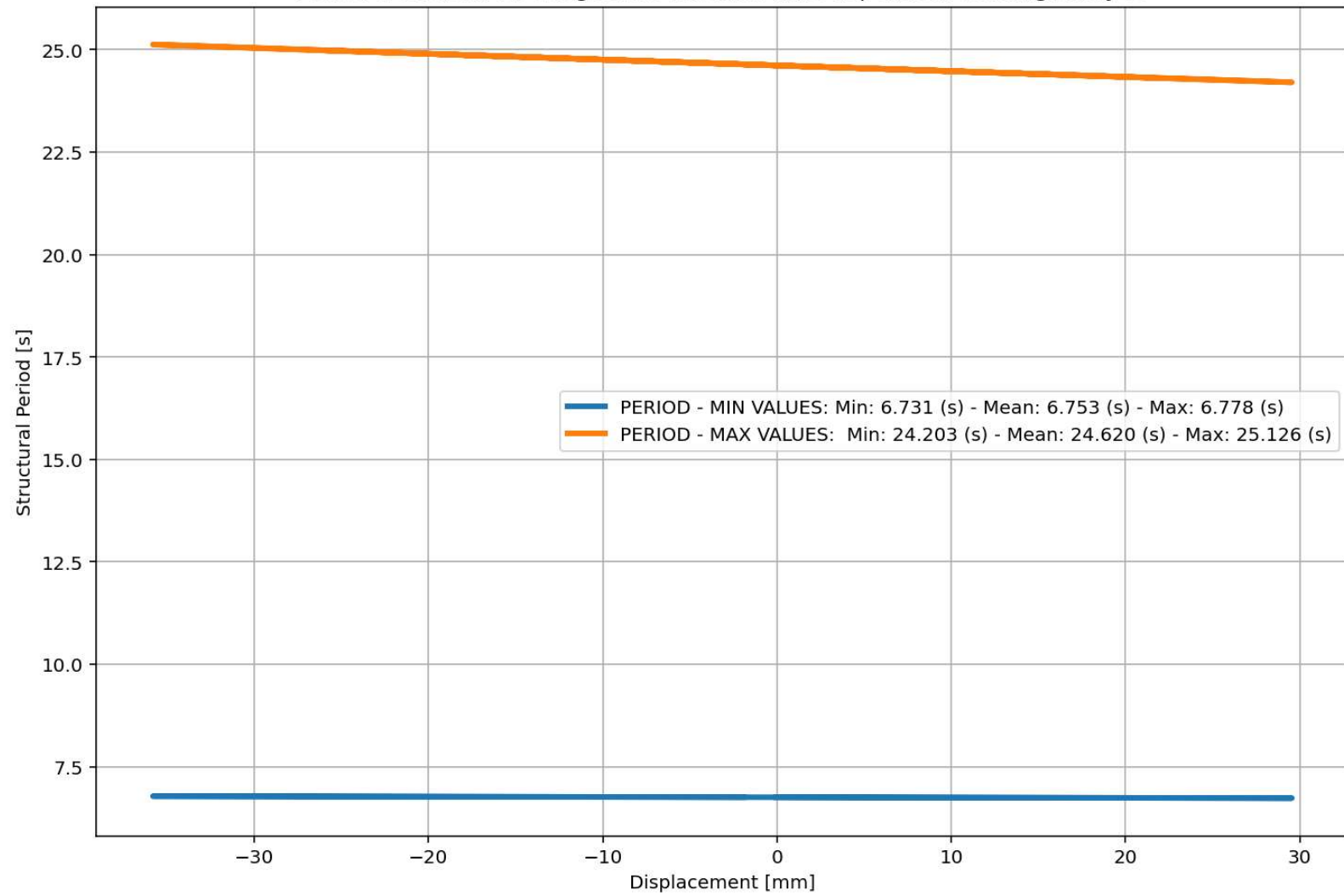
Inline Conda: anaconda3 (Python 3.12.7) ✓ LSP: Python Line 100, Col 45 UTF-8 CRLF RW Mem 48%

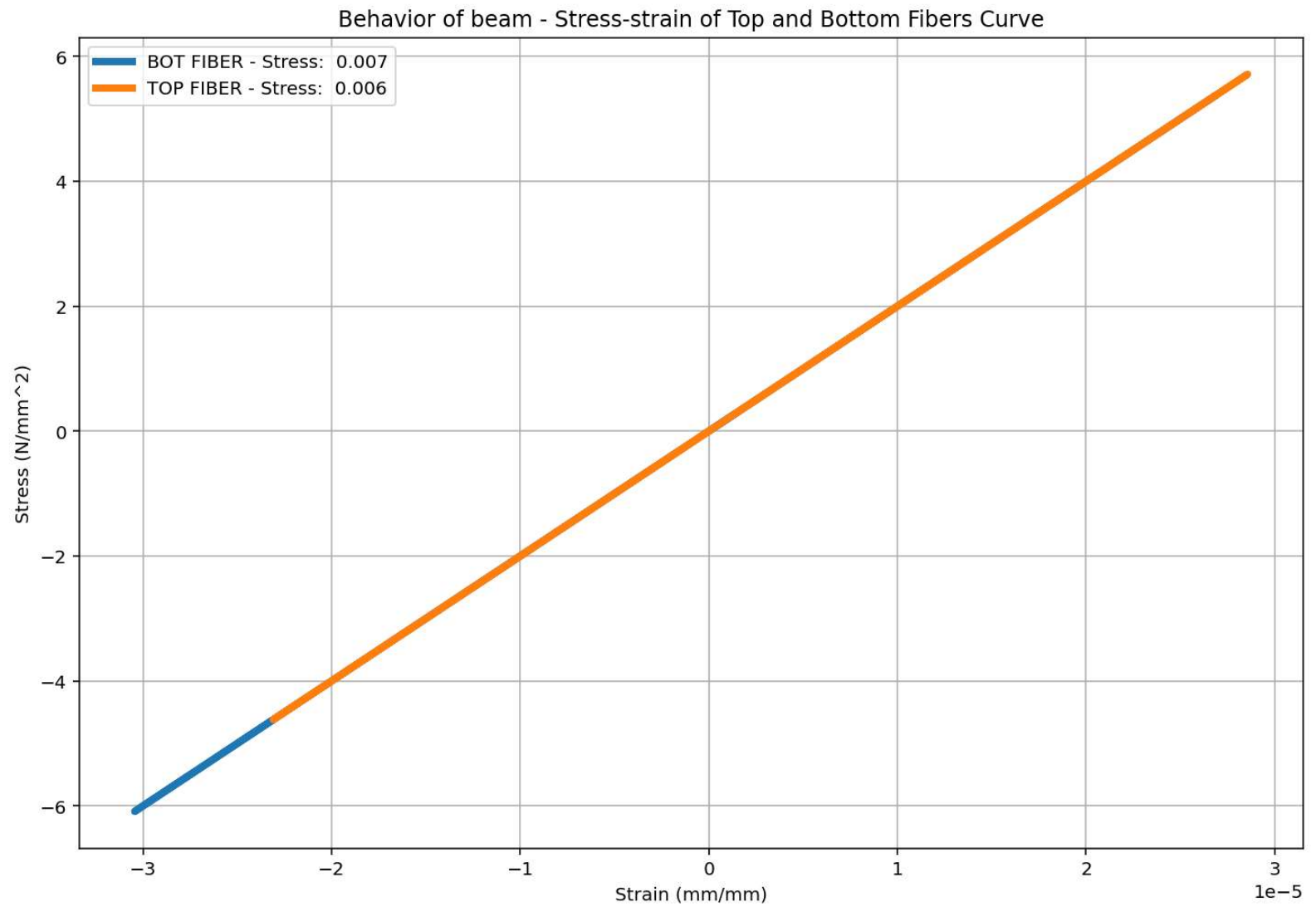


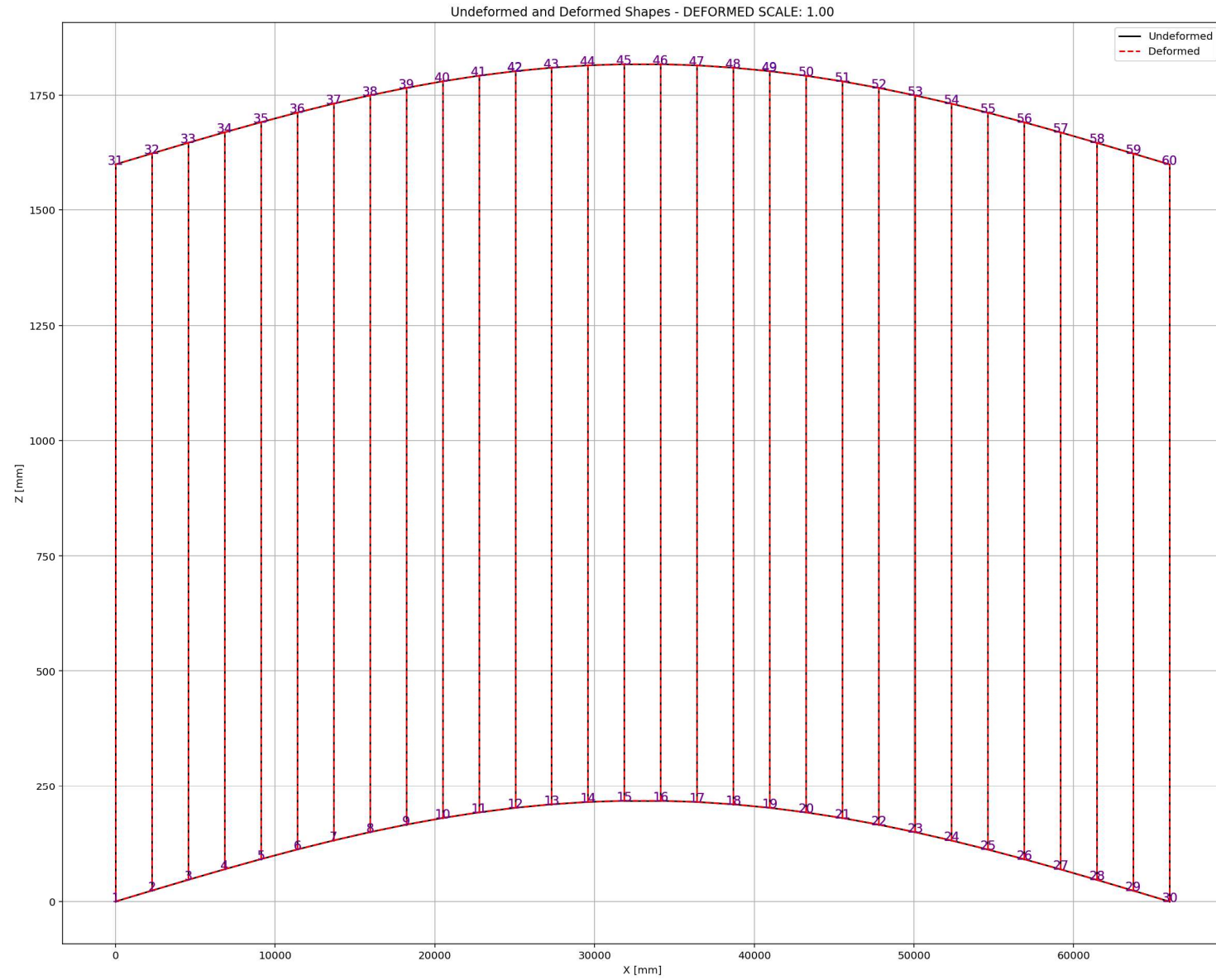
Base Reaction and Displacement of Structure During Static External Time-dependent Loading Analysis



Period of Structure During Static External Time-dependent Loading Analysis







DYNAMIC TIME-HISTORY WITH EXTERNAL TIME-DEPENDENT LOADING ANALYSIS

Spyder (Python 3.12)

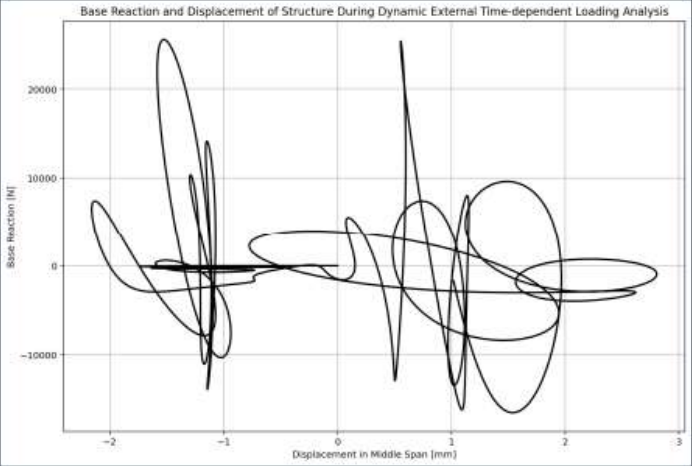
File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY...(t)_PEDESTRIAN_BRIDGE_VIERENDEEL_GIRDERS_8_ANA.py

P(t)_PEDESTRIAN_BR...L_GIRDERS_8_ANA.py I_STEEL_PLATE_SECT...EIGHT_FUN_EXTRA.py

```
554
555 if ANAL TYPE == 'DYNAMIC EXTERNAL TIME-DEPENDENT LOADING': # DYNAMIC TIME-HISTORY ANAL
556     %% DEFINE EXTERNAL TIME-DEPENDENT LOADING PROPERTIES
557     duration = 20.0          # [s] Analysis duration
558     dt = 0.001              # [s] Time step
559     DT = dt                  # [s] Time step
560     DT_time = 5.0           # [s] Total external Load Analysis Durations [*****]
561     force_amplitude = 19600.0 # [N] Amplitude Force
562     omega_DT = 5.0715        # [rad/s] Natural angular frequency
563     DR = 0.03                # Damping Ratio
564
565     # Check function
566     def CHECK_FUN(DT_time ,duration):
567         if DT_time > duration:
568             print('\n\nAnalysis Duration Must be greater than External Load Duration\n')
569             exit()
570             return -1
571
572     CHECK_FUN(DT_time ,duration)
573     def EXTERNAL_TIME_DEPENDENT(force_amplitude, omega_DT, DT, DT_time): # P(t) = P0 sin
574         import numpy as np
575         import matplotlib.pyplot as plt
576         # External Load Durations
577         num_steps = int(DT_time / DT)
578         load_time = np.linspace(0, DT_time, num_steps)
579         target_frequency = 1.0 * omega_DT # Target excitation frequency
580         DT_load = force_amplitude * np.sin(target_frequency * load_time)
581         # Plot External Time-dependent Loading
582         plt.figure(figsize=(10, 6))
583         plt.plot(load_time, DT_load, label=f'External Loading - Max: {np.max(DT_load)}')
584         plt.title('External Time-dependent Loading Over Time')
585         plt.xlabel('Time (s)')
586         plt.ylabel('Force (N)')
587         plt.grid(True)
```

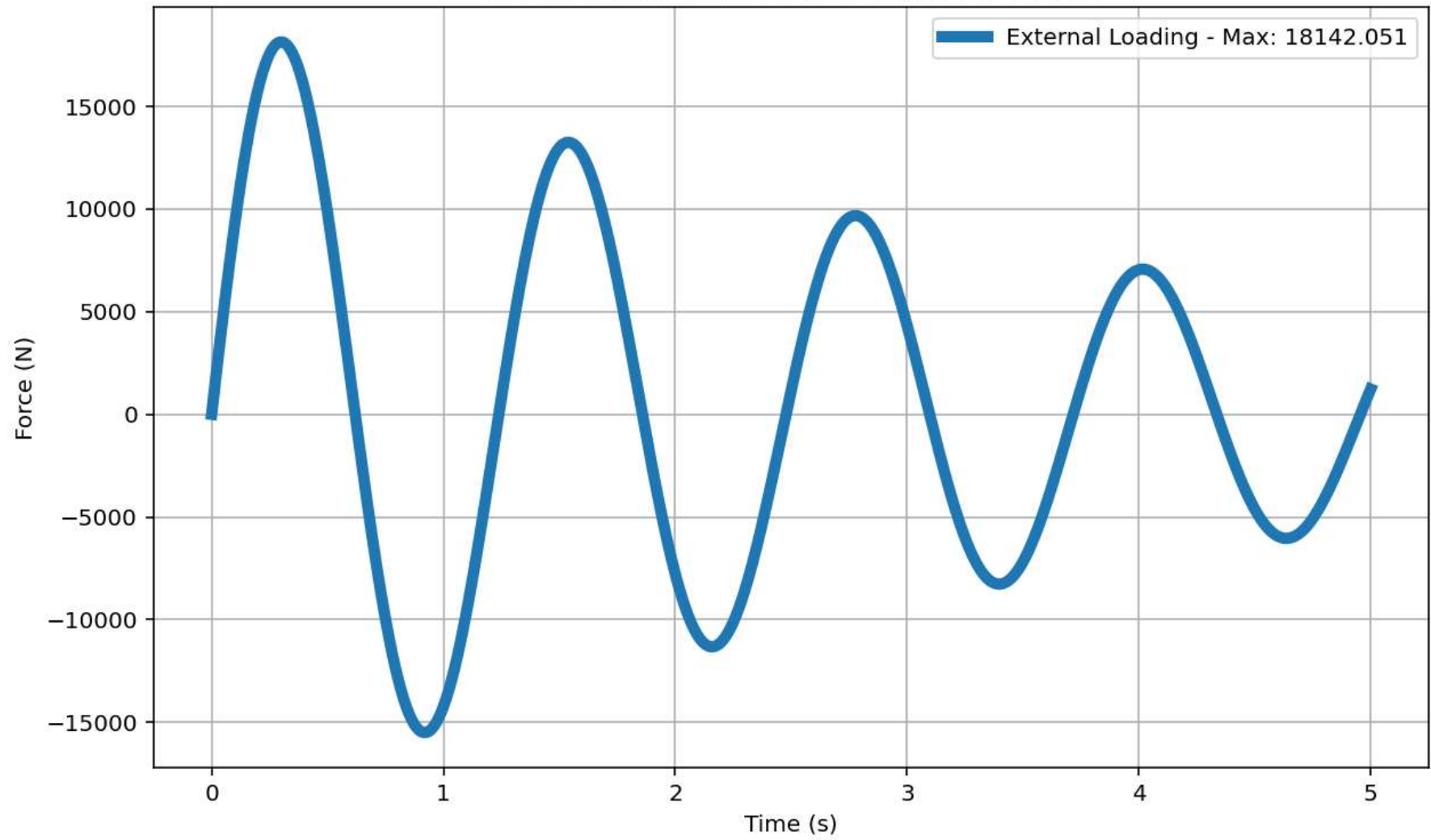
Base Reaction and Displacement of Structure During Dynamic External Time-dependent Loading Analysis



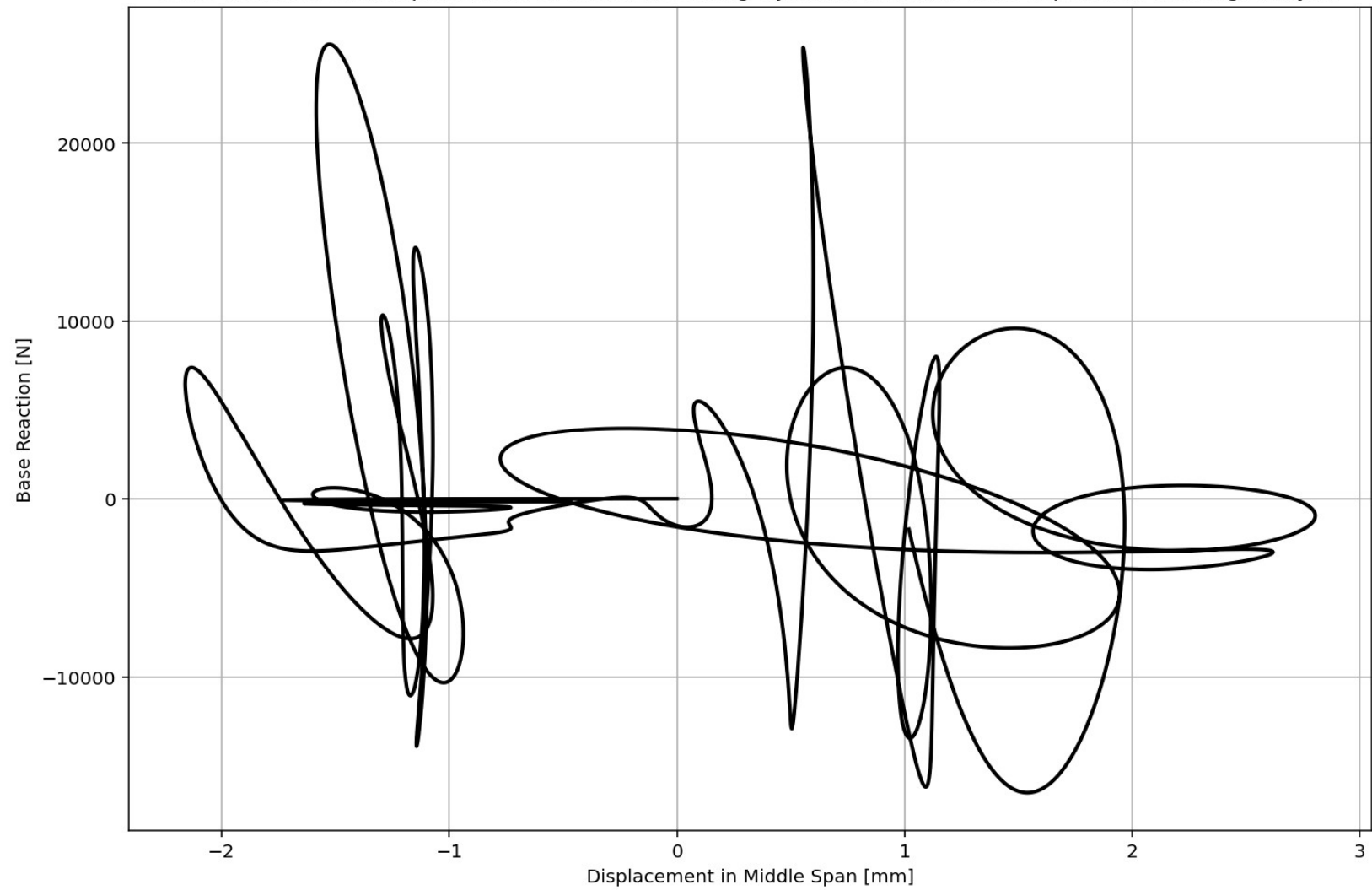
IPython Console Files Help Variable Explorer Debugger Plots History

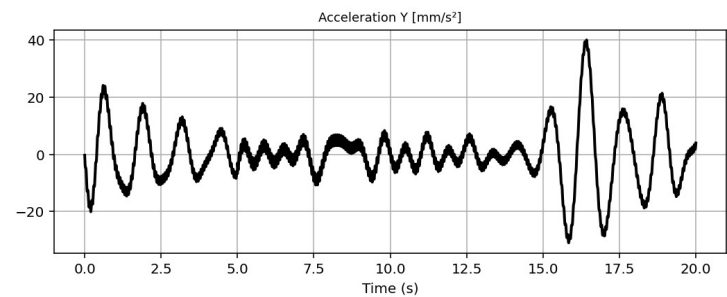
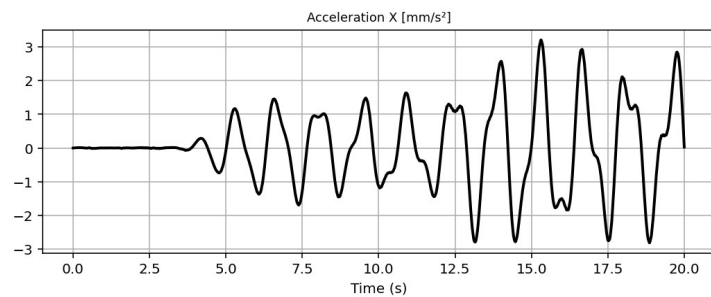
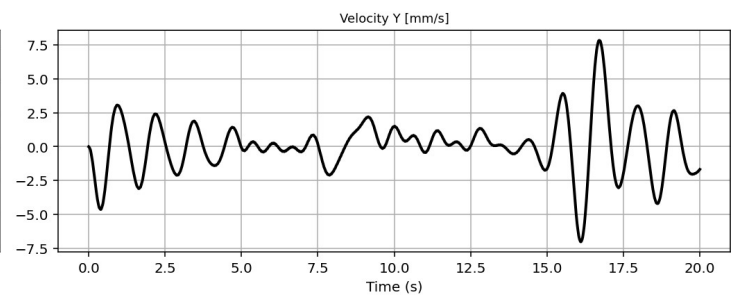
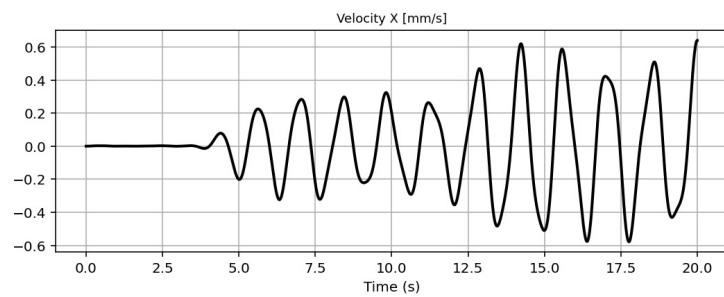
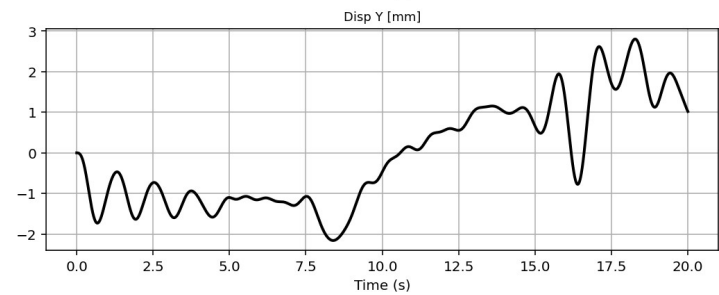
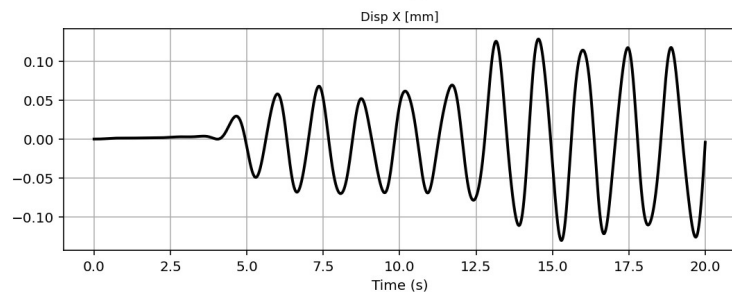
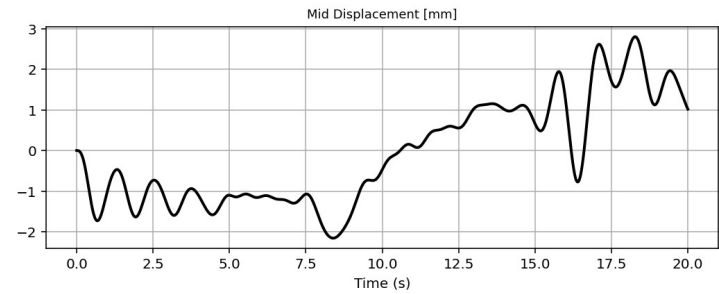
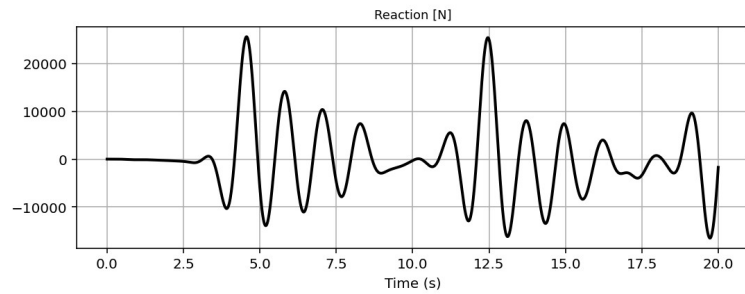
Inline Conda: anaconda3 (Python 3.12.7) ✓ LSP: Python Line 100, Col 45 UTF-8 CRLF RW Mem 44%

External Time-dependent Loading Over Time

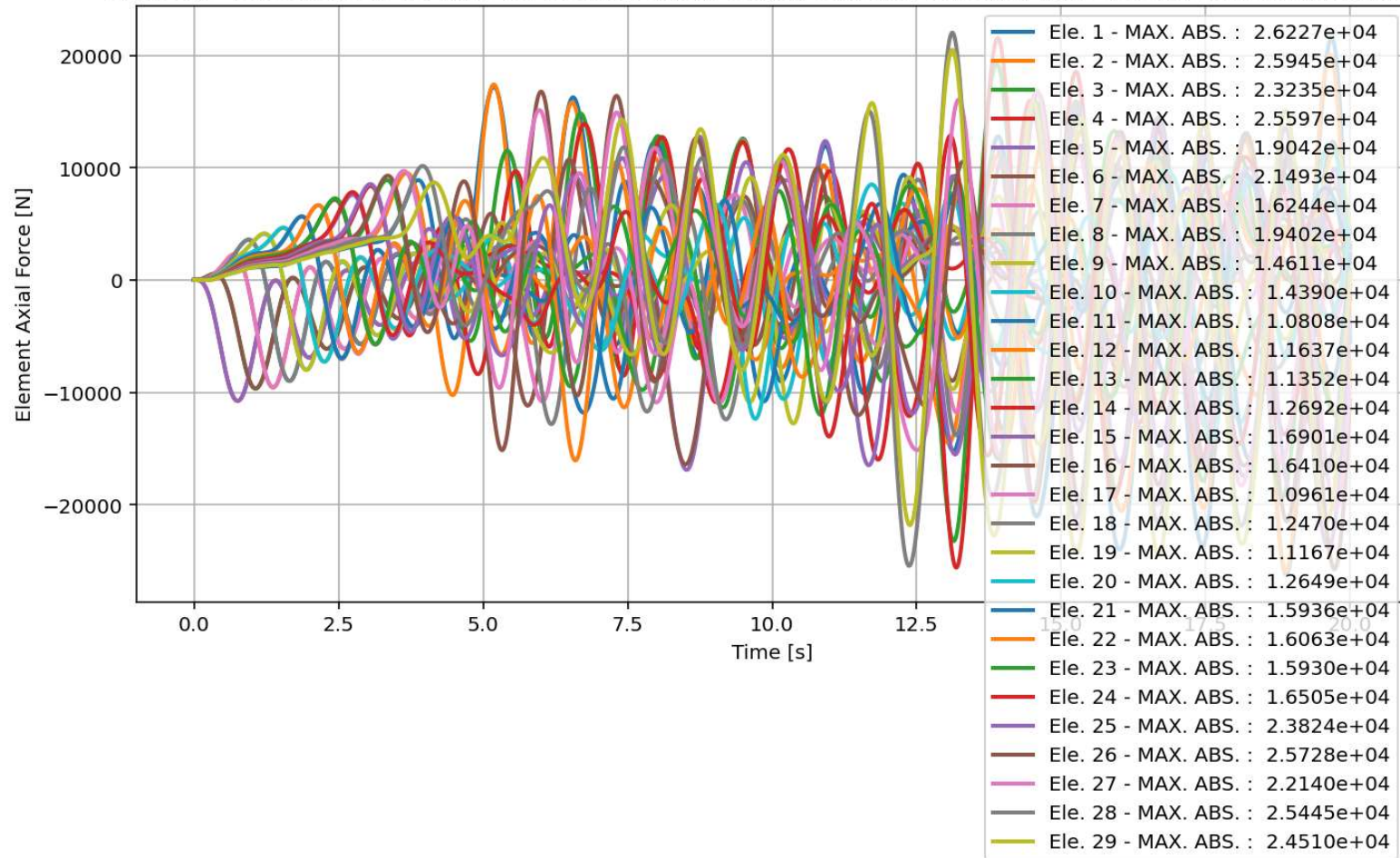


Base Reaction and Displacement of Structure During Dynamic External Time-dependent Loading Analysis

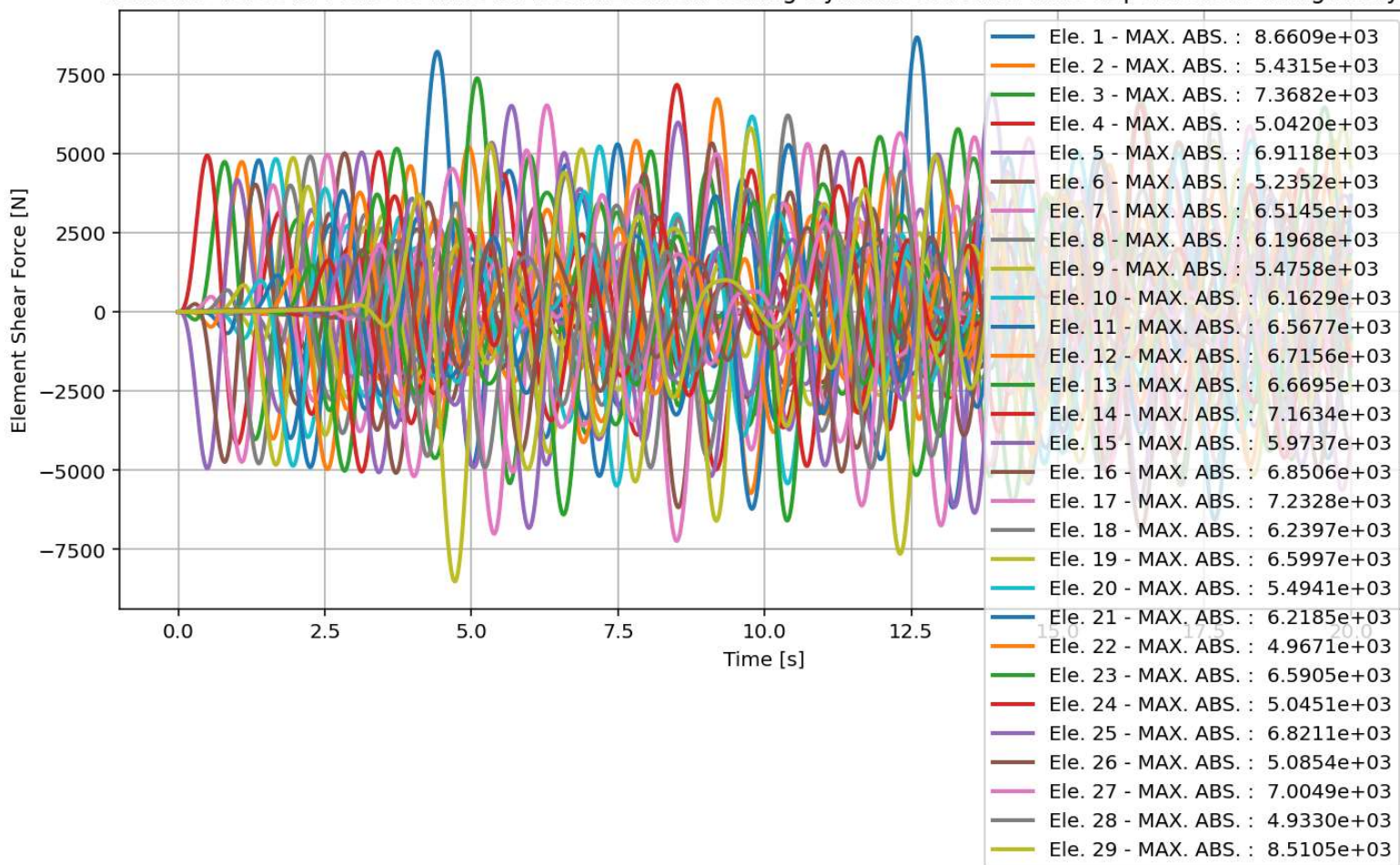




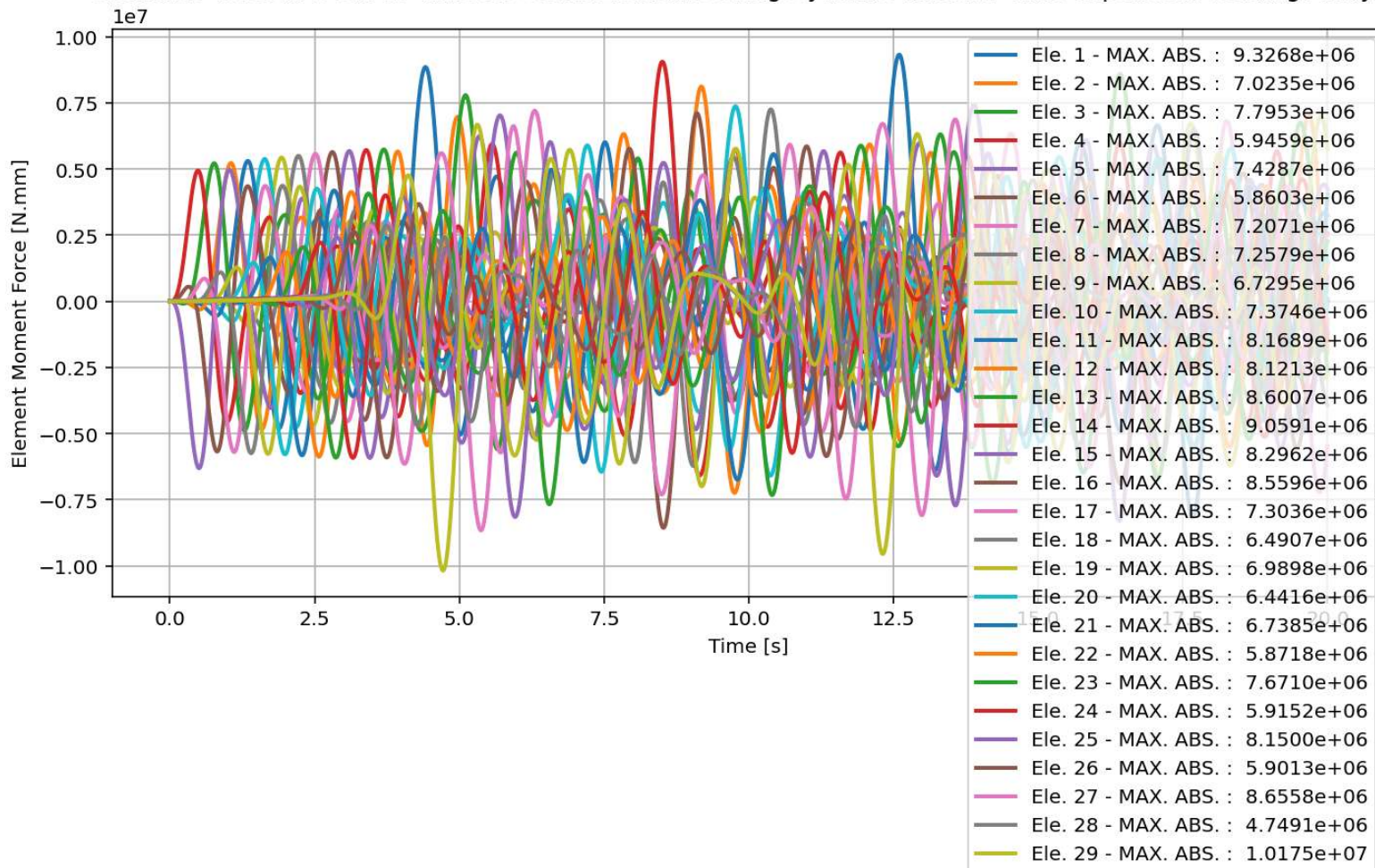
elements force in X Dir. vs Time for Beam Element During Dynamic External Time-dependent Loading Analysis



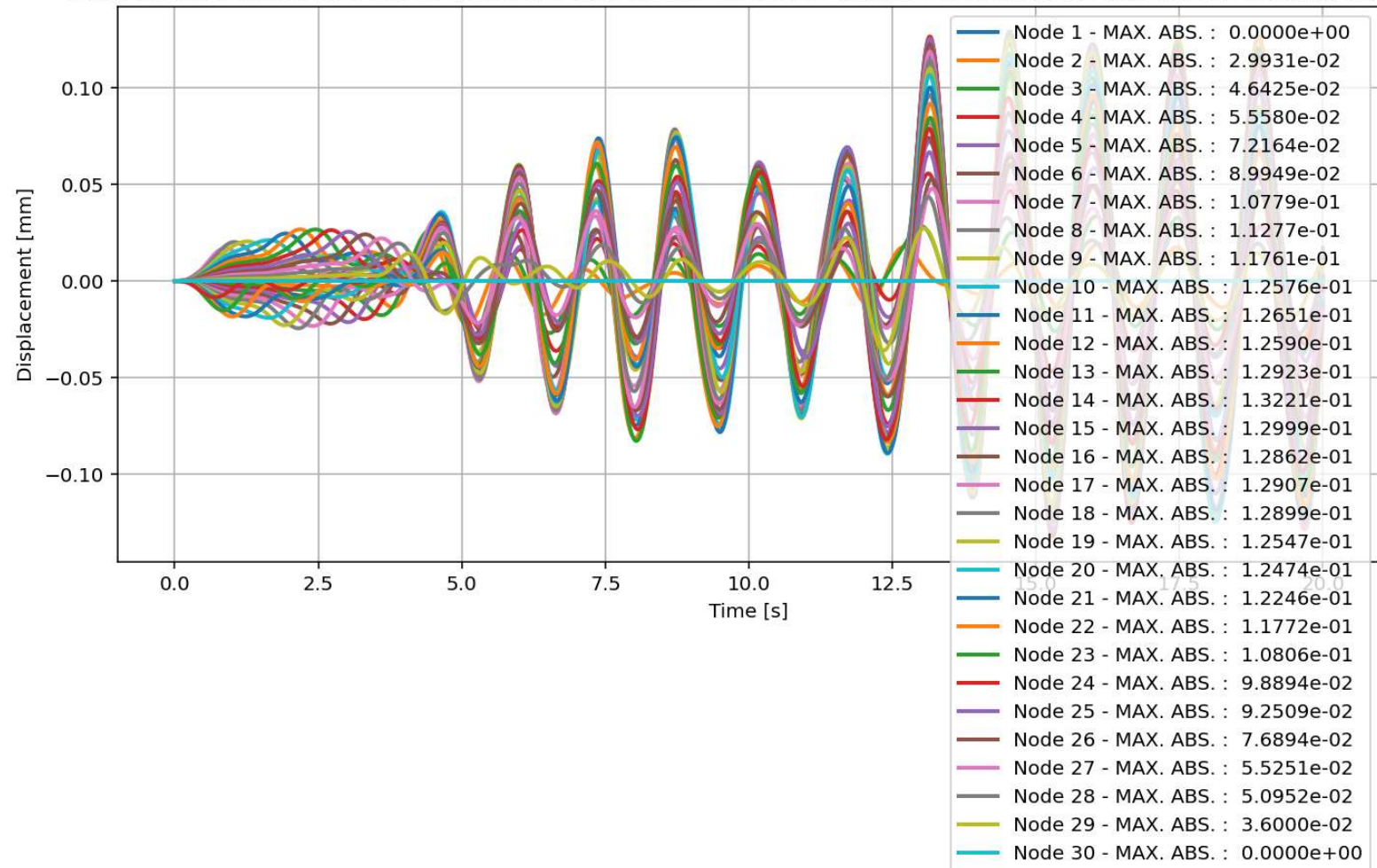
elements force in Y Dir. vs Time for Beam Element During Dynamic External Time-dependent Loading Analysis



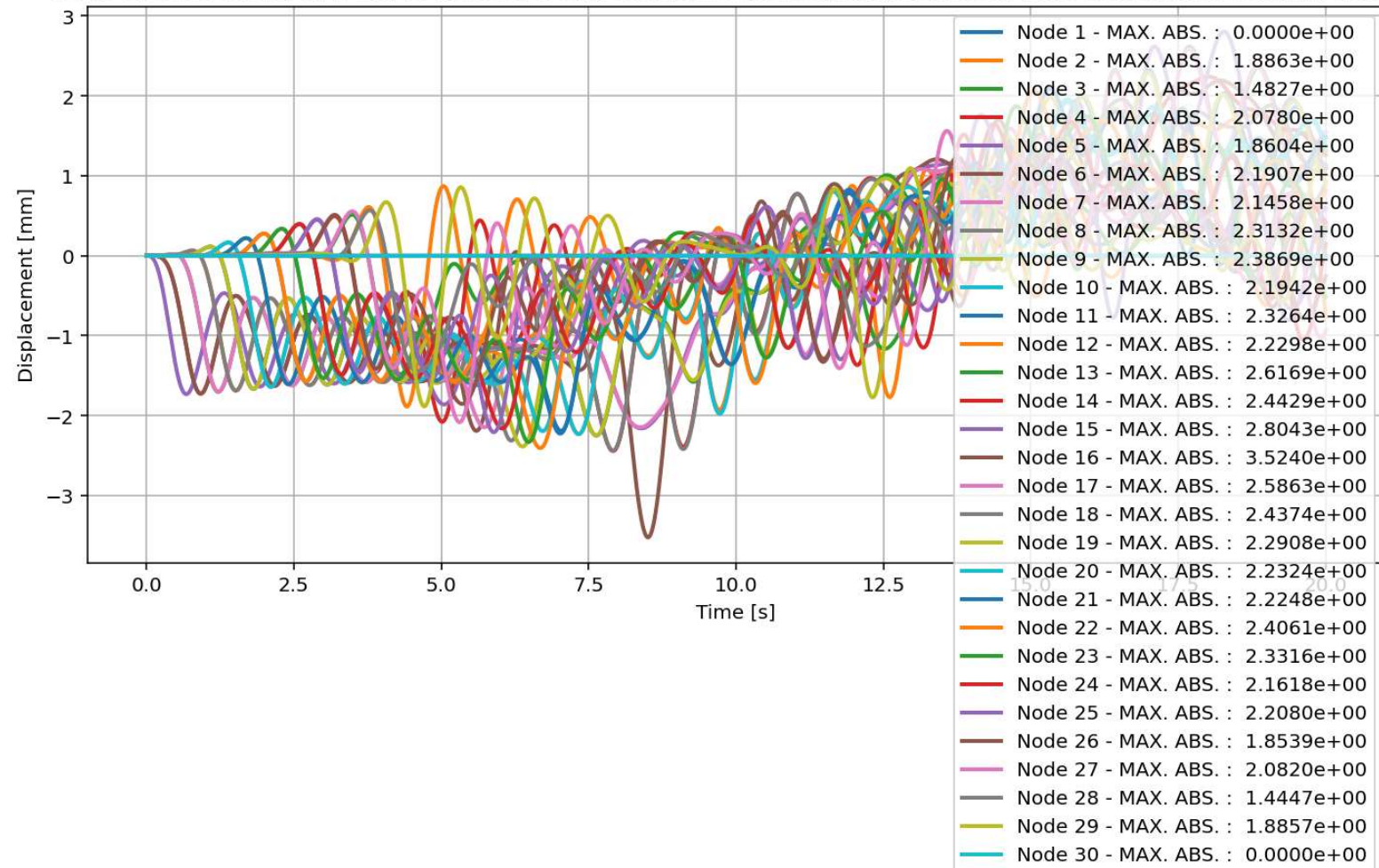
elements force in Z Dir. vs Time for Beam Element During Dynamic External Time-dependent Loading Analysis



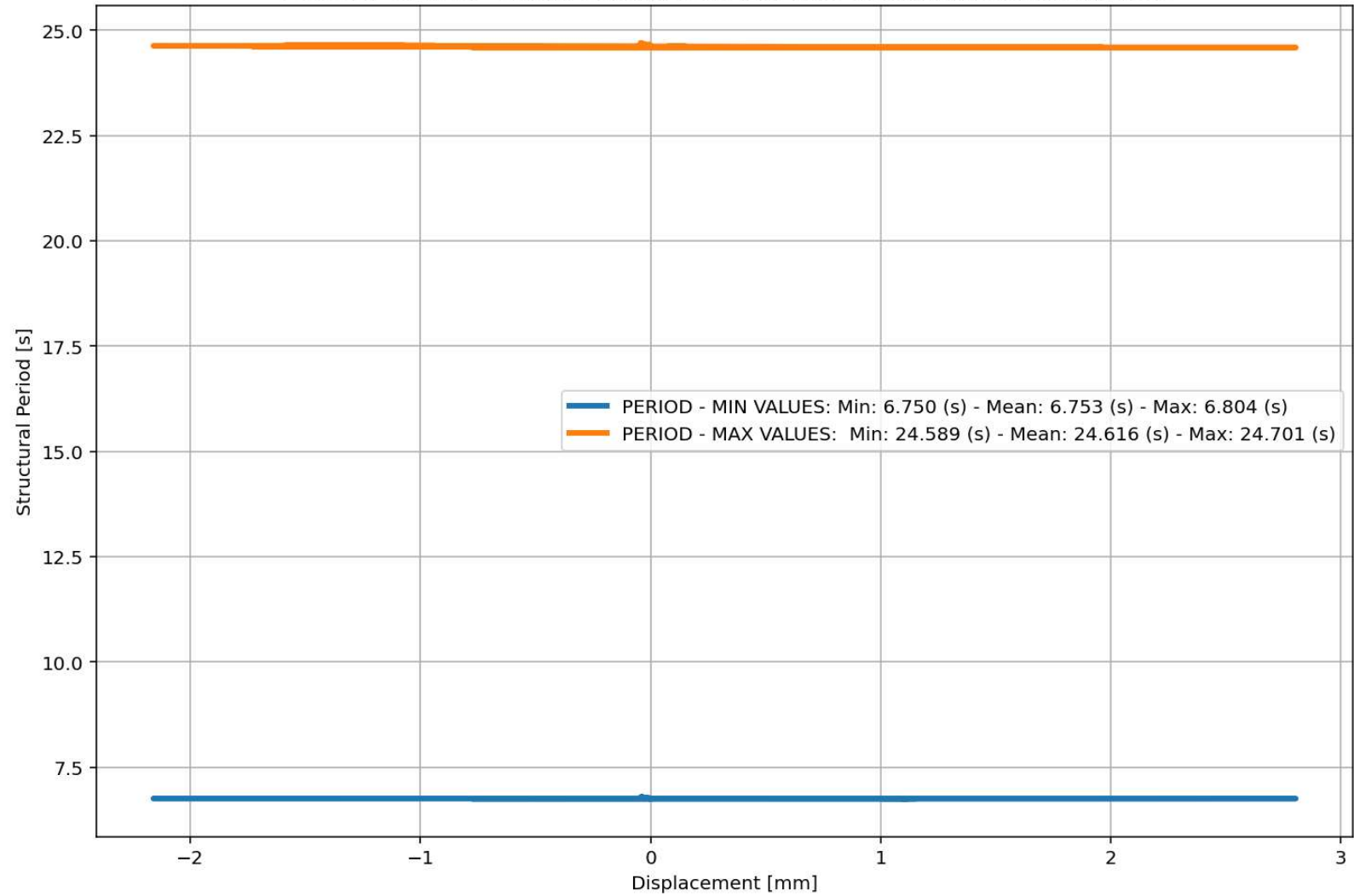
Node Displacements in X Dir. vs Time for Beam Element During Dynamic External Time-dependent Loading Analysis

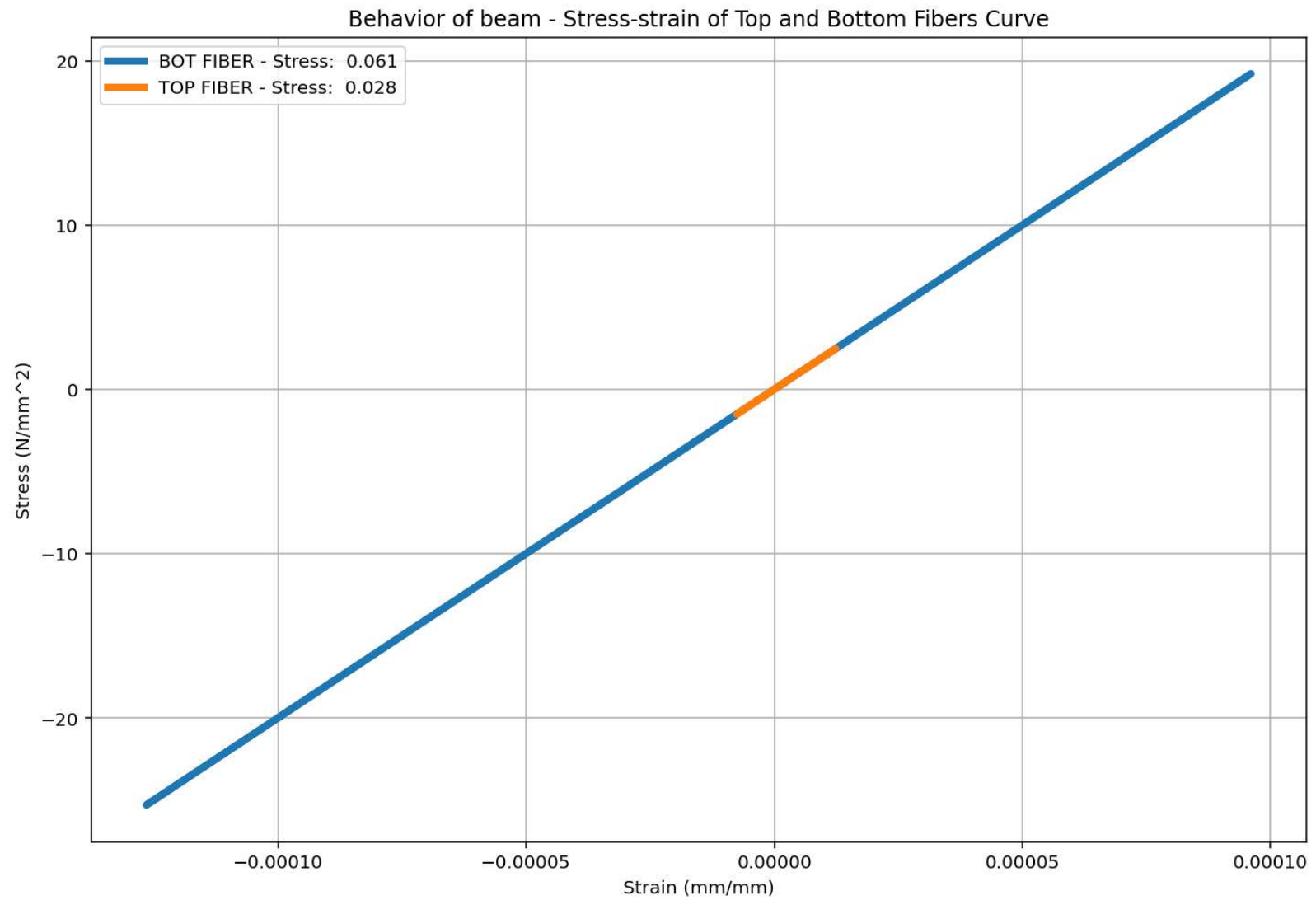


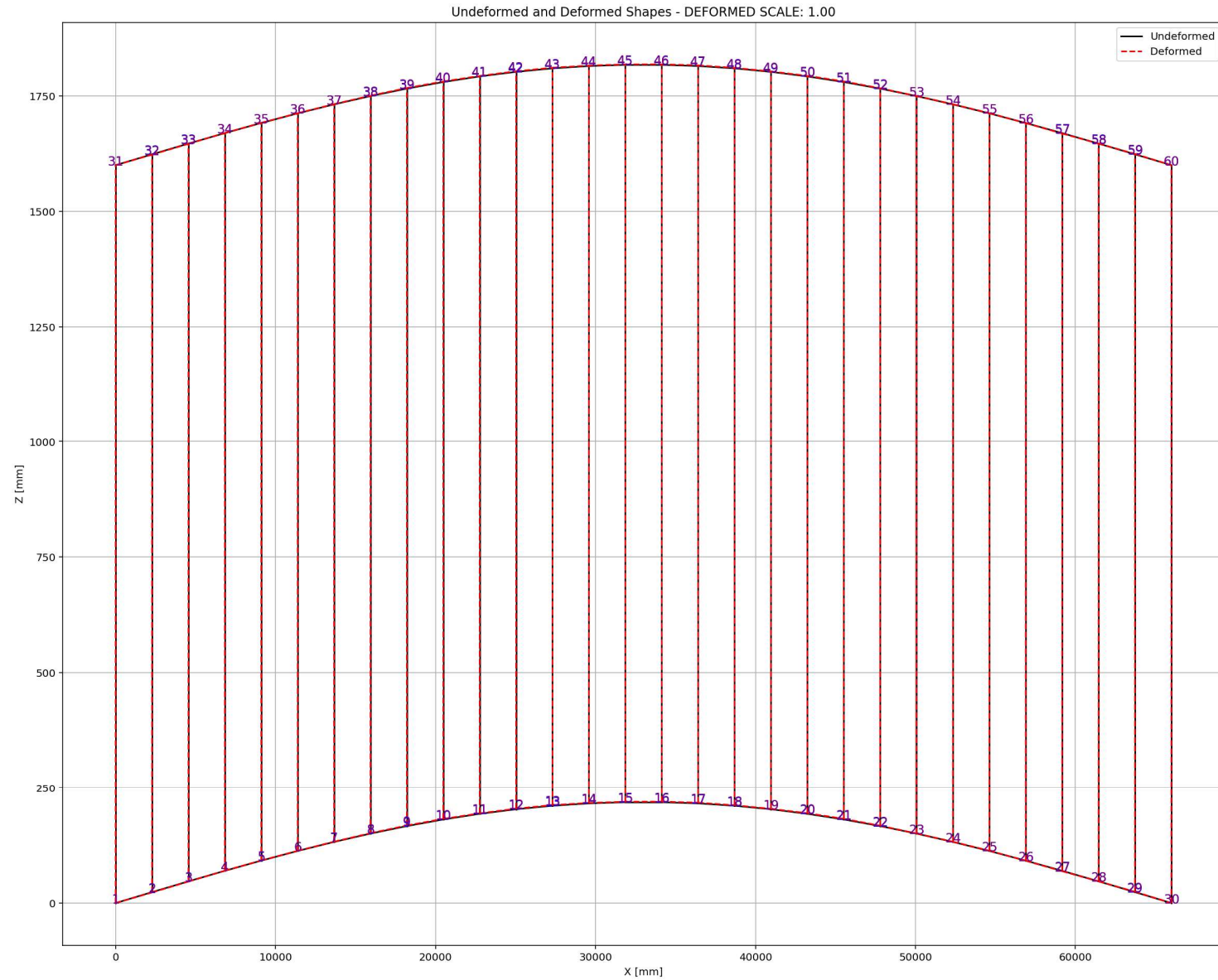
Node Displacements in Y Dir. vs Time for Beam Element During Dynamic External Time-dependent Loading Analysis



Period of Structure During Dynamic External Time-dependent Loading Analysis







FREE-VIBRATION ANALYSIS

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY...(t)_PEDESTRIAN_BRIDGE_VIERENDEEL_GIRDERS_8_ANA.py

P(t)_PEDESTRIAN_BR...L_GIRDERS_8_ANA.py x I_STEEL_PLATE_SECT...EIGHT_FUN_EXTRA.py x

```
702
703 if ANAL_TYPE == 'FREE-VIBRATION': # DYNAMIC TIME-HISTORY ANALYSIS
704     ### DEFINE PARAMETERS FOR FREE-VIBRATION ANALYSIS
705     u0 = -0.00015 # [mm] Initial displacement
706     v0 = 5.15 # [mm/s] Initial velocity
707     a0 = 0.065 # [mm/s^2] Initial acceleration
708     IU = True # Free Vibration with Initial Displacement
709     IV = True # Free Vibration with Initial Velocity
710     IA = True # Free Vibration with Initial Acceleration
711     duration = 20.0 # [s] Analysis duration
712     dt = 0.001 # [s] Time step
713     DR = 0.03 # Damping Ratio
714
715     if IU == True:
716         # Define initial displacment
717         ops.setNodeDisp(center_node, 2, u0, '-commit') # INFO LINK: https://openseespy
718     if IV == True:
719         # Define initial velocity
720         ops.setNodeVel(center_node, 2, v0, '-commit') # INFO LINK: https://openseespy
721     if IA == True:
722         # Define initial acceleration
723         ops.setNodeAccel(center_node, 2, a0, '-commit') # INFO LINK: https://openseespy
724
725     ops.constraints('Plain')
726     ops.numberer('Plain')
727     ops.system('BandGeneral')
728     ops.test('NormDispIncr', MAX_TOLERANCE, MAX_ITERATIONS) # INFO LINK: https://openseespydoc.readthedocs.io/en/latest
729     #ops.integrator('CentralDifference') # JUST FOR LINEAR ANALYSIS - INFO LINK: https://openseespydoc.readthedocs.io/en/latest
730     alpha=0.5; beta=0.25;
731     ops.integrator('Newmark', alpha, beta) # INFO LINK: https://openseespydoc.readthedocs.io/en/latest
732     #alpha=2/3; gamma=1.5-alpha; gamma=1.5-alpha; beta=(2-alpha)**2/4;
733     #ops.integrator('HHT', alpha, gamma, beta) # INFO LINK: https://openseespydoc.readthedocs.io/en/latest
734     ops.algorithm('Newton') # INFO LINK: https://openseespydoc.readthedocs.io/en/latest
735     ops.analysis('Transient') # INFO LINK: https://openseespydoc.readthedocs.io/en/latest
```

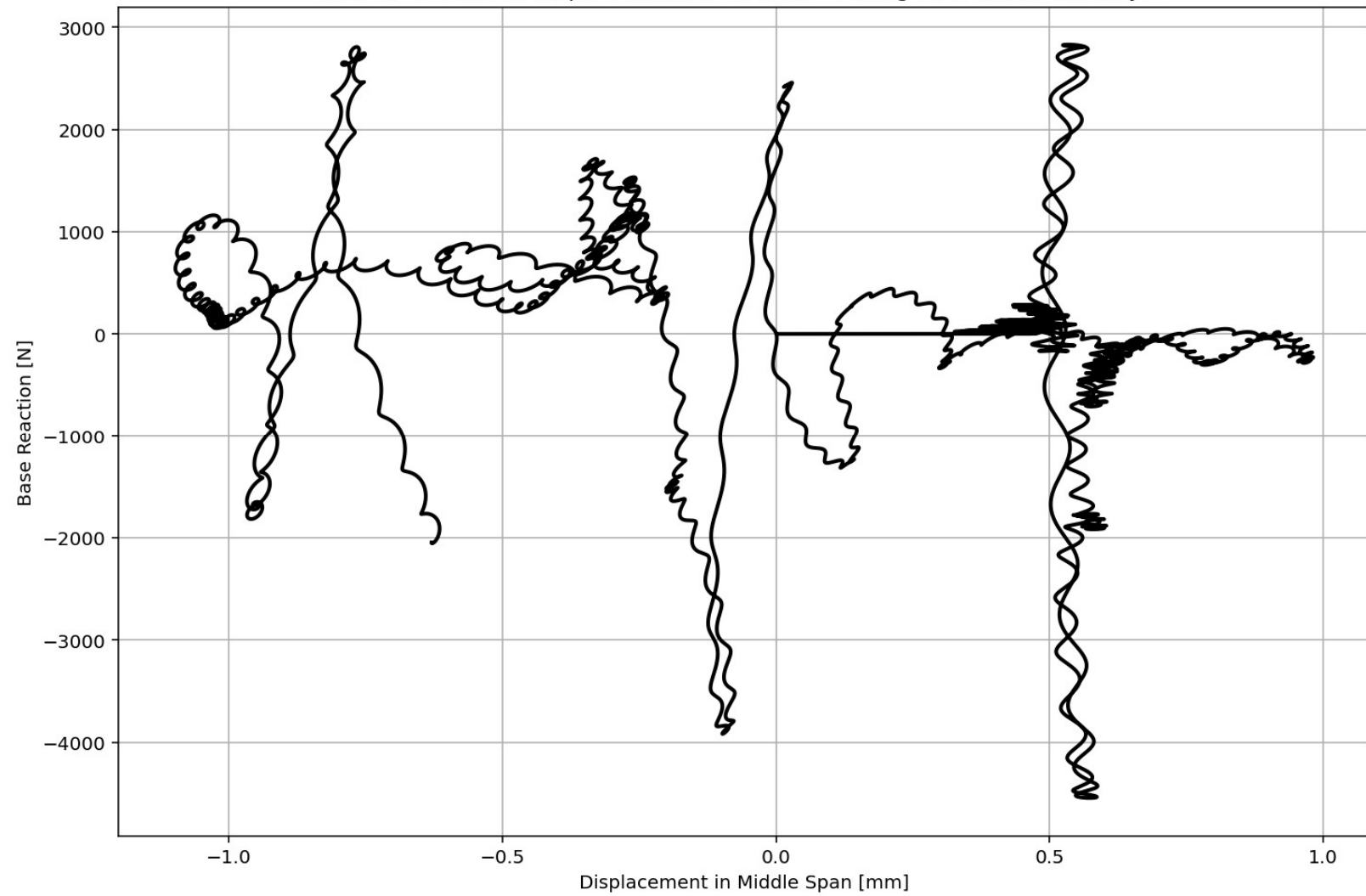
Base Reaction and Displacement of Structure During free-vibration Analysis

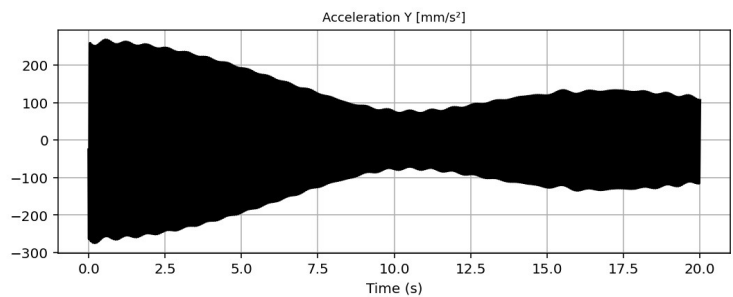
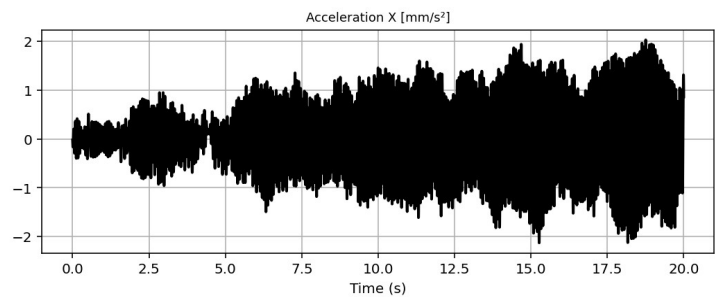
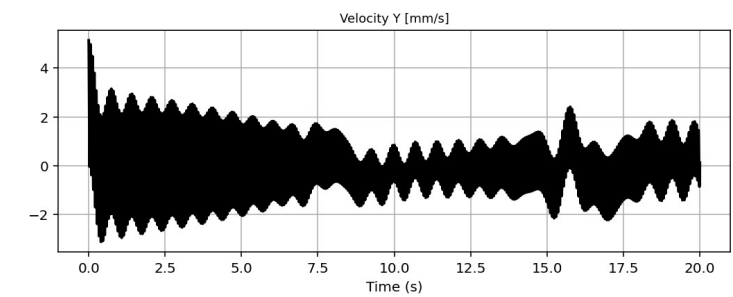
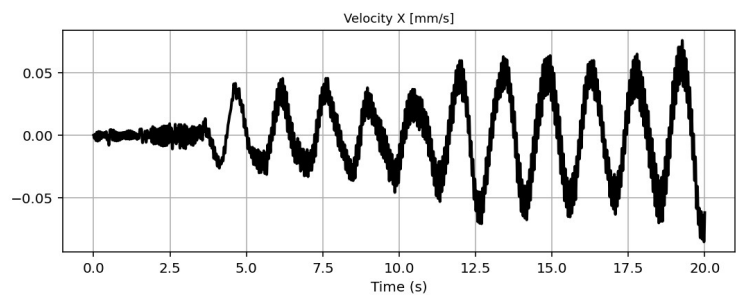
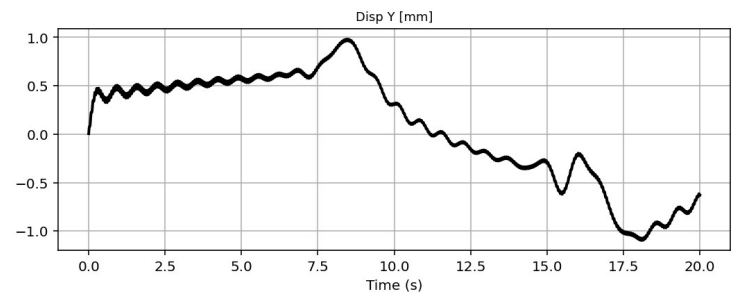
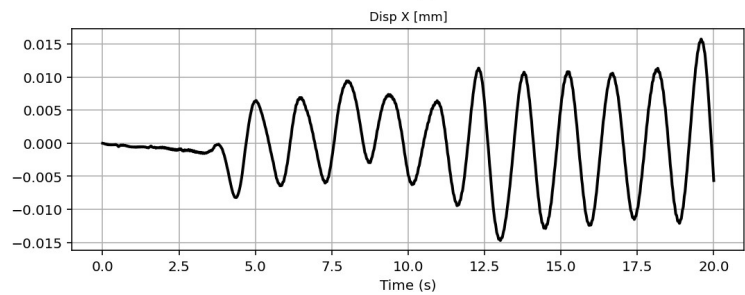
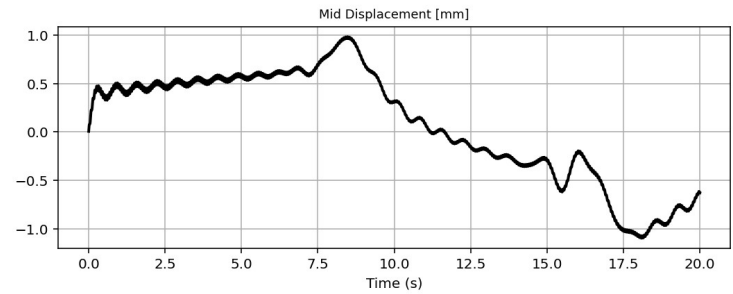
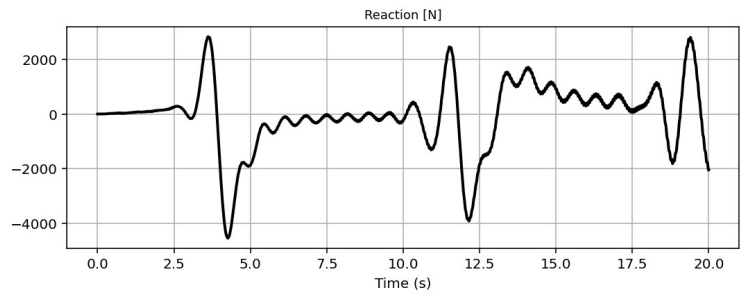
Ctrl + click to open url

IPython Console Files Help Variable Explorer Debugger Plots History

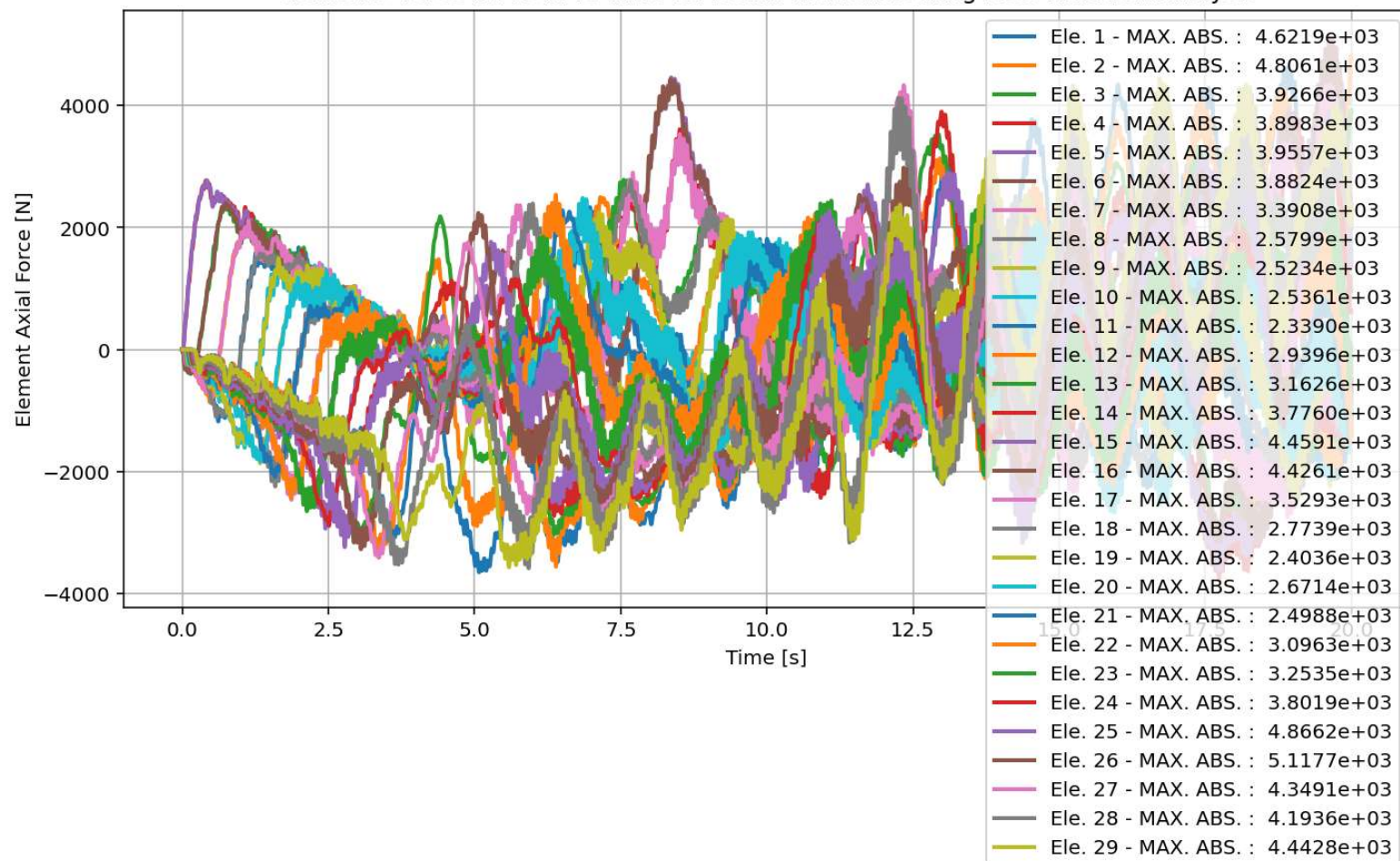
Inline Conda: anaconda3 (Python 3.12.7) LSP: Python Line 100, Col 45 UTF-8 CRLF RW Mem 45%

Base Reaction and Displacment of Structure During Free-vibration Analysis

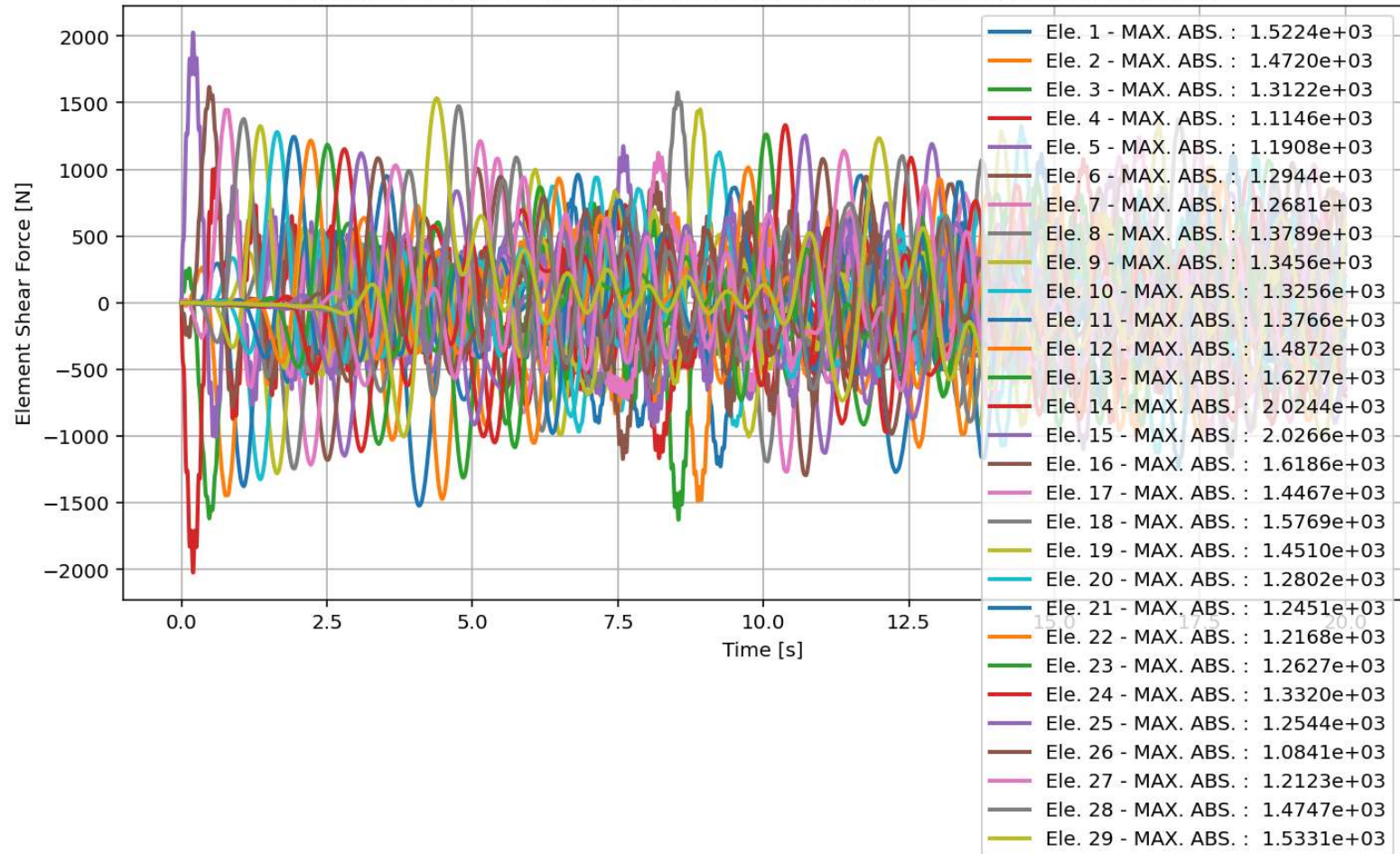


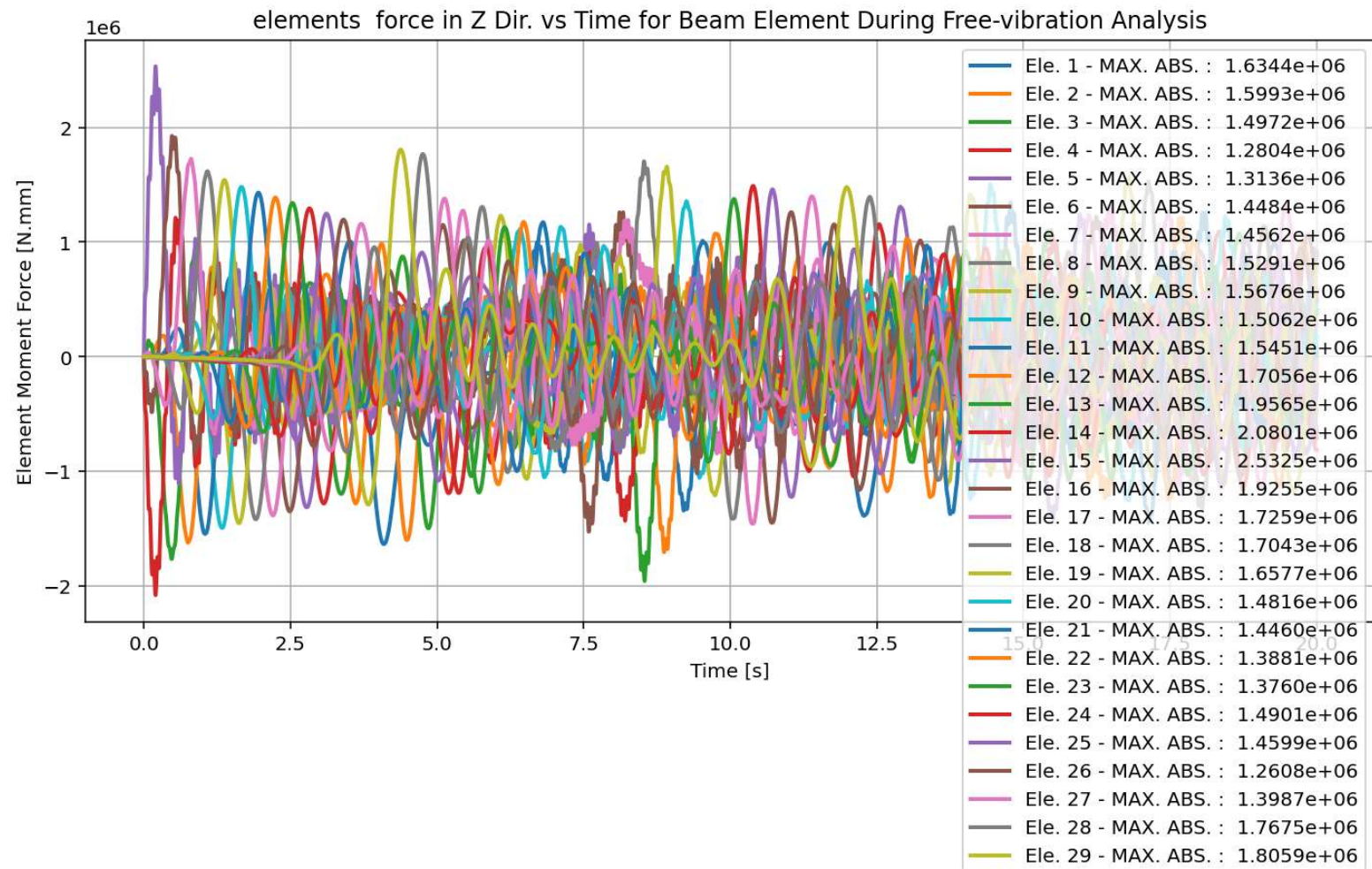


elements force in X Dir. vs Time for Beam Element During Free-vibration Analysis

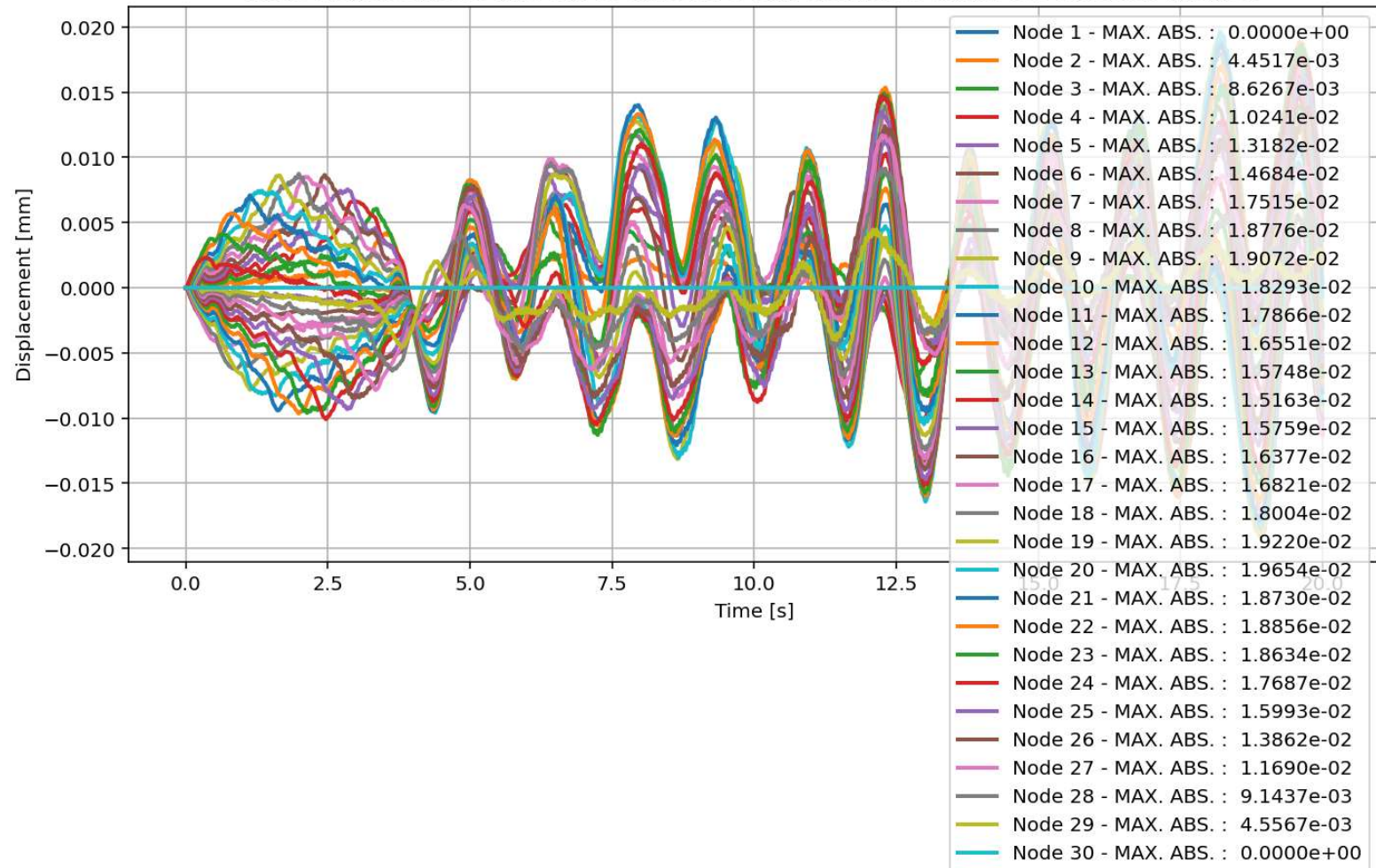


elements force in Y Dir. vs Time for Beam Element During Free-vibration Analysis

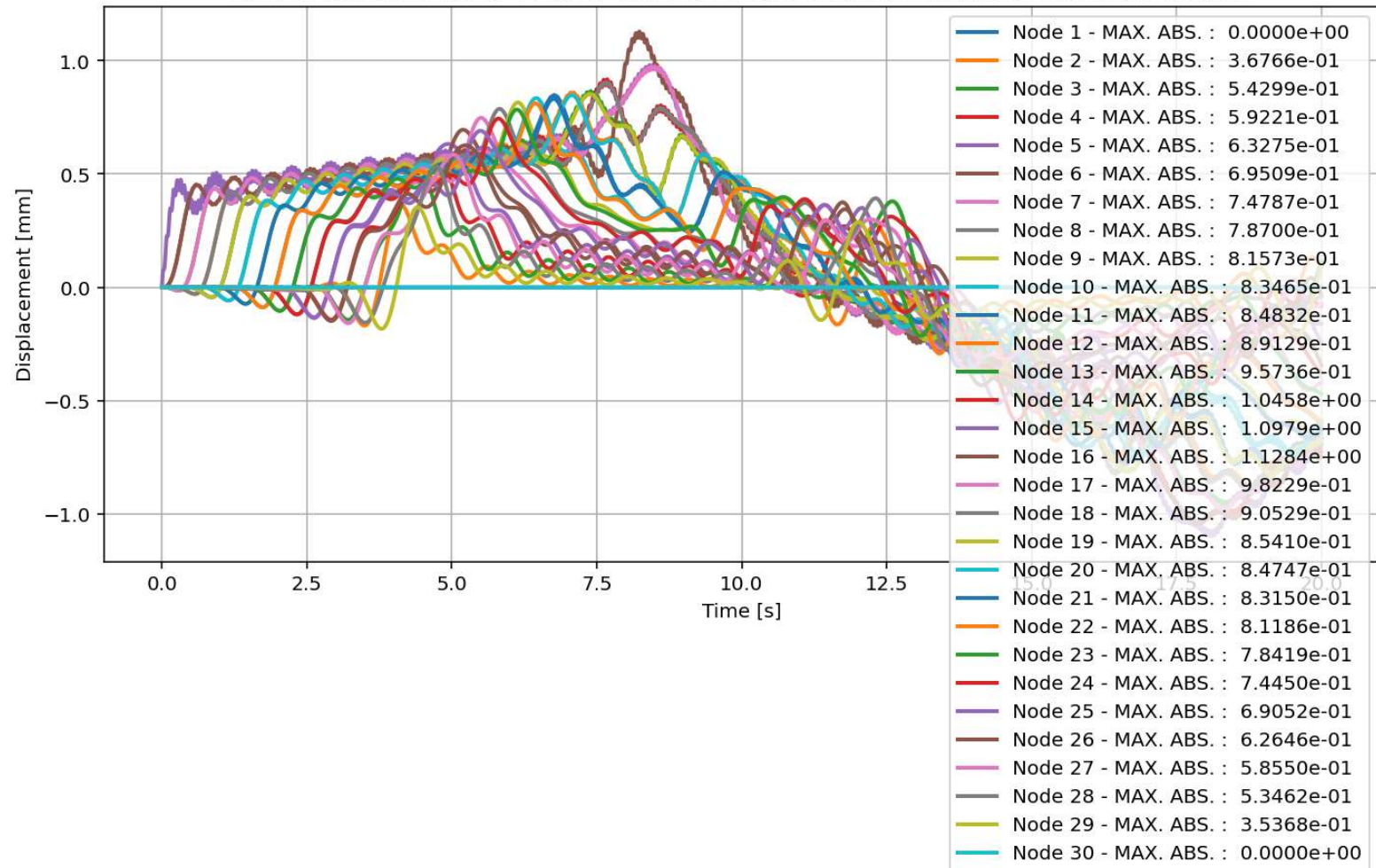




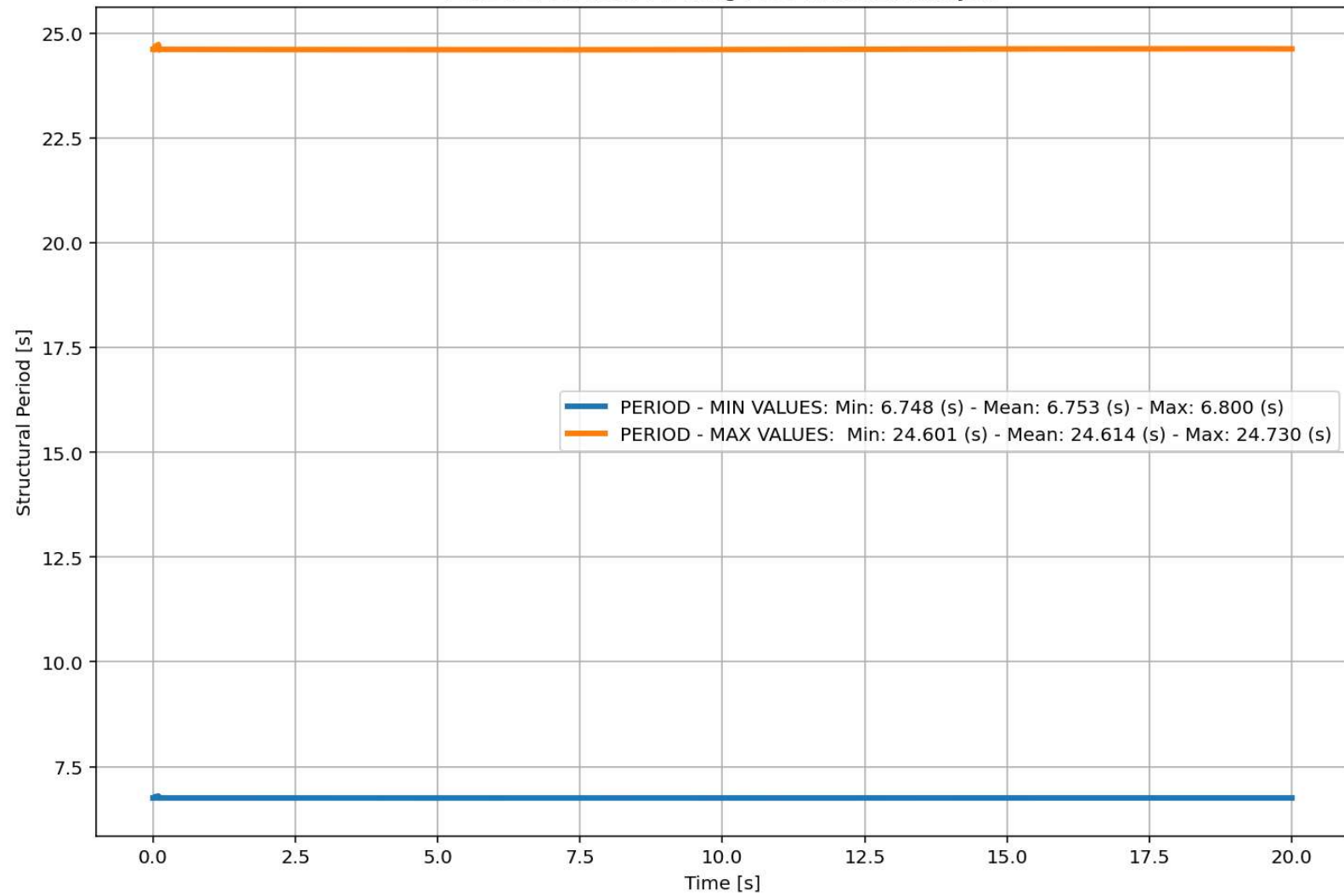
Node Displacements in X Dir. vs Time for Beam Element During Free-vibration Analysis



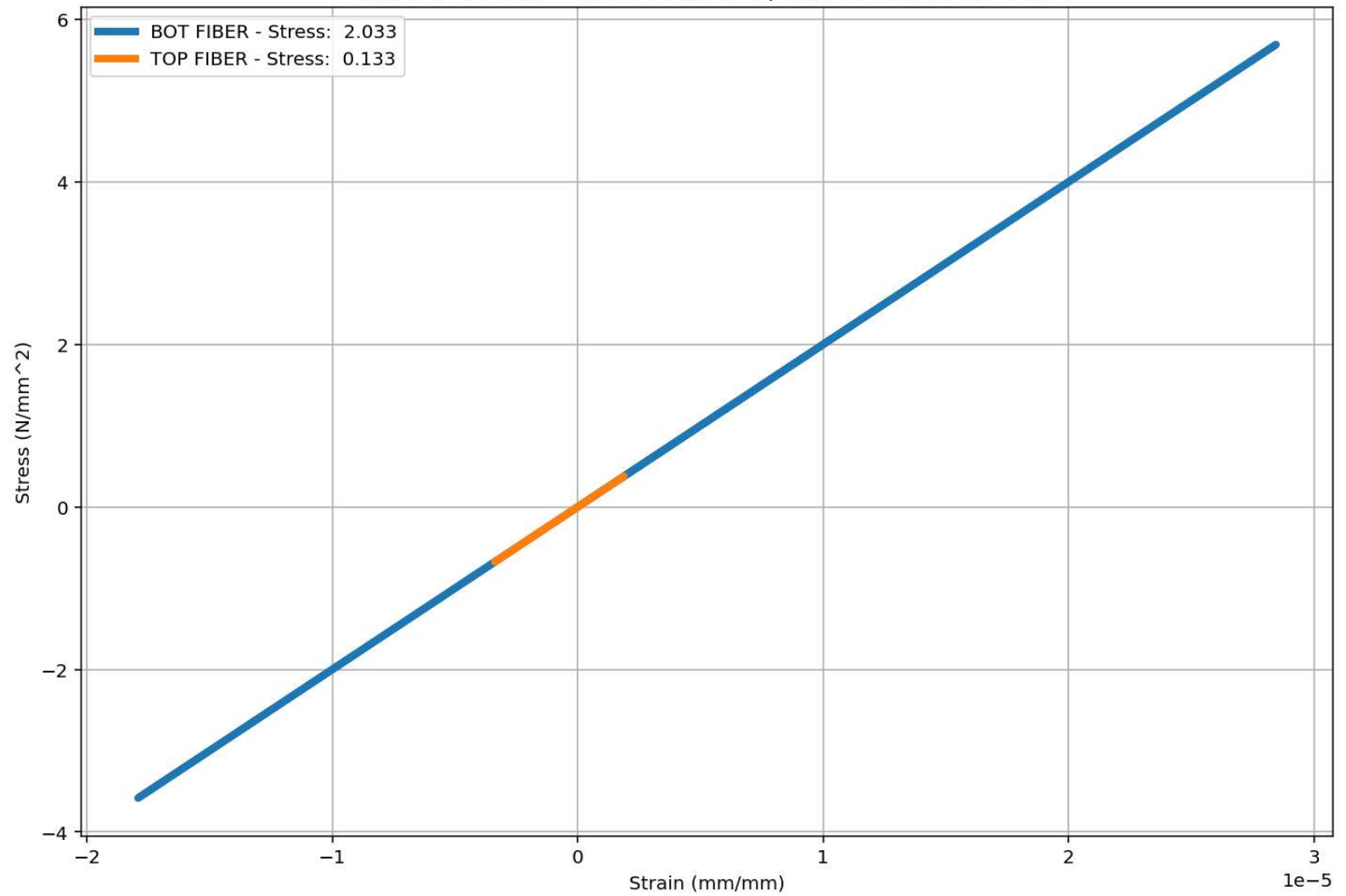
Node Displacements in Y Dir. vs Time for Beam Element During Free-vibration Analysis



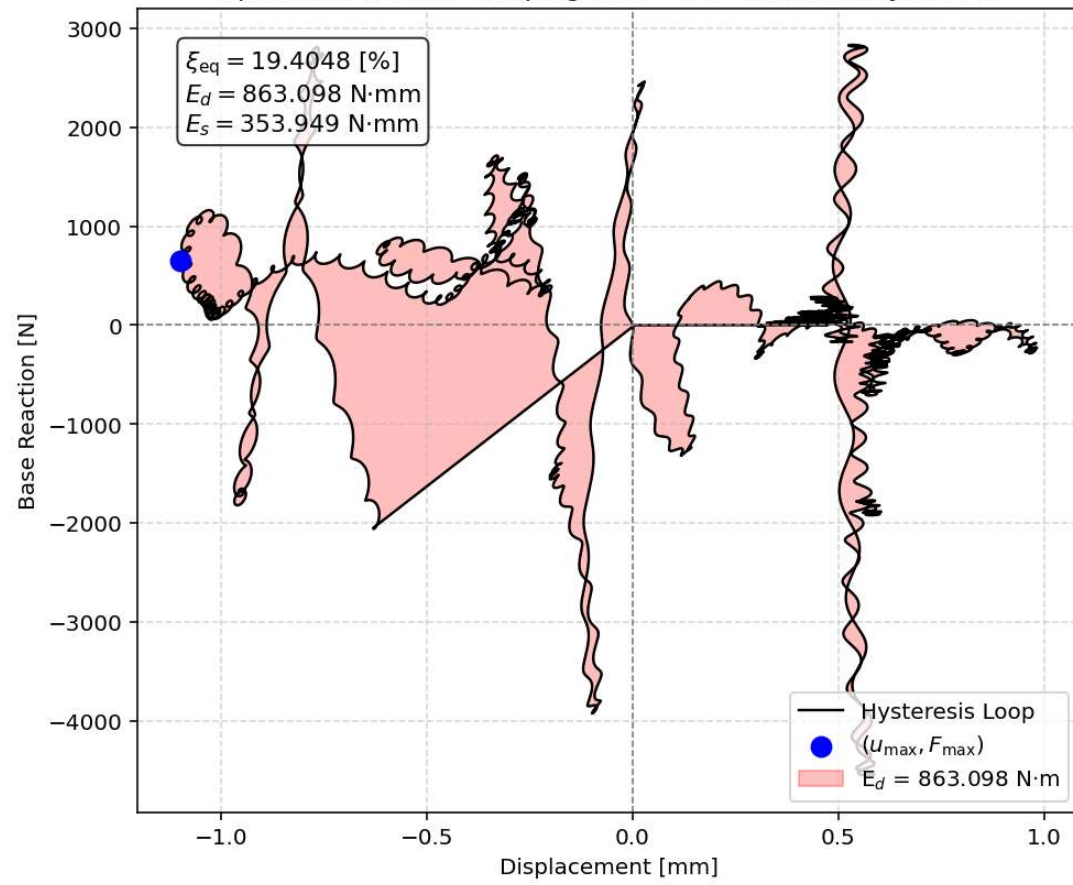
Period of Structure During Free-vibration Analysis

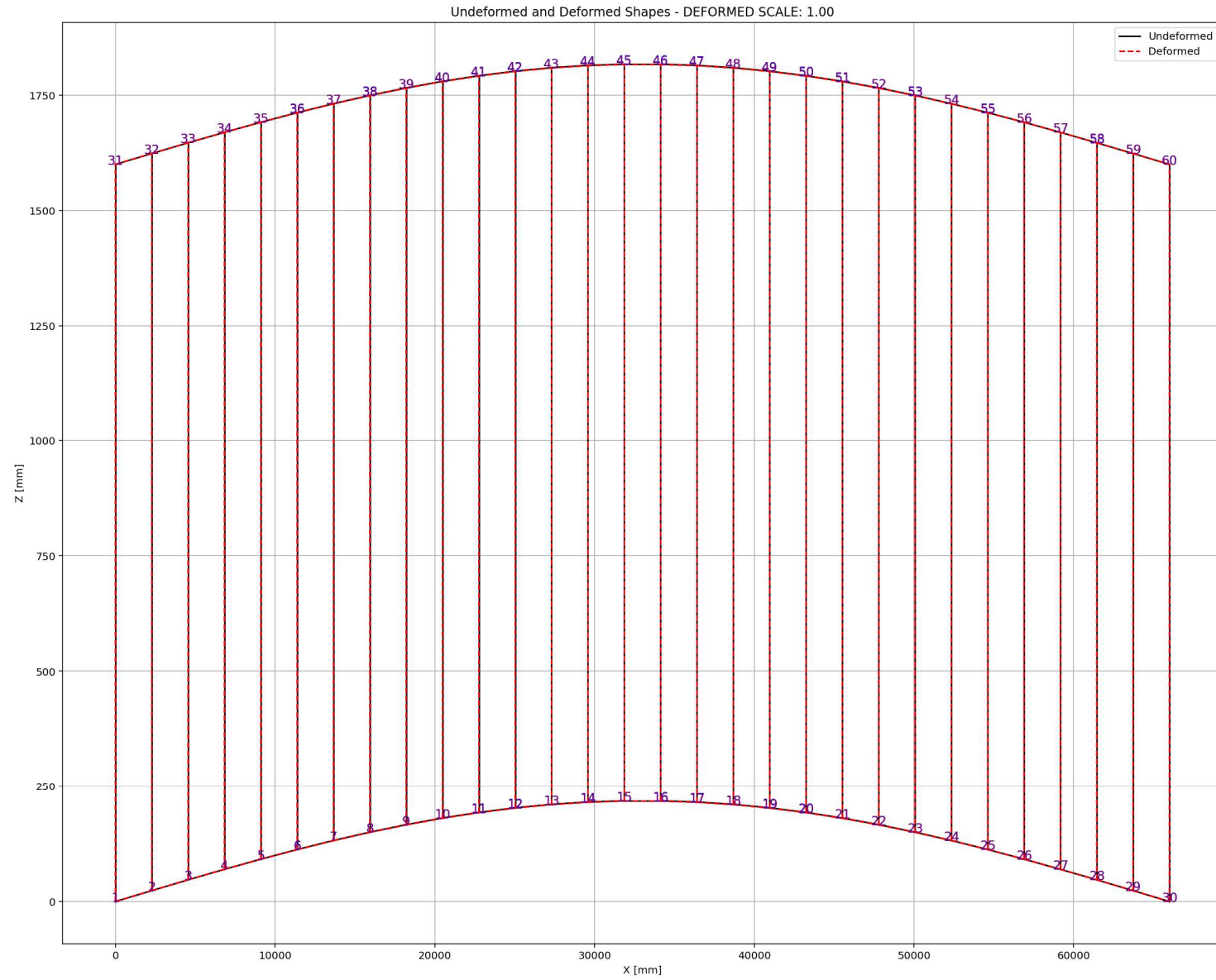


Behavior of beam - Stress-strain of Top and Bottom Fibers Curve



Equivalent viscous damping ratio - Free-vibration Hysteresis





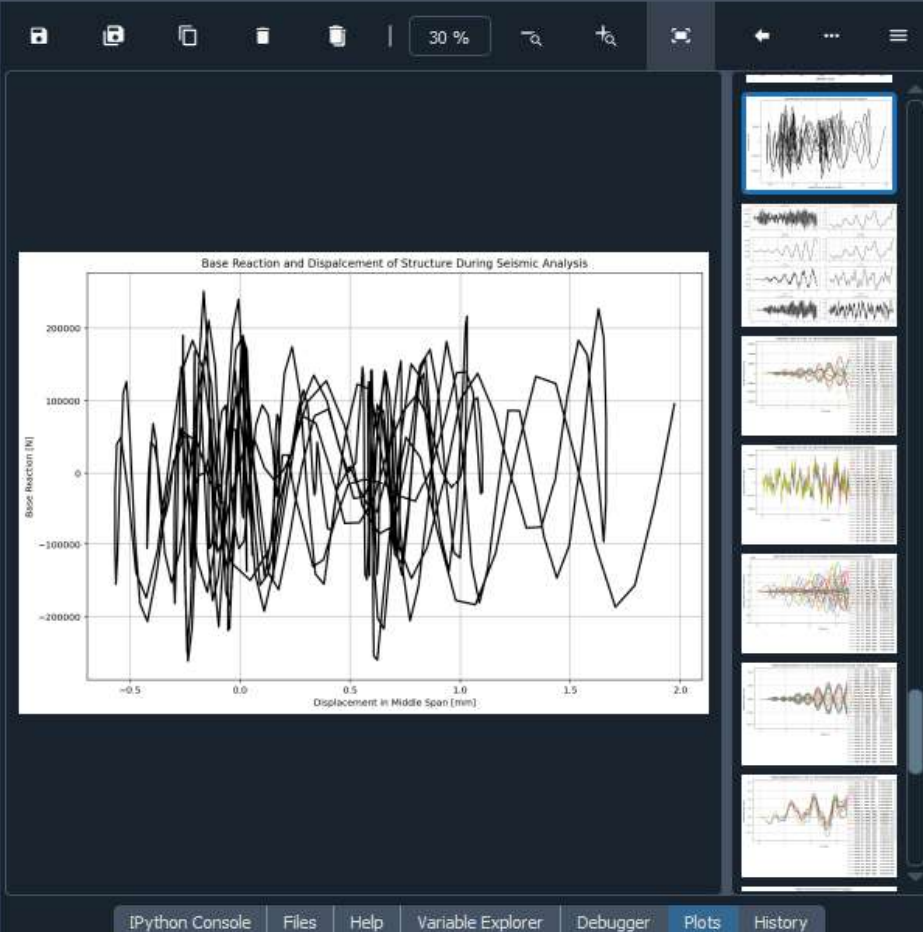
SEISMIC ANALYSIS

C:\Users\ DELL\Desktop\OPENSEES_FILES\HEIGHT_ANALY...(t)_PEDESTRIAN_BRIDGE_VIERENDEEL_GIRDERS_8_ANA+

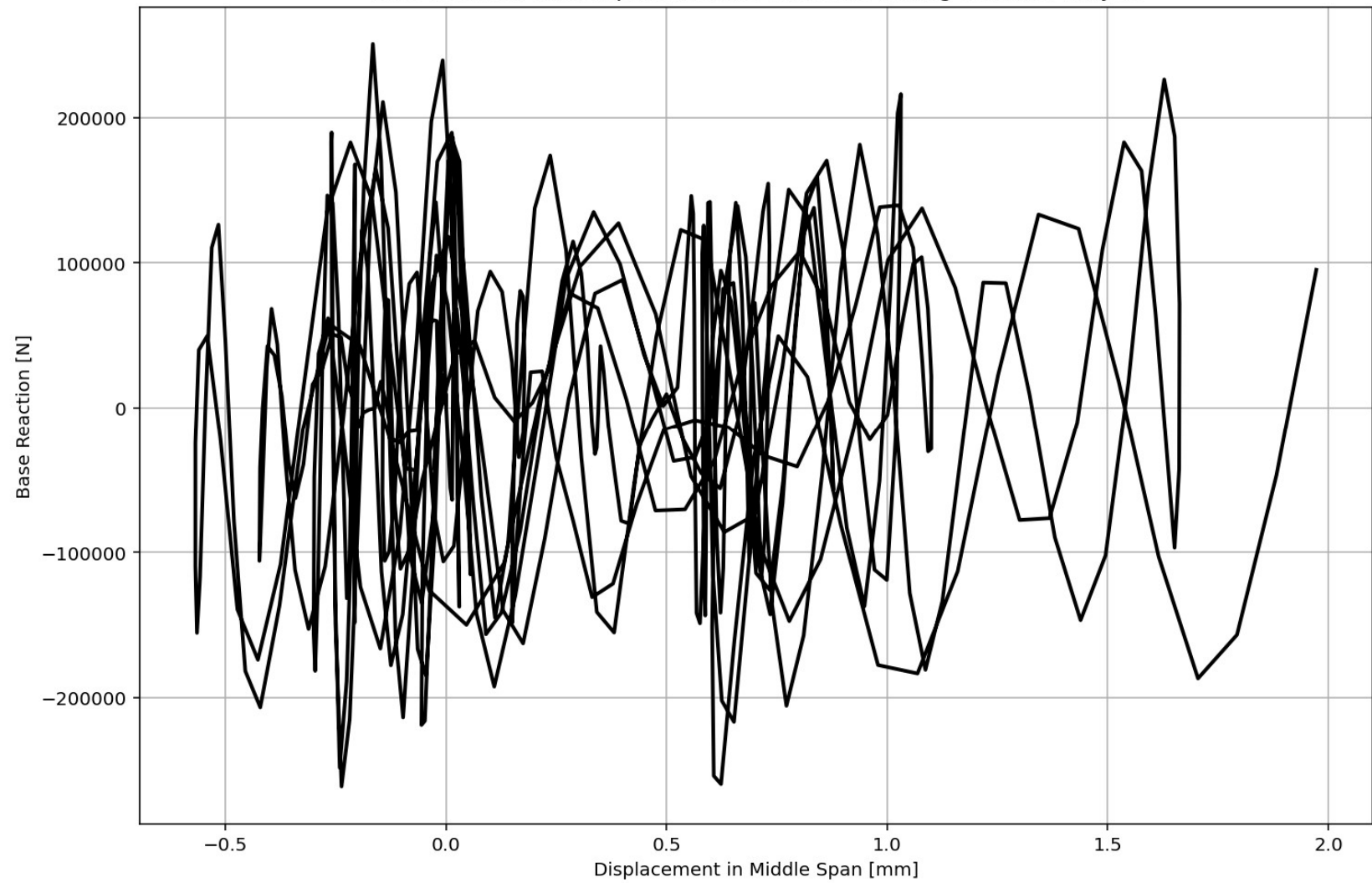
```

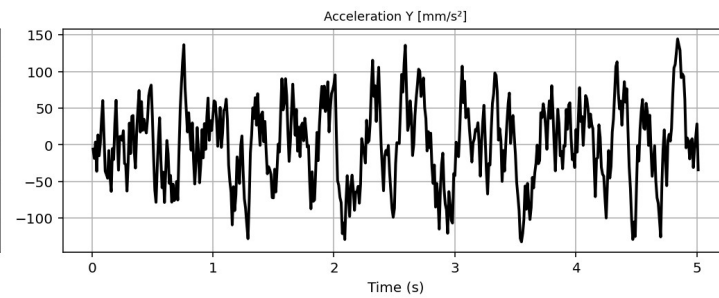
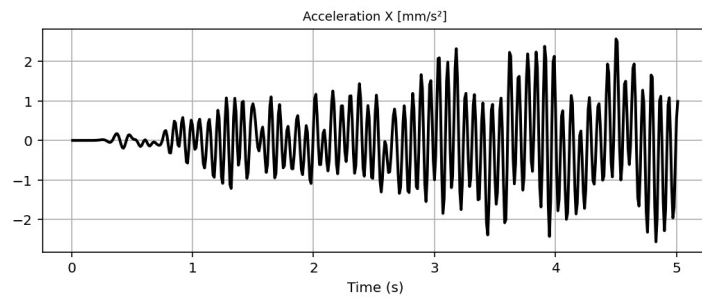
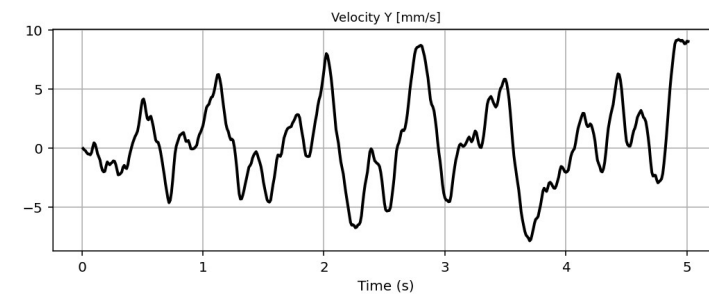
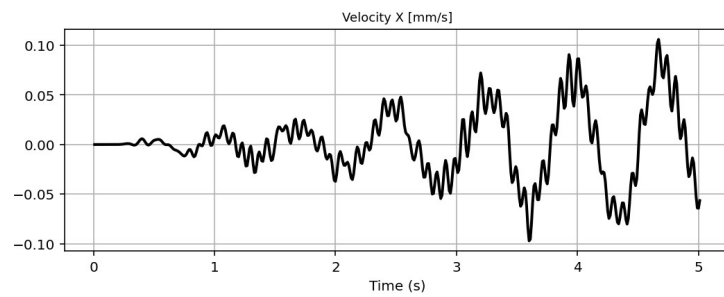
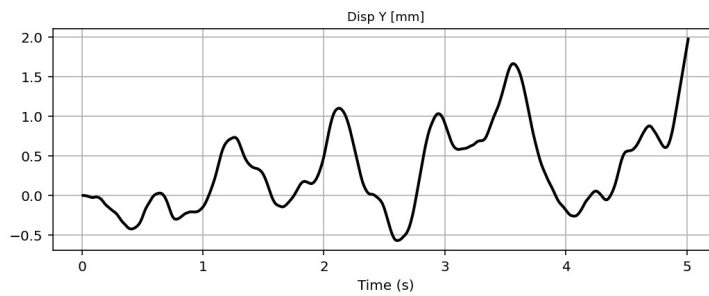
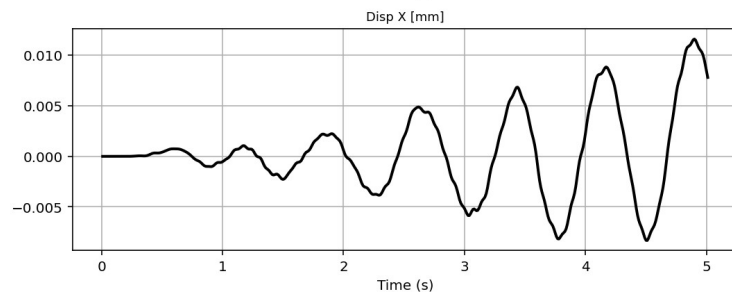
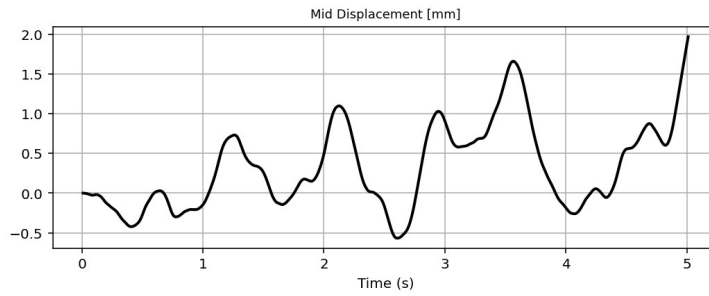
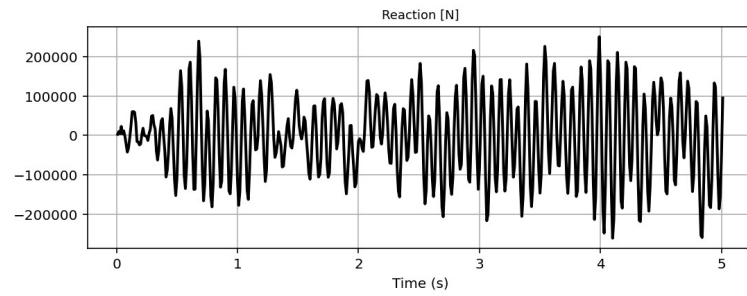
803
804 if ANAL_TYPE == 'SEISMIC': # DYNAMIC TIME-HISTORY ANALYSIS
805     ### DEFINE PARAMETERS FOR SEISMIC ANALYSIS
806     duration = 5.0           # [s] Analysis duration
807     dt = 0.01               # [s] Time step
808     GMfact = 9810           # [mm/s^2] standard acceleration of gravity or st
809     SSF_X = 0.5             # Seismic Acceleration Scale Factor in X Direct
810     SSF_Y = 0.5             # Seismic Acceleration Scale Factor in Y Direct
811     iv0_X = 0.005          # [mm/s] Initial velocity applied to the node
812     iv0_Y = 0.005          # [mm/s] Initial velocity applied to the node
813     st_iv0 = 0.0           # [s] Initial velocity applied starting time
814     SEI = 'Y'              # Seismic Direction
815     DR = 0.03              # Damping ratio
816
817 # Define time series for input motion (Acceleration time history)
818 if SEI == 'X':
819     SEISMIC_TAG_01 = 100
820     ops.timeSeries('Path', SEISMIC_TAG_01, '-dt', dt, '-filePath', 'Ground_Acceler
821     # Define load patterns
822     # pattern UniformExcitation $patternTag $dof -accel $tsTag <-vel0 $vel0> <-fac
823     ops.pattern('UniformExcitation', SEISMIC_TAG_01, 1, '-accel', SEISMIC_TAG_01,
824 if SEI == 'Y':
825     SEISMIC_TAG_02 = 200
826     ops.timeSeries('Path', SEISMIC_TAG_02, '-dt', dt, '-filePath', 'Ground_Acceler
827     ops.pattern('UniformExcitation', SEISMIC_TAG_02, 2, '-accel', SEISMIC_TAG_02,
828 if SEI == 'XY':
829     SEISMIC_TAG_01 = 100
830     ops.timeSeries('Path', SEISMIC_TAG_01, '-dt', dt, '-filePath', 'Ground_Acceler
831     # Define load patterns
832     # pattern UniformExcitation $patternTag $dof -accel $tsTag <-vel0 $vel0> <-fac
833     ops.pattern('UniformExcitation', SEISMIC_TAG_01, 1, '-accel', SEISMIC_TAG_01,
834     SEISMIC_TAG_02 = 200
835     ops.timeSeries('Path', SEISMIC_TAG_02, '-dt', dt, '-filePath', 'Ground_Acceler
836     ops.pattern('UniformExcitation', SEISMIC_TAG_02, 2, '-accel', SEISMIC_TAG_02,

```

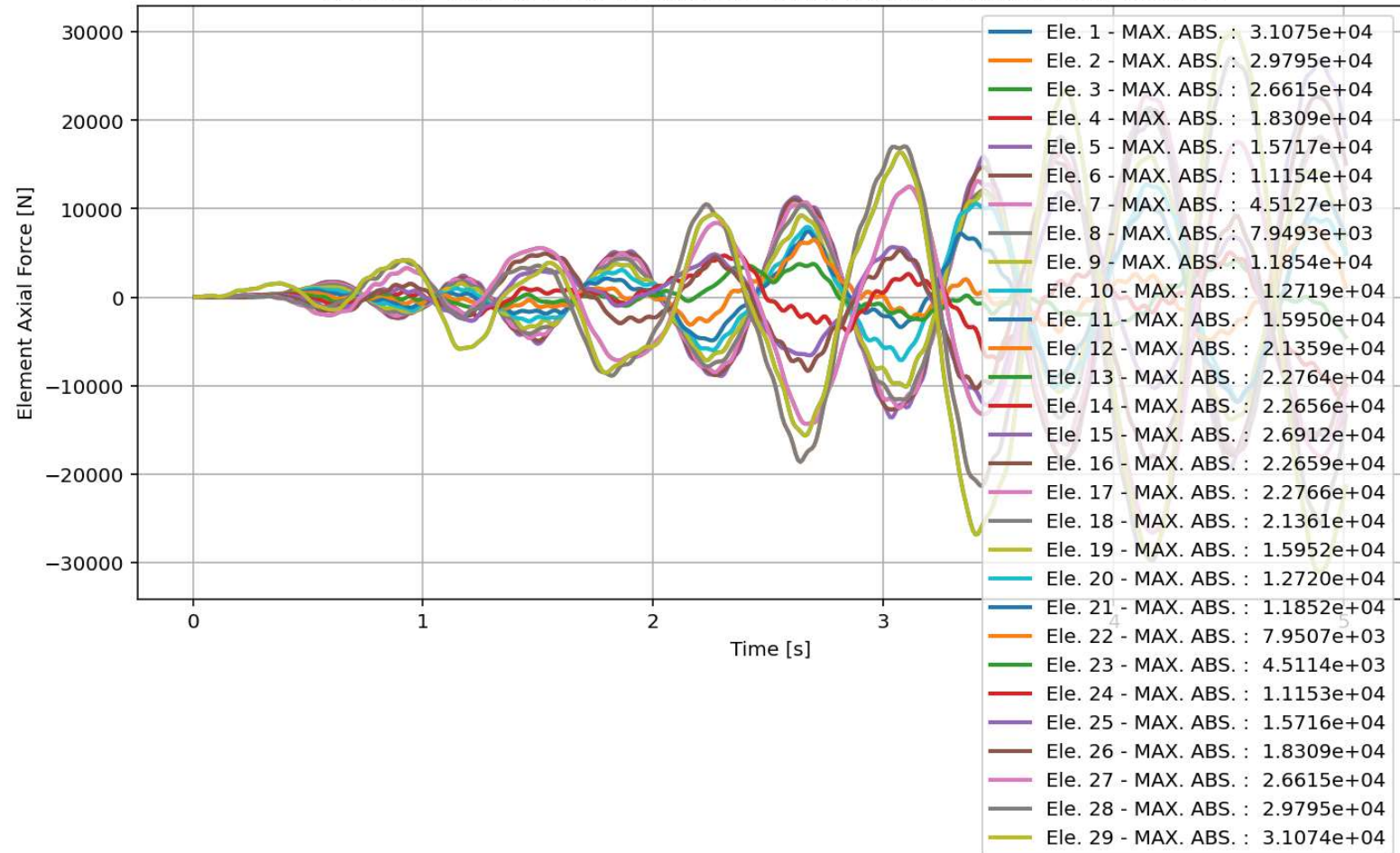


Base Reaction and Dispalcement of Structure During Seismic Analysis

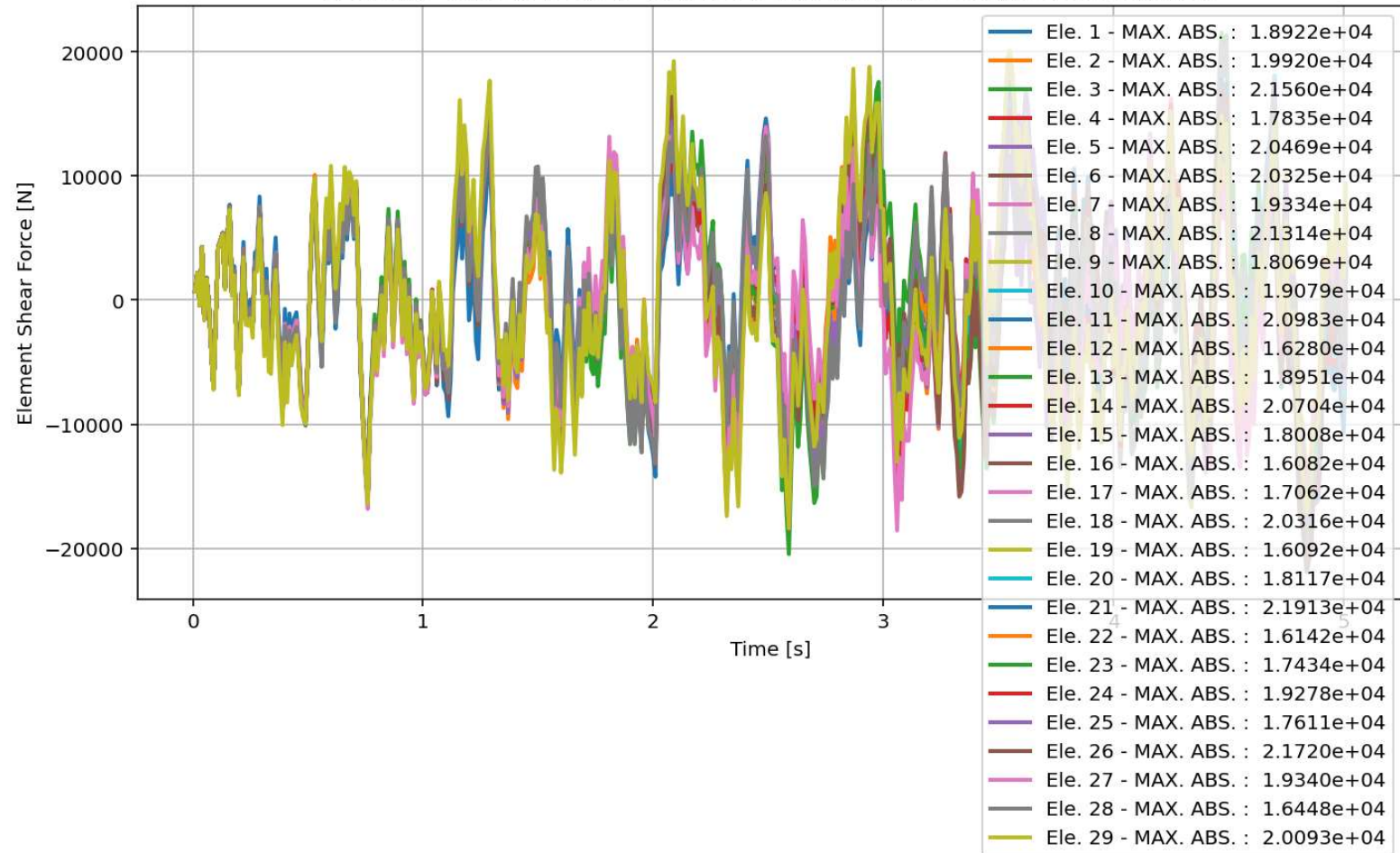


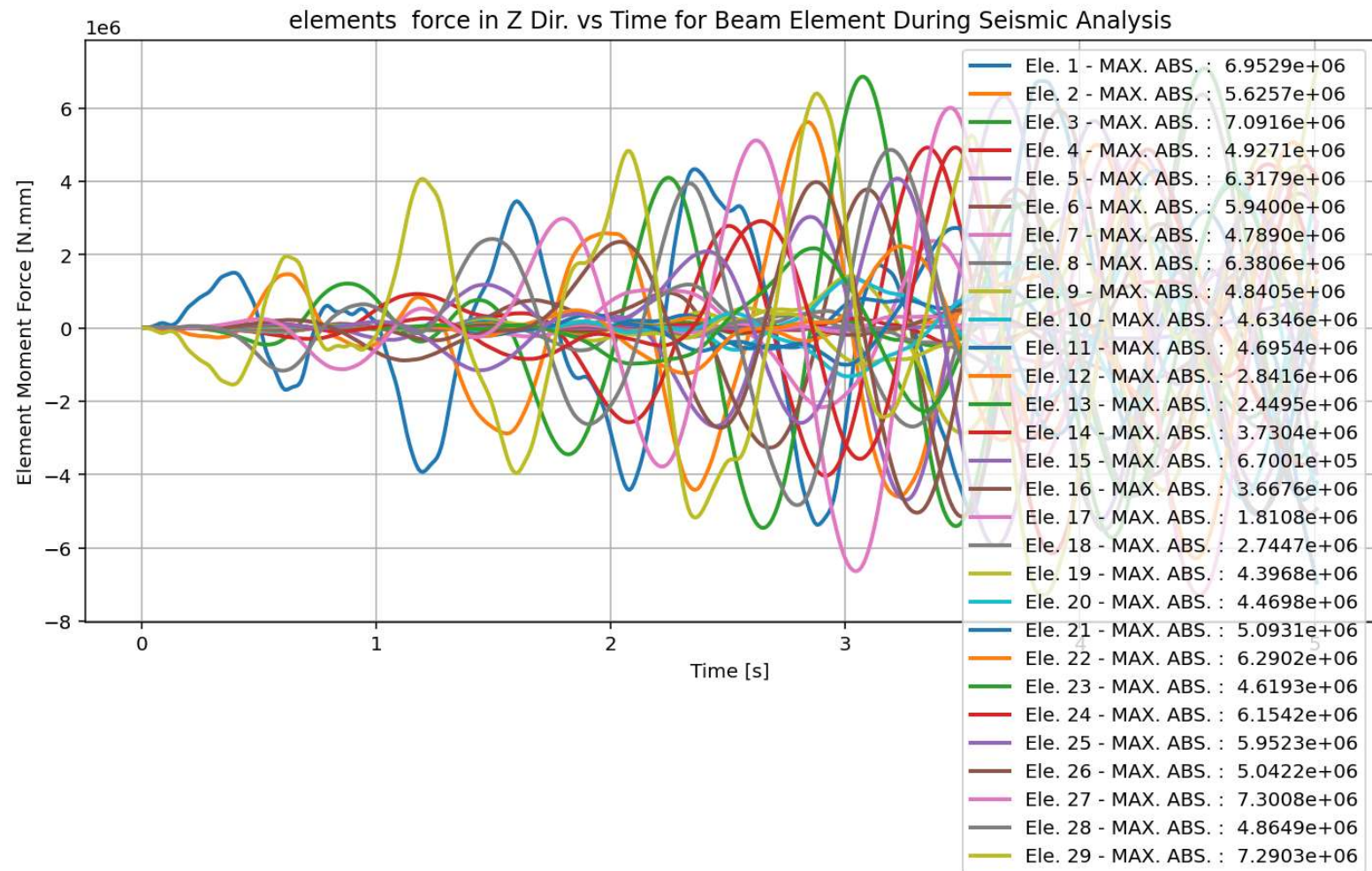


elements force in X Dir. vs Time for Beam Element During Seismic Analysis

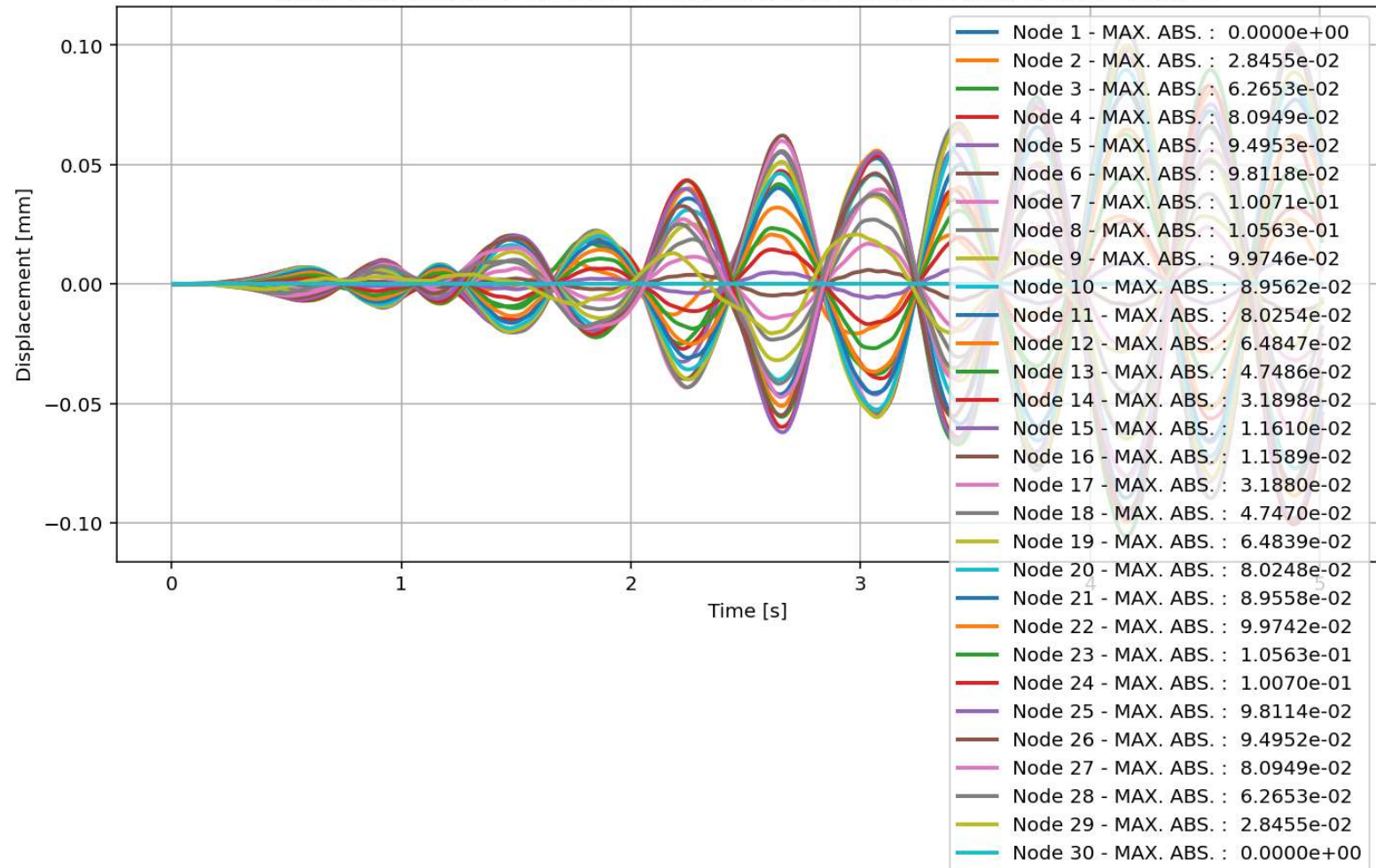


elements force in Y Dir. vs Time for Beam Element During Seismic Analysis

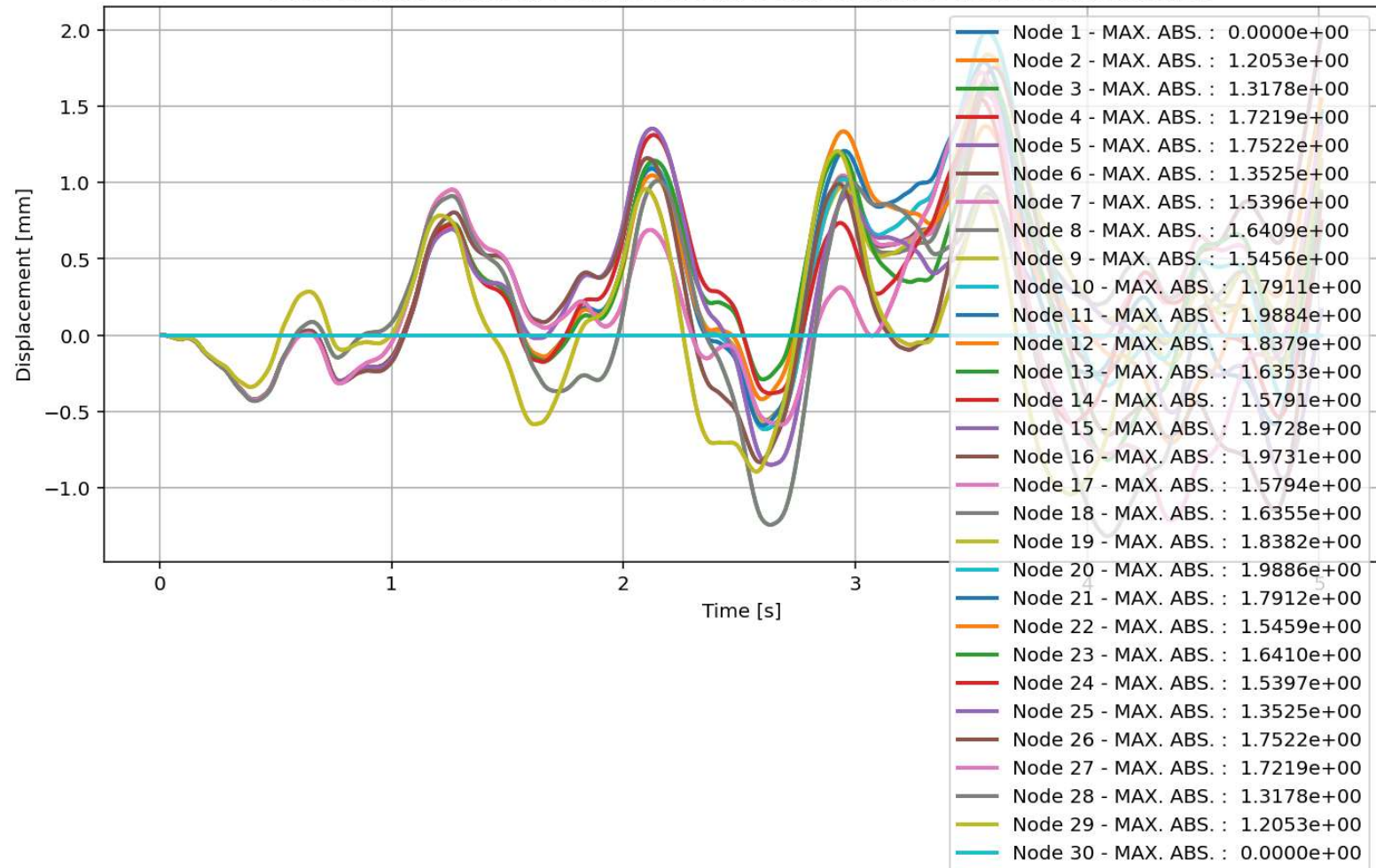




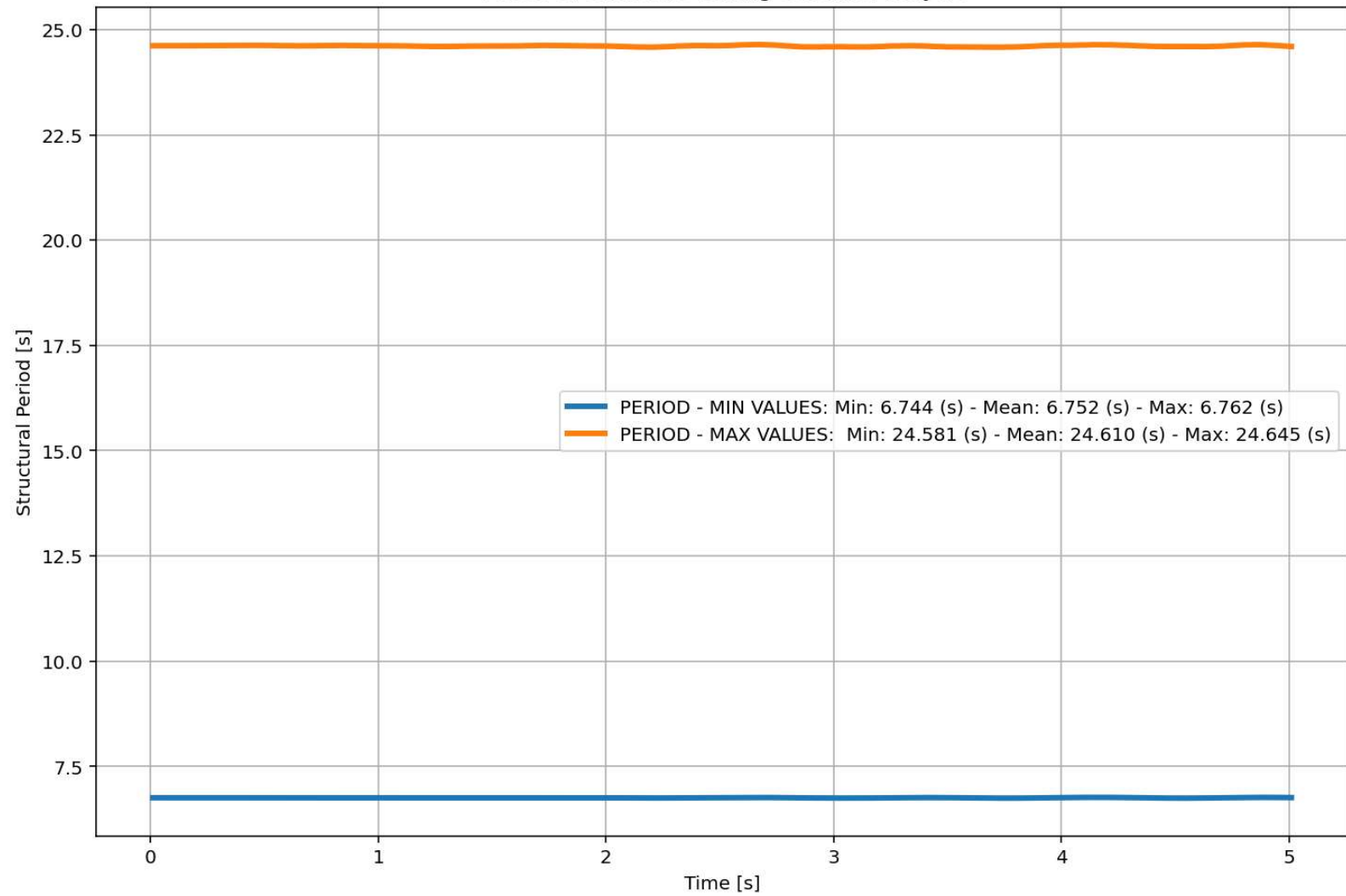
Node Displacements in X Dir. vs Time for Beam Element During Seismic Analysis



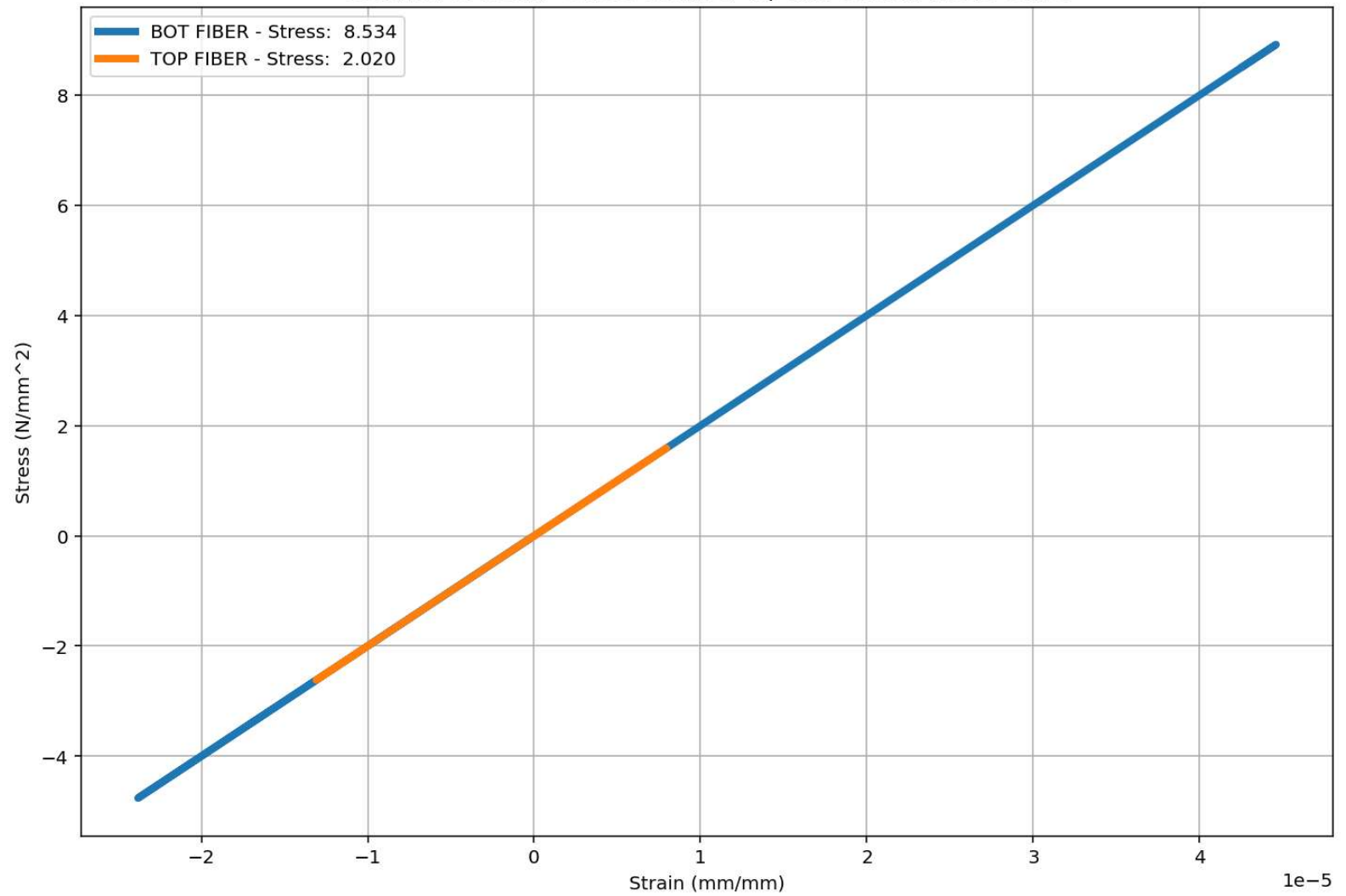
Node Displacements in Y Dir. vs Time for Beam Element During Seismic Analysis



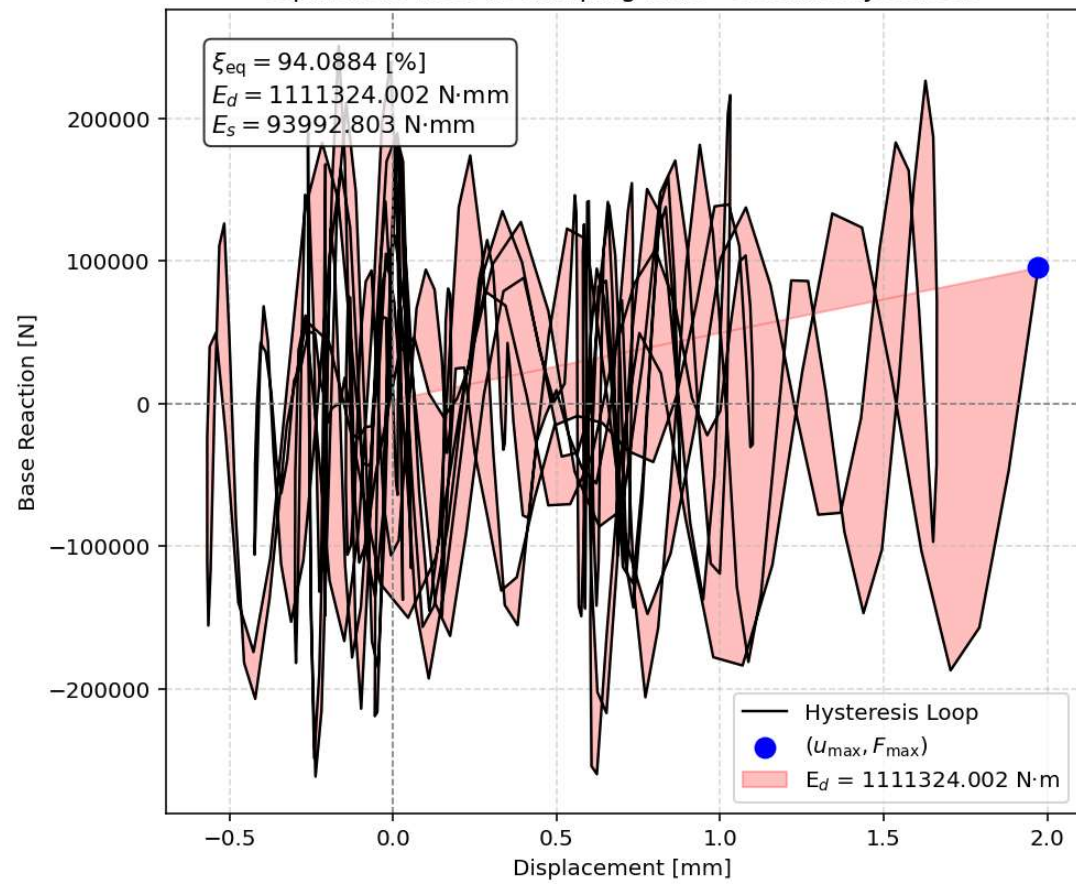
Period of Structure During Seismic Analysis

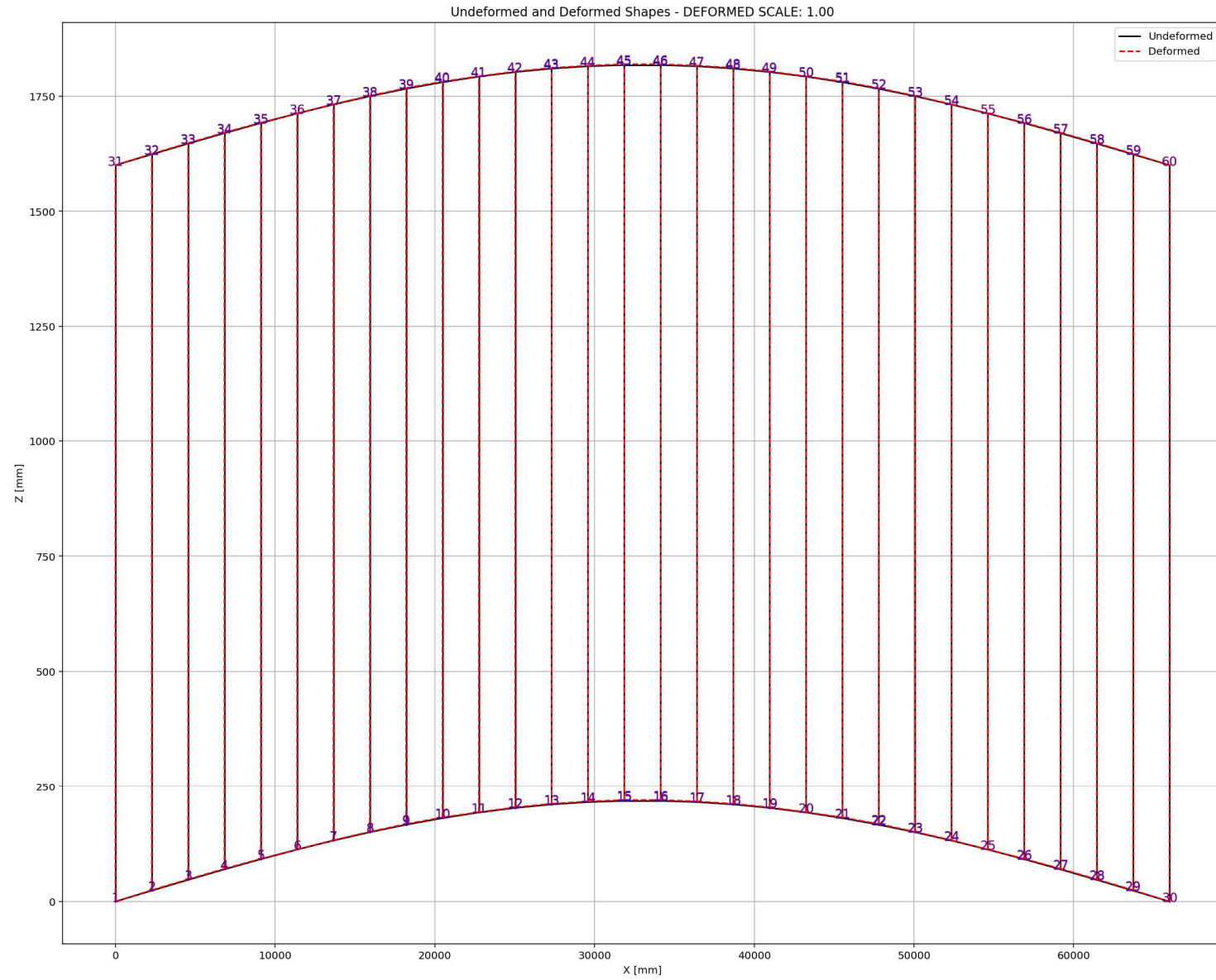


Behavior of beam - Stress-strain of Top and Bottom Fibers Curve

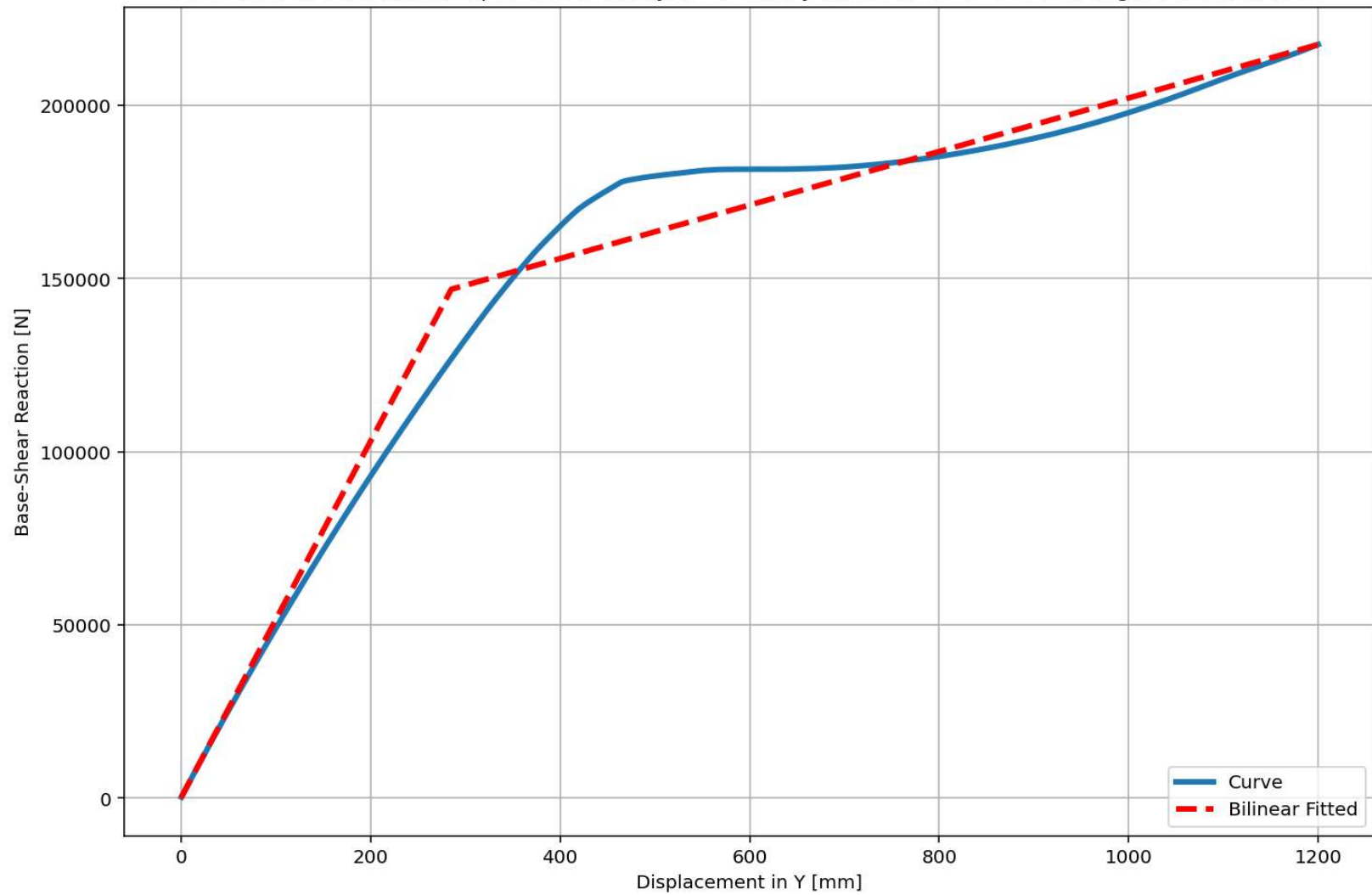


Equivalent viscous damping ratio - Seismic Hysteresis





Last Data of Base Axial-Displacement Analysis - Ductility Ratio: 4.2069 - Over Strength Factor: 1.4812



+=====+

= Analysis curve fitted =

Disp Base Shear

[[0. 0.]

[285.24327587 146849.7612848]

[1200. 217509.0131003]]

+=====+

+-----+

Structure Elastic Stiffness : 514.82

Structure Plastic Stiffness : 181.26

Structure Tangent Stiffness : 77.24

Structure Ductility Ratio : 4.21

Structure Over Strength Factor: 1.48

+-----+

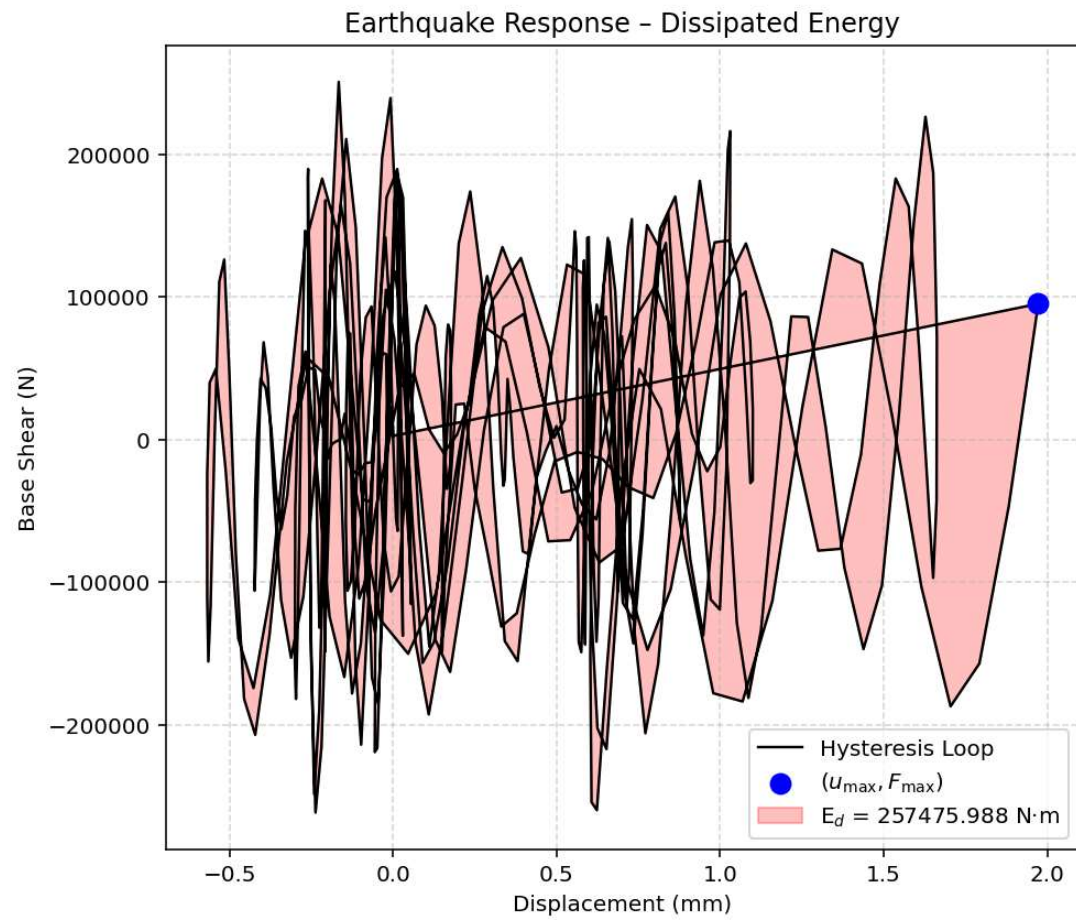
Over Strength Coefficient (Ω_0): 1.4812

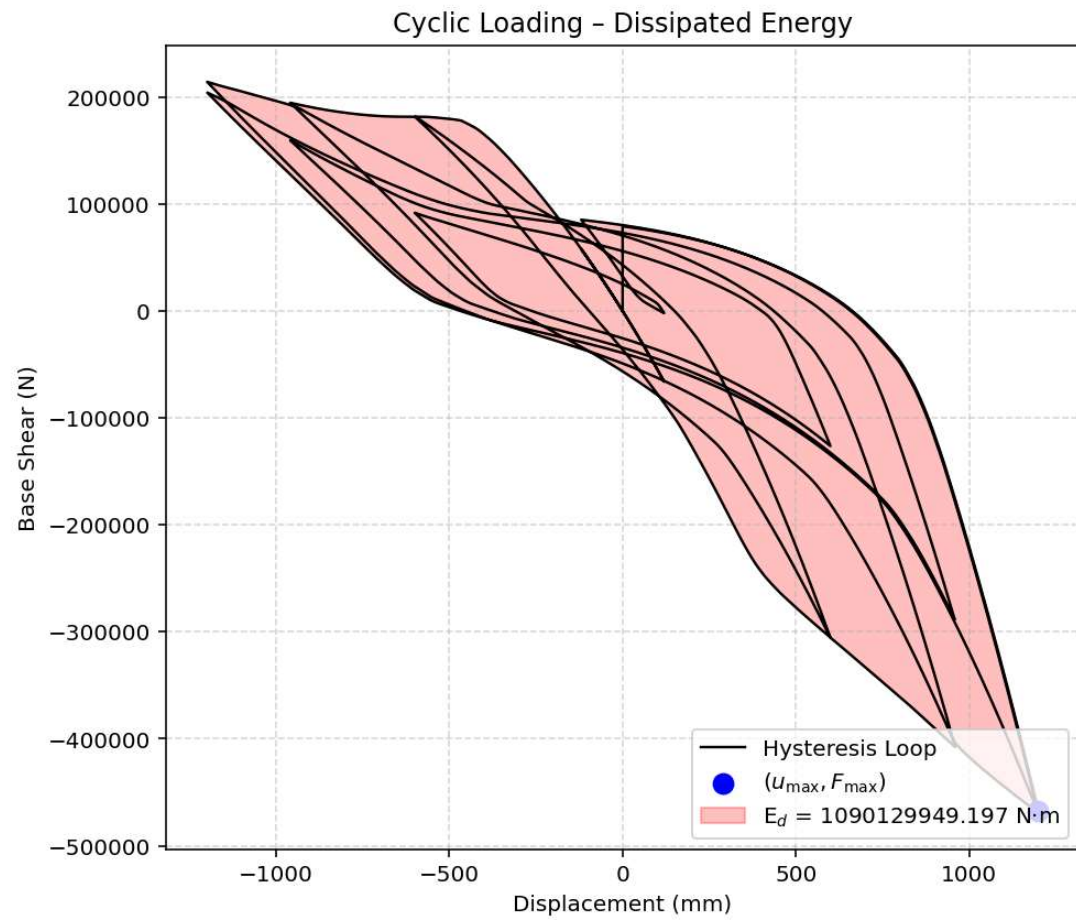
Displacement Ductility Ratio (μ): 4.2069

Ductility Coefficient ($R\mu$): 4.2069

Structural Behavior Coefficient (R): 6.2312

Structural Ductility Damage Index in Y Direction: -30.9668 (%)





Dissipated Energy from Earthquake= 257475.99 N·mm

Dissipated Energy from Cyclic Displacement= 1090129949.20 N·mm

DISSIPATED ENERGY CAPACITY INDEX: 0.024 [%]

ZONE 1: VERY MINOR DAMAGE

